OFFICIAL OMNICODE SYSTEM MANUAL

Volume 06: Track 5 - Infrastructure, Operations & Archives Manual

Systems Architecture, Logic & Algebra, CI/CD, Containerization & Operations

> ***Foundational Scripture Anchor:*** *Proverbs 3:19 (KJV) - The LORD by wisdom hath founded the earth*

Infrastructure engineering: logic and algebraic specifications, four-block architectures, autonomous restorative agents, build toolchains, and deployment archives.

SYSTEM TRACK
<TRACK 5> : {Infrastructure, Operations & Archives}

TOTAL VOLUME CHAPTERS
2 Chapters

CATALOGED SOURCE FILES
29 files (4,662 lines)

TOTAL WORD COUNT
23,471 words

VOLUME CHAPTERS CATALOG

CHAPTER	TITLE	SUBSYSTEM COVERAGE
CH 23	Chapter 23: Track 5 - Systems Architecture & Balanced Logic	Coordinate Computation, Execution Pipelines, 4-Block Architecture & Quad-State Loops
CH 24	Chapter 24: Track 5 - Cognitive Systems, Infrastructure & CI/CD	Neural-Symbolic Feedback, Autonomous Restorator, Build Scripts & DNS Infrastructure

VOLUME 06: TRACK 5 - INFRASTRUCTURE, OPERATIONS & ARCHIVES MANUAL

Chapter 23: Track 5 - Systems Architecture & Balanced Logic

Coordinate Computation, Execution Pipelines, 4-Block Architecture & Quad-State Loops

> ***Witness:*** *Proverbs 3:19 (KJV) - The LORD by wisdom hath founded the earth*

SYSTEM TRACK
<TRACK 5> : {Infrastructure, Operations & Archives}

CHAPTER IDENTIFIER
CH 23

NOTEBOOKLM WORD COUNT
4,613 words

CATALOGED CHAPTER FILES
6 files (620 lines)

RECOMMENDED NOTEBOOKLM COGNITIVE QUERY

"Explain the architectural synergy between the 4-block layout and balanced ternary arithmetic."

CHAPTER MANIFEST & INDEX (6 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 5> : {Infrastructure, Operations & Archives}/docs/03_architecture/01_coordinate_system_of_computation.md**	MD	39	212	1.7
2	**<TRACK 5> : {Infrastructure, Operations & Archives}/docs/03_architecture/02_execution_pipeline.md**	MD	56	507	3.9
3	**<TRACK 5> : {Infrastructure, Operations & Archives}/docs/03_architecture/03_four_block_architecture.md**	MD	134	739	6.4
4	**<TRACK 5> : {Infrastructure, Operations & Archives}/docs/02_logic_and_algebra/01_balanced_ternary_quad_state.md**	MD	76	536	3.3
5	**<TRACK 5> : {Infrastructure, Operations & Archives}/docs/02_logic_and_algebra/02_relational_connectives_and_operators.md**	MD	56	365	2.2
6	**<TRACK 5> : {Infrastructure, Operations & Archives}/docs/02_logic_and_algebra/03_the_quad_state_accumulator_loop_and_named_registers.md**	MD	259	2,254	17.9

<TRACK 5> : {Infrastructure, Operations & Archives}/docs/03_architecture/01_coordinate_system_of_computation.md

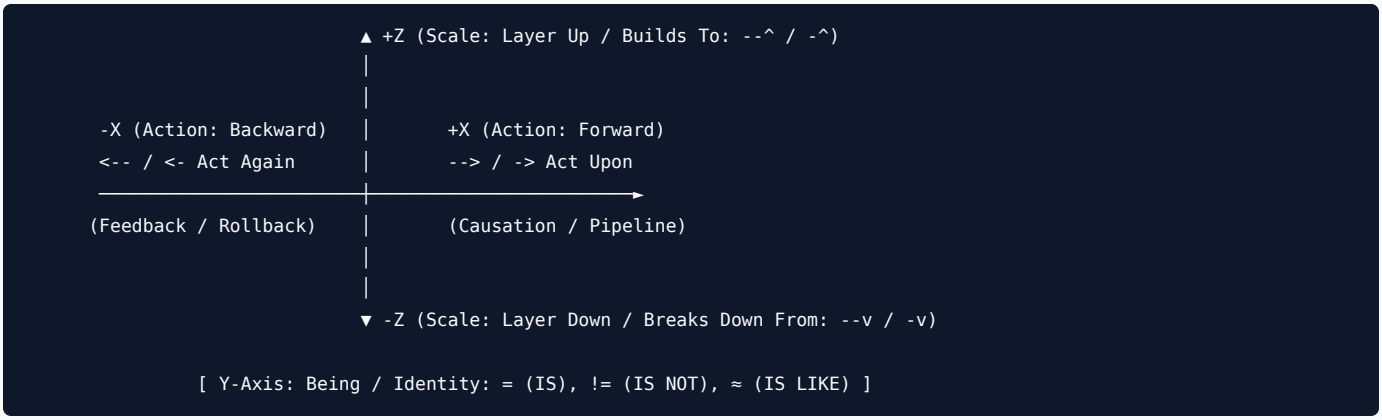
MARKDOWN / DOC | 39 LINES | 212 WORDS | 1.7 KB

Volume 3: Architecture & Execution

Document 01: The 3D Coordinate System of Computation

1. The Three Orthogonal Axes

Every computational entity, transformation, and expression in OmniCode is positioned within a 3-dimensional coordinate space:



2. The Orthogonal Dimensions

1. **The X-Axis: Temporal / Causal Vector (`-->` / `->` vs `<--` / `<-`)**
2. Governs forward causation vs. retroactive feedback/unwinding.
3. Interacts directly with the Quad-State engine: `+1` moves along $+X$, `-1` moves along $-X$.
4. **The Y-Axis: Ontological Identity (`=` , `!=` , `≈`)**
5. Governs equivalence and relation of being in state space.
6. **The Z-Axis: Abstraction Scale (`--^` / `-^` vs `--v` / `-v`)**
7. Governs the movement between low-level hardware representation and high-level conceptual modeling.
8. `--^` (shorthand `-^`) constructs high-level types from low-level memory layouts.
9. `--v` (shorthand `-v`) projects high-level models down to machine primitives.

`<TRACK 5> : {Infrastructure, Operations & Archives}/docs/03_architecture/02_execution_pipeline.md` MARKDOWN / DOC | 56 LINES | 507 WORDS | 3.9 KB

Volume 3: Architecture & Execution

Document 02: The Unified Execution Pipeline

1. Unified Architecture Across Three Execution Modes

OmniCode allows the exact same code to be:

1. **Compiled (AOT Native):** Ahead-of-time compilation via LLVM or assembler to standalone native binaries.
2. **Interpreted:** Real-time in-memory execution via AST/bytecode evaluation for interactive REPL and hot-reloading.
3. **Transpiled:** Source-to-source emission of ISO C99, C++, or WebAssembly for universal deployment.

```
flowchart TD
    Source["OmniCode Source (.omni)"] --> Lexer["Lexer (Boundary & Directional Tokenizer)"]
    Lexer --> Parser["Ontological Parser"]
    Parser --> AST["Abstract Syntax Tree with First-Class Asides"]
    AST --> Semantics["Ontological Type & Context Verifier"]
    Semantics --> HIR["High-Level Intermediate Representation"]

    HIR -->|Mode 1: Interpreted| Evaluator["Quad-State VM / Evaluator (Immediate REPL)"]
    HIR -->|Mode 2: Compiled| LLVMBackend["LLVM IR Generator -> Native ELF/PE/Mach-O"]
    HIR -->|Mode 3: Transpiled| CEmitter["C99 / WASM Emitter (Portable Source)"]
```

2. Role of the Boundary Elements in the Pipeline

- **Denotation `[...]` and Invariant `<...>` :** Directly mapped to data structures, memory definitions, and logic bodies in the target backend.
- **Connotation `{...}` , `{...]` , and `[...}` :** Injected into the semantic checker to establish safety constraints, stack/heap qualifiers, and variable resolution rules.
- **Aside `(...)` :** Maintained in debug tables, emitted as formatted doc-comments in transpiled code, and evaluated by the verifier for formal invariant validation.
- **Quad-State Engine (`-1, 0, 1, NULL`):**
- In the interpreter: Drives the runtime state machine directly.
- In native C/LLVM compilation: Translated to integer return codes, branch vectors, and zero-cost error frames.

3. The Interactive Terminal Environment (REPL): Why It Is Essential

The interactive terminal environment (`omnicode` / `cargo run -- repl`) is not an afterthought; it is an essential foundational pillar of OmniCode's early development:

1. **Immediate Feedback Loop for Unprecedented Physics:** OmniCode introduces concepts absent in traditional C/Python/Rust languages: balanced ternary logic ($\mathbb{T}_4 = {-1, 0, +1, \text{NULL}}$), morphological boundary closures (`[...]` , `<...>` , `{...}` , `{...]` , `[...}`), and 3D coordinate vectors (`-->` , `<--` , `^--` , `v--`). Requiring developers to write full 4-block files and compile them AOT just to understand how a caveat (`!`) suppresses a transaction introduces excessive friction. The terminal environment gives sub-millisecond feedback for every single delimiter and operator.
2. **Real-time Manifestation of the Interpreted Pillar:** By establishing an interactive in-memory evaluator from day one, we guarantee that the grammar and AST definitions are cleanly decoupled from low-level LLVM code generation. The interpreter proves that the AST contains all necessary semantic and ontological data to execute autonomously.
3. **Living Failpoint & Transaction Rollback Observatory:** In conventional languages, runtime failures manifest as unhandled exceptions or fatal panics. In OmniCode, transitions into `-1` (Rollback) or `NULL` (Halt) are first-class ontological occurrences. The terminal environment acts as a living observatory, logging microsecond timestamps, tracking transaction snapshot unwindings, and visually displaying failpoints without crashing the session.
4. **Human-AI Co-Reasoning Sandbox:** Because OmniCode features first-class Asides `(...)` designed for machine-readable intent, the interactive terminal provides an ideal conversational scratchpad where human developers and AI coding agents can interactively enter

expressions, verify boundary contracts, and inspect state bindings before committing logic to permanent source repositories.

Volume 3: Architecture & Execution

Document 03: The Universal 4-Block Architecture & The Rust Foundation

1. The Universal 4-Block Lifecycle

Every code file in OmniCode—regardless of whether it is an `.omni` source file or a compiler module—adheres to a strict **4-Block Lifecycle Protocol**:

```
┌──────────────────────────────────────────────────────┐
│               BLOCK 1: METADATA BLOCK                │
│   (Module Identity, Invariant Axioms, Semantics, Asides)│
├──────────────────────────────────────────────────────┤
│               BLOCK 2: OPENING BLOCK                 │
│   (Context Setup, Preconditions, Scoped Declarations)  │
├──────────────────────────────────────────────────────┤
│               BLOCK 3: BODY BLOCK                    │
│   (Core Transformations, Causality Vectors, Logic)     │
├──────────────────────────────────────────────────────┤
│               BLOCK 4: CLOSING BLOCK                 │
│   (Postconditions, Invariant Integrity Proofs, Commit) │
└──────────────────────────────────────────────────────┘
```

A. Block 1: Metadata Block

- **Role:** Establishes the ontological identity of the module, licensing, semantic dependencies, and invariant axioms.
- **Grammar Elements:** `Module < Name > ( Intent Aside ) { ... }`, Invariants `< ... >`.

B. Block 2: Opening Block

- **Role:** Sets up the execution environment, verifies preconditions, initializes state instances, and binds contextual universes.
- **Grammar Elements:** `Opening { requires: [ ... ], preconditions: [ ... ] }`, Instances `[ ... ]`.

C. Block 3: Body Block

- **Role:** Implements the operational logic, directional pipelines, being relations, and caveat constraints.
- **Grammar Elements:** `Body [ source --> target ! caveat ]`, Vectors `-->`, `<--`, `^--`, `v--`.

D. Block 4: Closing Block

- **Role:** Verifies postconditions, proves invariant satisfaction, handles rollback teardown, and yields the final commit QuadState.
- **Grammar Elements:** `Closing { verify: [ ... ], commit_status: [ +1 ] }`.

2. Concrete Example: [examples/canonical_notation.omni]

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ================================================================================
Module < CanonicalNotationShowcase > ( Foundations of Being, Causality, and Abstraction ) {
    version: < "1.0.0" >,
    author: < "ProfessorSeanEX" >,
    system: < OmniCode.Core >,
    semantics: < BalancedTernary :: QuadState :: BoundaryPhysics >
}

# Axiomatic Categories (Invariant Sets)
A < +1 > ( Invariant category A: axiomatic ground truth )
B < +1 > ( Invariant category B: independent set domain )
```

```
# ============================================================================
# BLOCK 2: OPENING BLOCK
# ============================================================================
Opening {
    requires: [ MemoryAligned :: TypeChecking ],
    initial_truth: [ +1 ]
}

a [ +1 ] ( Particular instance 'a' with mutable state )
b [ +1 ] ( Particular instance 'b' with mutable state )
a_in_A [ a ] { A } ( Instance 'a' validated within contextual universe A )

# ============================================================================
# BLOCK 3: BODY BLOCK
# ============================================================================
Body [
    instance_typing [ a : A ]
    identity_check [ a = a ]
    forward_action [ a --> b ]
    synthesis [ a ^-- A ]
    pipeline [ a --> b ! [ a : A :: b : B ] ]
]

# ============================================================================
# BLOCK 4: CLOSING BLOCK
# ============================================================================
Closing {
    verify: [
        identity_check = +1 ::
        forward_action = +1 ::
        pipeline = +1
    ],
    on_rollback: [ <-- unwind_state ],
    commit_status: [ +1 ]
}
```

---

3. Reasoning for Using Rust as the Core Engine

To ensure that the language engine is uncompromising at both low and high levels, the OmniCode compiler and runtime were implemented in **Rust**:

```
flowchart TD
    subgraph RustAdvantages ["Why Rust as the OmniCode Foundation"]
        R1["Memory Safety Without Garbage Collection\nZero-cost pointers, no stop-the-world pauses"]
        R2["Algebraic Data Types & Exhaustive Matching\nGuarantees all 6 boundaries & 4 quad-states are handled"]
        R3["Zero Runtime Buckling\nImpervious to memory leaks, buffer overruns, and data races"]
        R4["LLVM & C ABI Native Compatibility\nDirect bindings to LLVM IR (inkwell) and bare-metal compilation"]
    end
```

1. Zero Runtime Buckling

A language that layers up from bare metal cannot afford to buckle under concurrency or memory corruption. Rust's strict borrow checker and ownership model ensure that the OmniCode compiler, parser, and diagnostic engines execute with zero undefined behavior.

2. Pattern Matching Over Morphological Boundaries

The OmniCode boundary system (`[]` , `<>` , `{}` , `{]` , `[}` , `()`) requires non-trivial parsing logic. Rust's pattern matching (`match`) allows the parser and evaluator to handle asymmetric boundaries and directional vectors cleanly with compile-time exhaustiveness checks.

3. Deterministic Transactional Snapshots

The balanced ternary `-1` (Rollback) state requires snapshotting the environment and unwinding state upon failure. In Rust, cloning scoped bindings and performing deterministic unwinding is explicit, predictable, and fast.

4. Foundation for Future AOT & Transpilation

Because Rust has direct access to LLVM C/C++ APIs, the OmniCode core is primed to generate LLVM IR for AOT native compilation and emit optimized ISO C99 code for universal cross-platform deployment.

`<TRACK 5> : {Infrastructure, Operations & Archives}/docs/02_logic_and_algebra/01_balanced_ternary_quad_state.md` MARKDOWN / DOC | 76 LINES | 536 WORDS | 3.3 KB

Volume 2: Logic & Algebra

Document 01: Balanced Ternary & The Quad-State Engine

1. The Quad-State Truth Space & Micro/Macro Duality

OmniCode replaces Boolean logic (`0` and `1`) with a 4-state system grounded in **Balanced Ternary Logic** with a terminal **Null State**:

$$\mathbb{T}_4 = { -1, \; 0, \; +1, \; \text{NULL} }$$

The engine operates across a fundamental **Micro / Macro Duality**:

State Symbol	Micro Level (Epistemic / Stance)	Macro Level (Systems / Physics)	Directional Vector	OS / Machine Meaning
`+1`	`yes` (Affirmation / Condition holds)	`PROCEED` (Monotonic commit)	`-->` / `->`	Advance PC, commit transaction frame
`0`	`ok` (Neutral equilibrium / Balance)	`AWAIT` (Standby / Async yield)	`---`	Yield thread, wait queue, idle / `HLT`
`-1`	`no` (Refutation / Disagreement)	`ROLLBACK` (Retroactive unwind)	`<--` / `<-`	Unwind memory snapshot, retry vector
`NULL`	`void` (Absence / Non-existence)	`HALT` (Boundary trap / Bottom $\perp$)	`-x-`	Trap handler, kernel panic, abort

```
Micro (Linguistic):     [ no ]      ---      [ ok ]      ---      [ yes ]       -x-  [ void ]
                          |                    |                    |                    |
Value:                   -1                    0                   +1                  NULL
                          |                    |                    |                    |
Macro (Systems):      ROLLBACK    <--        AWAIT       -->     PROCEED        -x-     HALT
```

2. Algebraic Symmetry and Negation (`--`)

In binary logic, negation flips $0 \leftrightarrow 1$. In OmniCode, negation is pure algebraic reflection around the zero equilibrium point:

$$\operatorname{NOT}(x) = -x$$

- $-(+1) = -1$ (NOT Proceed is **Rollback**)
- $-(-1) = +1$ (NOT Rollback is **Proceed**)
- $-(0) = 0$ (NOT Equilibrium is **Equilibrium**)
- $-(\text{NULL}) = \text{NULL}$ (NOT Void is **Void**)

3. Truth Tables for Fundamental Connectives

Conjunction (`::` AND)

Evaluates to the greatest lower bound (minimum): $$\operatorname{AND}(A, B) = \min(A, B) \quad (\text{with NULL acting as an absorbing trap})$$

Disjunction (`|` OR)

Evaluates to the least upper bound (maximum): $$\operatorname{OR}(A, B) = \max(A, B) \quad (\text{with NULL acting as an absorbing trap})$$

A	B	$A :: B$ (AND)	$A \mid B$ (OR)	$--A$ (NOT)
$+1$	$+1$	$+1$	$+1$	-1
$+1$	0	0	$+1$	-1
$+1$	-1	-1	$+1$	-1
0	0	0	0	0

A	B	$A :: B$ (AND)	$A \mid B$ (OR)	$--A$ (NOT)
0	-1	-1	0	0
-1	-1	-1	-1	$+1$
NULL	$*$	NULL	NULL	NULL

4. Direct Mapping to Runtime Mechanics

1. **State `+1` (Proceed):** Advances the program counter, commits current state mutations, and passes through the forward arrow `-->`.
2. **State `0` (Await):** Suspends execution without failure, yielding CPU or awaiting asynchronous event resolution.
3. **State `-1` (Rollback):** Reverts all mutations executed within the current boundary and invokes the backward arrow `<--`.
4. **State `NULL` (Halt):** Immediate abort. Used for fatal assertions, memory out-of-bounds, or program exit.

Volume 2: Logic & Algebra

Document 02: Relational Connectives & Operators

1. Relational Connectives

OmniCode incorporates natural linguistic and relational operators into its formal grammar:

A. The Adversative Conjunction (`!` BUT)

In traditional Boolean logic, "A but B" is flattened into a naive "A and B". In OmniCode, `!` **(BUT)** introduces an explicit **adversative constraint / caveat**: $$A \text{ ! } B \iff A \text{ executes, BUT must satisfy caveat } B$$ If caveat B evaluates to `-1` (Rollback) or `NULL` (Halt), the execution of A is immediately unwound and suppressed.

B. Cumulative Conjunction (`::` AND)

Represents co-presence and mutual satisfaction. Both sides must hold or execute simultaneously.

C. Relational Projection (`:` IN RELATION TO / TO BE AS)

Maps an entity to a perspective, category, or role.

- Used for typing: `x : Integer` ("x in relation to Integer").
- Used for contextual roles: `user : Admin` ("user in relation to Admin role").

D. Disjunction (`|` OR)

Represents branching alternatives and non-deterministic choice.

E. Negation (`--` NOT)

Inverts the quad-state value across the zero balance line.

2. Relations of Being (Ontological Identity)

Operator	Plain Language	Mathematical Concept	Meaning
`=`	**IS / TO BE**	Identity ($\equiv$)	Exact equivalence in value, shape, and essence.
`!=`	**IS NOT / TO BE NOT**	Non-identity ($\not\equiv$)	Distinctness; exclusion.
`≈` or `~=`	**TO BE LIKE / Liken To**	Homomorphism / Analogy	Structural compatibility; behavioral similarity without identical essence.

3. Directional Flow Operators

Operator	Plain Language	Vector Direction	Role
`-->`	**Act Upon**	Forward ($\rightarrow$)	Causation, pipeline, execution step, event dispatch.
`<--`	**Act Again**	Backward ($\leftarrow$)	Feedback, retry, undo, rollback, reactive subscription.
`^--`	**Which Builds To**	Upward ($\uparrow$)	Layering up, abstraction, synthesis, composition.
`v--`	**Which Breaks Down From**	Downward ($\downarrow$)	Decomposition, reduction, destructuring, compilation to IR.

`<TRACK 5> : {Infrastructure, Operations & Archives}/docs/02_logic_and_algebra/03_the_quad_state_accumulator_loop_and_named_registers.md` MARKDOWN / DOC | 259 LINES | 2254 WORDS | 17.9 KB

Volume 2: Logic & Algebra

Document 03: The Quad-State Accumulator Loop & Named Silicon Registers

1. Architectural Genesis: The Death of the Imperative Jump

In legacy computer architecture (x86, ARM, C, Python), a "loop" is treated as an imperative jump instruction (`JMP / JNZ`) wrapped around an impoverished 2-valued boolean (`true / false`). Because binary logic lacks vocabulary for intermediate states, languages bolted on a fragile patchwork of external keywords:

- `continue` (advance to next)
- `break` (escape early)
- `yield` / `sleep` (wait for hardware/scheduler)
- `retry` / `catch` (undo a bad mutation)

In **OmniCode**, a loop is neither a jump nor an imperative hack. **A loop is an ontologically grounded State Machine and Balanced Ternary Accumulator Register cycling through the Quad-State truth space:**

$$\mathbb{T}_4 = { -1, \quad 0, \quad +1, \quad \text{NULL} }$$

2. How Computers Use Registers: History & Functional Roles

Why Registers Exist in Hardware

Registers are the smallest, fastest storage elements in a computing machine, situated directly inside the CPU core alongside the Arithmetic Logic Unit (ALU).

- **Zero Latency:** Accessing a register takes 0 to 1 clock cycles ($< 0.5 \text{ ns}$), whereas accessing L1 cache takes $\sim 1 \text{ ns}$, L3 cache takes $\sim 10\text{--}20 \text{ ns}$, and main RAM takes $\sim 60\text{--}100 \text{ ns}$.
- **Physical Constraint:** The ALU cannot execute vector transformations or balance comparisons directly across external memory buses without stalling execution pipelines. Data must be loaded into local registers, transformed, and retired back to storehouses.

Historical Evolution of Register Counts

- **8-Bit Epoch (6502, Z80):** 3 to 14 registers (`A` accumulator, `X` / `Y` index, `SP` , `PC` , `P` flags).
- **x86-32 CISC:** 8 general-purpose registers (`EAX` , `ECX` , `EDX` , `EBX` , `ESP` , `EBP` , `ESI` , `EDI`) + `EIP` + `EFLAGS` .
- **x86-64 AMD64:** 16 general-purpose registers (`RAX` .. `R15`) + `RIP` + `RFLAGS` .
- **Modern RISC (ARM64, RISC-V):** 32 general-purpose registers (`R0` .. `R31` / `x0` .. `x31`).

Why Binary Registers Use Cryptic Shorthand

Legacy architectures named registers with 1970s shorthand (`AX` = Accumulator, `CX` = Count, `DX` = Data, `BX` = Base, `SP` = Stack Pointer, `BP` = Base Pointer, `SI` = Source Index, `DI` = Destination Index) or anonymous numbers (`r0` .. `r31`). Over decades of compiler optimization, these meaningful names devolved into anonymous scratchpads, widening the cognitive gap between human programmers, machine models, and physical silicon.

3. OmniCode Named Silicon Registers: The ALL CAPS Duality

OmniCode rejects anonymous register indices. Grounded in the principle that **Nouns are Defined and Verbs are Symbolized**, every register in the machine represents an explicit, named ontological actor.

Why ALL CAPS (`IT`) vs. Lowercase (`it`) Matters

In natural language and computation, casing is not decorative—it encodes **ontological hierarchy**:

1. **The Container vs. The Content:**
2. **`IT` (ALL CAPS)** signals the **Physical Silicon Hardware Register** (the physical container on the chip / architectural machine state).
3. **`it` (lowercase)** signals the **Dynamic Linguistic Pronoun** (the ephemeral value currently flowing through the sentence as an antecedent).
4. **Sovereign Invariance vs. Ephemeral Flow:**
5. In OmniCode, ALL CAPS is reserved exclusively for **Sovereign System Invariants**:

- Quad-States: `PROCEED`, `AWAIT`, `ROLLBACK`, `HALT`
- Cardinal Clauses: `WHO`, `WHAT`, `WHEN`, `WHERE`, `WHY`, `HOW`
- Invariant Nouns: `TRIT_NOUN`, `CELL_NOUN`, `STOREHOUSE_NOUN`
- Hardware Registers: `IT`, `ORIGIN`, `HERE`, `SELF`, `CALLER`, `OWNER`, `STEP`, `STATE`, `BOUND`, `INPUT`, `OUTPUT`

6. When the compiler or human reads `IT`, it signals **Direct Silicon Machine Binding**, bypassing userland scope lookups. When reading `it`, it signals **Linguistic Flow**, binding to the antecedent of the previous phrase.
7. **Zero Identifier Collision:**
8. A user or kernel function can safely have a variable or conversational pronoun `it` without ever colliding with the machine register `IT`.

```
┌──────────────────────────────────────────────────────────────────────────────────────┐
│                              OmniCode Canonical Named Silicon Registers               │
├──────────────┬──────────────┬───────┬──────────────────────────────┬─────────────────────┤
| Hardware Name| Natural Word | Index | Ontological & Linguistic Role | Loop & Machine Register Responsibility|
├──────────────┼──────────────┼───────┼──────────────────────────────┼─────────────────────┤
| HEAD         | head / here  |   0   | Tape Head Witness / Cursor    | Instruction Pointer (IP / PC)         |
| HOME         | home / origin|   1   | Genesis Base Pointer          | Pristine rollback landmark: HOME->IT  |
| SPOOL        | spool / owner|   2   | Ribbon Stack Spool            | Partition allocation stack ceiling    |
| IT           | it           |   3   | Pipeline Antecedent (`it`)    | Primary accumulator running sum: Acc+Δ|
| SELF         | self / me    |   4   | 1st-Person Identity (`self`)  | Active executing process principal    |
| CALLER       | caller / you |   5   | 2nd-Person Principal (`you`)  | Invoking parent / return authority    |
| OTHER        | other / them |   6   | 3rd-Person Peer / Swarm       | Worker thread pool / quorum handle    |
| STATE        | state        |   7   | Status / QuadState Balance    | Holds current T4 outcome (+1,0,-1,NULL|
| TURN         | turn / step  |   8   | Cadence / Cycle / Turn Count  | Loop cycle counter: TURN = TURN + 1   |
| BOUND        | bound / limit|   9   | Invariant Axiom Watchdog      | < Invariant > boundary watchdog limit |
| FROM         | from / in    |  10   | Reading Transducer Port       | Hardware streaming source coordinate  |
| INTO         | into / to    |  11   | Writing Transducer Port       | Hardware streaming destination coord  |
| DISCARD      | _ / void     |  12   | Anonymous Discard Sink        | Hardware null sink (like x0 / /dev/null|
└──────────────┴──────────────┴───────┴──────────────────────────────┴─────────────────────┘
```

---

4. The Loop as a Balanced Ternary Accumulator

In an OmniCode loop, the execution unit functions as a physical **Accumulator Register (`IT`)** evaluating deltas along the Radix-3 number line:

$$\dots \xleftarrow{\text{ROLLBACK (-1)}} \mathbf{-1} \quad \xleftrightarrow{\text{AWAIT (0)}} \quad \mathbf{0} \quad \xrightarrow{\text{PROCEED (+1)}} \mathbf{+1} \dots \quad \underset{\text{HALT (NULL)}}{\mathbf{-\times-}}$$

$$\operatorname{IT}_{t+1} = \operatorname{IT}_t + \Delta_{\mathbb{T}_4}$$

State Outcome	Machine Delta ($\Delta_{\mathbb{T}_4}$)	Silicon Register Action	Operational Loop Meaning
`+1: PROCEED`	**`+1`**	`IT += 1` :: `TURN += 1`	**Tick Up:** Advance to next turn, commit transaction frame.
`0: AWAIT`	**`0`**	`IT += 0` (No delta)	**Stop / Equilibrium:** Hold position, yield quantum, idle wait.
`-1: ROLLBACK`	**`-1`**	`IT -= 1` (or restore `HOME --> IT`)	**Tick Down / Unwind:** Revert turn, pop dirty frame, retry.
`NULL: HALT`	**Sentinel**	Latch `IT`, escape loop boundary	**Break / Terminate:** Goal achieved or watchdog boundary hit.

---

5. Syntactic Architecture of the STATE Loop

An OmniCode loop brings together the **Ontological Triad** (`<>`, `[]`, `{}`) with the **4-Branch Causality Tree**:

```
# ============================================================================
# CANONICAL QUAD-STATE LOOP DECLARATION (HARDWARE MACHINE REGISTERS)
# ============================================================================
STATE (
    < BOUND = 1000 >,                          # Rank 1: Invariant Watchdog Ceiling
```

```
    [ IT = 0, TURN = 0 ],                          # Rank 2: Silicon Accumulator & Cadence Registers
    { FROM @ COM1_Serial, INTO @ RAM_Buffer }      # Rank 3: Hardware Transducer Ports
) = [
    # 1. Advance Vector (+1): Ticks Up
    PROCEED  -WHEN-> { (FROM --> IT) & (IT --> INTO) & (IT plus 1 --> IT) & (TURN plus 1 --> TURN) },

    # 2. Standby Vector (0): Zero Velocity
    AWAIT    -WHEN-> { SENSOR_EMPTY | BUSY },

    # 3. Retroactive Vector (-1): Ticks Down / Rewinds
    ROLLBACK -WHEN-> { CHECKSUM_FAIL & (HOME --> IT) },

    # 4. Terminal Vector (NULL): Breaks out of continuum
    HALT     -WHEN-> { TURN = BOUND }
]
```

Dual Structural Symmetry: Push vs. Pull

Both syntactic orientations represent the same underlying circuit:

- **Form A (Inward / Pull - State Integration):**

`omnicode [ IT ] <-- STATE ( <BOUND>, {Context} ) -WHEN-> [ ... ]`

The accumulator **pulls** the ternary delta inward, integrating state across time ($\int \Delta dt$).

- **Form B (Outward / Push - State Actuation):**

`omnicode STATE ( <BOUND>, [IT], {Context} ) --> Action -WHEN-> [ ... ]`

The accumulator **pushes** its delta outward, actuating transformations upon physical memory or devices.

---

6. Physical Bare-Metal Implementation & Machine Word Encoding

When lowered by [`crate::runtime::assembler::OmniAssembler`], the `STATE` loop maps into 64-bit balanced-ternary machine words ([`Word64`]):

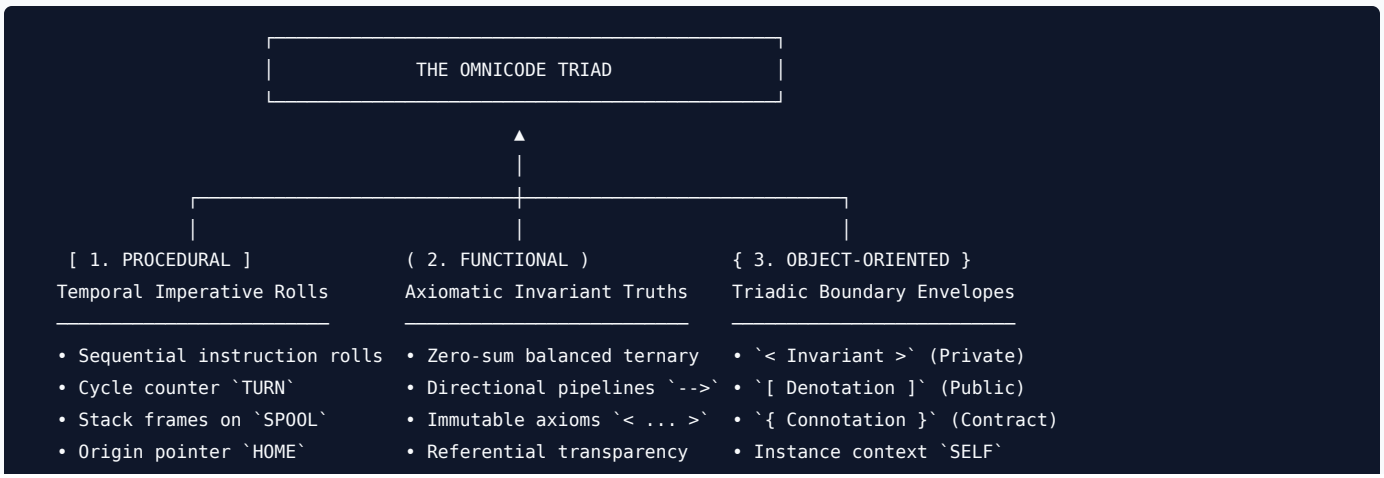
1. **Opcode `ASSERT_INVARIANT` :** Compares `STEP` against `BOUND` . If exceeded, triggers `HALT` .
2. **Opcode `BRANCH_TERNARY` :** Evaluates the `STATE` register (`%flags`):
3. `+1` : Dispatches forward action, adds $+1$ to `IT` , jumps to loop head.
4. `0` : Dispatches `PAUSE / HLT` instruction or cooperative task yield.
5. `-1` : Copies `ORIGIN` back into `IT` , rolling back uncommitted storehouse cells.
6. `NULL` : Unlinks stack frame and proceeds to post-loop instruction.

This unifies high-level human readability, machine AI reasoning, and bare-metal CPU hardware determinism into a single invariant mathematical construct.

---

7. The Triadic Paradigm Synthesis: Procedural, Functional, and OOP as One

OmniCode does not pick one programming paradigm at the expense of others. Instead, it synthesizes the greatest strengths of **Procedural**, **Functional**, and **Object-Oriented Programming** into a single coherent medium:



```
• Program counter `HEAD`       • Algebraic zero equilibrium  • Invoking client `CALLER`
• Natural sentence grammar     • Pure mathematical state     • Subtyping via `v--`
```

1. **The Procedural Core (Temporal Process Ribbon):**
2. Natural language sentences are procedural by design: *Subject* (`[Kernel]`) --> *Verb/Act* (`PROCEED`) to *Object* (`{RAM}`).
3. Deterministic temporal progression step-by-step: each roll advances `TURN += 1` , moves the tape head (`HEAD`), references the base frame (`HOME`), and pushes/pops stack contexts on `SPOOL` .
4. Explicit return values accumulate in `IT` , inputs enter through `FROM` , and results stream to `INTO` .
5. **The Functional Core (Axiomatic Invariant Equilibrium):**
6. Balanced ternary $\mathbb{T}_3 = {-1, 0, +1}$ is an algebra of pure equilibrium (zero-sum symmetry), eliminating dirty hidden states.
7. Directional vectors (`-->`) represent pure monadic pipeline composition: `source --> filter --> transform --> target` .
8. Immutable axioms `< Invariant >` are referentially transparent; Closing Block proofs (`verify: [ ... ]`) formally verify invariants before execution.
9. **The Object-Oriented Core (The Triadic Boundary):**
10. Encapsulation is enforced by physical grammar delimiters rather than compiler keywords:
    - `< Invariant >` : Private invariants, type constraints, and class invariants.
    - `[ Denotation ]` : Public instance variables, mutable storage fields, and concrete state.
    - `{ Connotation }` : Interfaces, external dependencies, and environmental contracts.
11. Identity: `SELF` denotes the active entity instance; `CALLER` denotes the message sender.
12. Specialization & Inheritance: `Parent v-- Child` partitions and derives subtypes.
13. Message Passing: `Receiver --> Method(Arguments)` communicates between objects.

---

8. The Dual-Syntax Engine & The Transpiling Bridge: `.omni` vs. `.tape`

OmniCode introduces an intentional architectural duality between its two primary file formats:

Property	`.omni` (Spatial Architecture Reel)	`.tape` (Process Execution Ribbon)
Execution Model	**Compiled** (multi-pass verification & layout)	**Interpreted** (line-by-line streaming)
Verbosity	Verbose, rigorous, fully declarative	Lean, concise, natural-sentence syntax
Primary Scope	Spatial layout, types, 4-Block architecture	Temporal roll sequence, active execution head
Verification	Invariants proved in Closing Block	Pre-verified rolls executed immediately
Silicon Role	Memory map synthesis & Storehouse carving	Tape Head (`HEAD`) stepping roll by roll

Why `.omni` Files Are Compiled and Verbose

A `.omni` file is an architectural blueprint. It defines the complete spatial structure:

1. **Metadata Block:** Identity, licensing, scheduling priority, and hardware topology.
2. **Opening Block:** Preconditions, imported modules, and memory partition allocations.
3. **Body Block:** Immutable axioms `< >` , entities `[ ]` , interfaces `{ }` , and state loops.
4. **Closing Block:** Formal verification proofs and the covenant commit seal `commit_status [ PROCEED ]` .

Because it establishes spatial topologies and proves theorems, the compiler must analyze the entire file, verify delimiter physics, resolve symbols, allocate silicon offsets (`@ 0x0000`), and compile the 4 blocks into physical machine words ([`Word64`]).

Why `.tape` Files Are Interpreted Line by Line

A `.tape` file is a process ribbon designed for a physical or virtual tape head (`HEAD`). The CPU or interpreter scans one roll at a time, evaluates the sentence, updates registers (`TURN` , `IT`), and executes I/O. It does not need to re-verify global theorems at runtime because the structural invariants were established upstream.

The Transpilation Bridge and Road to Self-Hosting

The bidirectional lowering engine ([`OmniAssembler`]) provides the transpiling bridge: * **Compilation (`.omni` $\longrightarrow$ `.tape`):** Lowers declarative 4-Block spatial reels into sequential instruction rolls executable on bare-metal ribbons. * **Lifting / Decompilation (`.tape` $\longrightarrow$ `.omni`):** Reconstructs the 4-block architecture, opening preconditions, and closing verification proofs from procedural rolls.

Self-Hosting Strategy: Lowering the Rust Host Engine into OmniCode

Because Rust is a procedural systems language with functional idioms and OOP traits, the bridge between `.omni` and `.tape` directly maps to Rust's constructs: * **Rust Structs + Traits** $\longrightarrow$ OmniCode Triadic Entities (`< Invariants > [ Fields ] { Traits }`). *

Rust Functions $\longrightarrow$ OmniCode Instruction Rolls on `SPOOL` using `CALLER`, `SELF`, `FROM`, `IT`, `INTO`. * **Rust Matches & Loops** $\longrightarrow$ OmniCode `STATE` loops driving QuadState transitions. * **Rust Borrow Checker** $\longrightarrow$ OmniCode Delimiter Physics (`< >` immutable, `[ ]` exclusive mutable, `{ }` ambient context).

This enables OmniCode to compile its own host engine and operating system micro-kernel entirely in native OmniCode, achieving true bare-metal self-hosting.

VOLUME 06: TRACK 5 - INFRASTRUCTURE, OPERATIONS & ARCHIVES MANUAL

Chapter 24: Track 5 - Cognitive Systems, Infrastructure & CI/CD

Neural-Symbolic Feedback, Autonomous Restorator, Build Scripts & DNS Infrastructure

> ***Witness:*** *Proverbs 3:19 (KJV) - The LORD by wisdom hath founded the earth*

SYSTEM TRACK
<TRACK 5> : {Infrastructure, Operations & Archives}

CHAPTER IDENTIFIER
CH 24

NOTEBOOKLM WORD COUNT
18,858 words

CATALOGED CHAPTER FILES
23 files (4,042 lines)

RECOMMENDED NOTEBOOKLM COGNITIVE QUERY

"How does the autonomous restorator interface with cross-platform toolchains to certify zero-defect builds?"

CHAPTER MANIFEST & INDEX (23 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 5> : {Infrastructure, Operations & Archives}/TRACK.md**	MD	12	57	0.5
2	**<TRACK 5> : {Infrastructure, Operations & Archives}/docs/README.md**	MD	149	1,286	9.5
3	**<TRACK 5> : {Infrastructure, Operations & Archives}/docs/dev_log.md**	MD	319	2,742	22.3
4	**<TRACK 5> : {Infrastructure, Operations & Archives}/docs/04_cognitive_and_autonomous_systems/01_ai_cortex_and_neural_symbolic_loop.md**	MD	118	607	5.4
5	**<TRACK 5> : {Infrastructure, Operations & Archives}/docs/04_cognitive_and_autonomous_systems/02_episodic_scrolls_and_experiential_learning.md**	MD	130	573	5.2
6	**<TRACK 5> : {Infrastructure, Operations & Archives}/docs/04_cognitive_and_autonomous_systems/03_autonomous_restorator_and_self_healing.md**	MD	111	570	4.6
7	**<TRACK 5> : {Infrastructure, Operations & Archives}/docs/04_cognitive_and_autonomous_systems/04_bare_metal_and_cross_platform_toolchain.md**	MD	68	382	3.0
8	**<TRACK 5> : {Infrastructure, Operations & Archives}/docs/04_cognitive_and_autonomous_systems/05_core_standard_libraries.md**	MD	64	333	2.7
9	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/systemd/omnicode-web.service**	SERVICE	22	34	0.5
10	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/docker/Dockerfile**	TEXT	178	947	7.8
11	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/docker/docker-compose.yml**	YML	113	650	5.7
12	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/check_toolchain.sh**	SH	57	227	2.5
13	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/deploy_oracle_cloud.sh**	SH	160	721	7.0
14	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/oci_security_list_helper.sh**	SH	87	474	4.1
15	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/omnicode_gui.sh**	SH	16	53	0.5
16	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/package_native.sh**	SH	125	395	4.2
17	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/setup_cloudflare_tunnel.sh**	SH	194	713	6.8
18	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/sync_oracle_vm.sh**	SH	133	577	5.6
19	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/dns/README.md**	MD	71	380	2.9
20	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/dns/omnicodestudio.com.zone**	ZONE	49	249	2.6
21	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/dns/zone_manager.py**	PY	269	925	10.7
22	**scripts/build_notebooklm_pdf.sh**	SH	82	393	3.6
23	**scripts/export_notebooklm_pdf.py**	PY	1,515	5,570	58.9

<TRACK 5> : {Infrastructure, Operations & Archives}/TRACK.md MARKDOWN / DOC 12 LINES 57 WORDS 0.5 KB

: {Infrastructure, Operations & Archives}

Scope & Invariant

Operations, containerization, DNS synchronization, packaging, and daemon management.

- `infra/dist/` : Standalone distribution packages (`.tar.gz`).
- `infra/dns/` : Dynamic DNS synchronization scripts & zone records.
- `infra/docker/` : Multi-architecture containerization and composition topologies.
- `infra/scripts/` : Toolchain packaging, cloud synchronization, and deployment automation.
- `infra/systemd/` : Production daemon service definitions.

<TRACK 5> : {Infrastructure, Operations & Archives}/docs/README.md MARKDOWN / DOC 149 LINES 1286 WORDS 9.5 KB

OmniCode Language: Foundational Documentation Series

Welcome to the architectural and theoretical specification of the **OmniCode Language**, a dual-readable, multi-tier programming language engineered for direct human comprehension and deterministic machine execution, grounded in the principles of **Formal Ontology**, **Set Theory**, and **Balanced Ternary Logic**, powered by a high-performance **Rust Core Engine**.

OmniCode bridges the entire spectrum of computing—operating with zero-overhead control at the bare-metal low level, while natively expressing high-level mathematical abstractions, declarative logic, balanced ternary state machines, and machine-readable cognitive intent.

1. Documentation Index & Topology

The specification is organized into four comprehensive volumes:

Volume 1: Foundations

- **01. Ontological Principles & Set Theory:** The philosophical and mathematical basis of entities, intrinsic essence vs. extrinsic context, and set comprehension.
- **02. Boundary Types & Asides:** The 6 morphological boundary delimiters (`[]` , `<>` , `{}` , `{]` , `[}` , `()`), the elimination of dead comments, and machine-readable intent.
- **03. The 8 Parts of Speech & Grammatical Matrix:** Universal linguistic grounding, the 8-word sentence proof, CS counterparts, and primary/secondary grammatical roles.

Volume 2: Logic & Algebra

- **01. Balanced Ternary & The Quad-State Engine:** The 4-state execution truth space (`-1` Rollback, `0` Await, `+1` Proceed, `NULL` Halt), algebraic negation, and physics-driven control flow.
- **02. Relational Connectives & Operators:** The connective algebra (`!` But, `::` And, `:` In relation to, `|` Or, `--` Not) and state relations (`=` Is, `!=` Is not, `≈` Liken to).

Volume 3: Architecture & Execution

- **01. The 3D Coordinate System of Computation:** The three orthogonal axes: Being (Ontology), Dynamics (Causality: `-->` and `<--`), and Scale (Layering: `^--` builds to, `v--` breaks down from).
- **02. The Unified Execution Pipeline:** How a single AST and IR seamlessly feeds AOT Native Compilation (LLVM), Real-time Interpretation (REPL/VM), and Cross-Platform Transpilation (C/WASM).
- **03. The Universal 4-Block Architecture & Rust Foundation:** The deterministic 4-block file lifecycle (`Metadata` $\to$ `Opening` $\to$ `Body` $\to$ `Closing`), the `OmniLinter` , and the technical rationale for using Rust as the core foundation.

Volume 4: Cognitive & Autonomous Systems

- **01. The AI Cortex & Neural-Symbolic Transduction Loop:** The end-to-end cognitive loop transforming natural language prompts into canonical 4-Block Turing reels, executing with quad-state determinism, and producing dual logs.
- **02. Episodic Scrolls & Experiential Learning Archive:** The dual inscription model (`.scroll` executable reels and `.log` telemetry) and `AILearner` episodic memory digestion.
- **03. The Restorator & Autonomous Self-Healing Diagnostics:** Autonomous boundary auto-healing, delimiter ranking hierarchy, missing block auto-scaffolding, and transactional rollback protection.
- **04. Bare-Metal OS & Cross-Platform Toolchains:** Bare-metal targets (`x86_64` , `aarch64` , `riscv64gc`), QEMU hardware emulation, Docker toolchain, and ISO bootloaders.
- **05. Core Standard Library Architecture:** Specification of the built-in standard library in `lib/core/` (`arithmetic.omni` , `logic.omni` , `vectors.omni` , `memory.omni`).

2. Symbol & Operator Matrix

Axis / Domain	Symbol	Syntax Example	Meaning
Being (Y-Axis)	`=`	`a = b`	Identity assertion (True if identical)
	`!=`	`a != b`	Distinctness assertion (True if distinct)
	`~=` / `≈`	`a ~= b`	Homomorphism / Structural similarity
Dynamics (X-Axis)	`-->`	`a --> b`	Forward causality: 'a' acts upon 'b'

Axis / Domain	Symbol	Syntax Example	Meaning
	`<--`	`b <-- a`	Retroactive feedback: 'b' reflects upon 'a'
Scale (Z-Axis)	`^--`	`a ^-- A`	Micro-to-Macro: 'a' builds up to 'A'
	`v--`	`A v-- a`	Macro-to-Micro: 'A' breaks down into 'a'
Connective Algebra	`!`	`a --> b ! [ c = +1 ]`	Adversative caveat (**BUT** condition)
	`::`	`a : A :: b : B`	Conjunction (**AND** / Consensus)
	`:`	`a : A`	Ontological aspect (**IN RELATION TO**)
	`|`	`a | b`	Disjunction (**OR**)
	`--`	`-- a`	Algebraic negation (**NOT**)
Morphological Boundaries	`[...]`	`[ data ]`	Rigid / Fixed / Physical Denotation
	`<...>`	`< type >`	Dynamic / Fluid / Invariant Process
	`{...}`	`{ universe }`	Universal / Ambient Context / Scope
	`{...]`	`{ A ]`	Top-Down / Absorber (Fluid in $\to$ Fixed out)
	`[...}`	`[ B }`	Bottom-Up / Generator (Fixed in $\to$ Fluid out)
	`(...)`	`( intent aside )`	First-class Aside (Machine-readable intent)
Balanced Ternary Truth	`+1`	`+1 / PROCEED`	Monotonic forward commit
	`0`	`0 / AWAIT`	Indeterminate / Awaiting consensus
	`-1`	`-1 / ROLLBACK`	Transactional contra / Veto / Rollback
	`NULL`	`NULL / HALT`	Terminal absorbing trap / Void

3. Canonical Showcase: `examples/canonical_notation.omni`

```
# ============================================================================
# BLOCK 1: METADATA BLOCK
# ============================================================================
Module < CanonicalNotationShowcase > ( Foundations of Being, Causality, and Abstraction ) {
    version: < "1.0.0" >,
    author: < "ProfessorSeanEX" >,
    system: < OmniCode.Core >,
    semantics: < BalancedTernary :: QuadState :: BoundaryPhysics >
}

A < +1 > ( Invariant category A: axiomatic ground truth )
B < +1 > ( Invariant category B: independent set domain )

# ============================================================================
# BLOCK 2: OPENING BLOCK
# ============================================================================
Opening {
    requires: [ MemoryAligned :: TypeChecking ],
    initial_truth: [ +1 ]
}

a [ +1 ] ( Particular instance 'a' with mutable state )
b [ +1 ] ( Particular instance 'b' with mutable state )
a_in_A [ a ] { A }  ( Instance 'a' validated within contextual universe A )
a_top_down { A ]    ( Outside-In: Environment A actively forces definition of a )
b_bottom_up [ B }   ( Inside-Out: Definition of b emerges from context B )

# ============================================================================
# BLOCK 3: BODY BLOCK
# ============================================================================
Body [
```

```
    instance_typing [ a : A ]    ( 'a' stands in relation to Category A )
    identity_check [ a = a ]     ( 'a' is identical to 'a' )
    distinction_check [ a != b ] ( 'a' is not 'b' )
    similarity_check [ a ~= b ]  ( 'a' is liken to 'b' in form )

    forward_action [ a --> b ]   ( 'a' acts upon 'b' forward in time )
    feedback_action [ b <-- a ]  ( 'b' reflects back upon 'a' as retroactive feedback )

    synthesis [ a ^-- A ]        ( Low-level primitive 'a' builds up to concept A )
    decomposition [ A v-- a ]    ( Category A decomposes down into primitive 'a' )

    negation [ -- a ]
    conjunction [ a : A :: b : B ]
    pipeline [ a --> b ! [ a : A :: b : B ] ] ( 'a' acts on 'b', BUT caveat must hold )
]

# ============================================================================
# BLOCK 4: CLOSING BLOCK
# ============================================================================
Closing {
    verify: [
        identity_check = +1 ::
        forward_action = +1 ::
        pipeline = +1
    ],
    commit_status: [ +1 ] ( Final transaction verification: Proceed )
}
```

4. Verification Suite & Invariant Seal

The theoretical specifications in this documentation are backed by executable code and passing unit/integration tests in the Rust core:

- **24 Automated Tests:**
- Unit tests validating boundary hierarchies, vector parsers, and quad-state logic in `src/lib.rs` .
- 17 Failpoint integration tests in `tests/test_failpoints.rs` .
- 2 AI Cortex and Episodic Learner integration tests in `tests/test_ai_model.rs` .
- **CLI Diagnostic Tools:**
- `omnicode lint <file>` : Enforces 4-Block architectural compliance.
- `omnicode restore <file>` : Autonomous self-healing verification.
- `omnicode ai "<prompt>"` : Cognitive loop verification.
- `omnicode scrolls` : Episodic memory archive verification.

All specifications herein are invariant and formally sealed.

<TRACK 5> : {Infrastructure, Operations & Archives}/docs/dev_log.md MARKDOWN / DOC 319 LINES 2742 WORDS 22.3 KB

OmniCode: Engineering & Architectural Developer Log

This log chronicles the architectural decisions, theoretical discoveries, and implementation milestones of the **OmniCode Language, Runtime, and Operating System Foundation**.

[2026-10-10] Milestone 14: Operating System Genesis, Multi-Track Scheduler & Process Tape Execution

1. Physical Computing Hierarchy: Roll $\longrightarrow$ Tape $\longrightarrow$ Track

- **The Breakthrough:** Transformed OmniCode from symbolic physics into a living, bootable Operating System:
- **Roll (`Roll`):** A single line of code / atomic state mutation in silicon memory.
- **Tape (`Tape` / `.tape`):** A sequential process ribbon consisting of a leading **Tape Head** covenant and an ordered stream of subordinate rolls.
- **Track (`Track`):** An execution channel. The scheduler provides 3 discrete physical channels:
  - `Track 0 [KERNEL]` : Hardware initialization, genesis daemons, protected kernel space.
  - `Track 1 [TERMINAL]` : Interactive console HUD, REPL environment, video buffers.
  - `Track 2 [APP]` : Userspace applications, background workers, cognitive models.

2. Genesis Boot Covenant (`lib/kernel/boot.tape`)

- Created the primordial OS boot sequence ribbon:
- Declares the 4096 physical cell `Root_RAM` storehouse.
- Carves the protected `Kernel_Partition` (1024 cells) and `Terminal_Partition` (512 cells) via `v--` Storehouse partitioning.
- Establishes initial kernel equilibrium state at cell 0 (`0 --> Kernel_Partition @ 0`).
- Seals the boot sequence with `commit_status [ PROCEED ]` .

3. Living In-Memory Terminal Shell & Cooperative Scheduler

- Upgraded `OmniScheduler` and `TerminalSession` :
- Multi-track process scheduling with priority queues (`High` , `Normal` , `Low` , `Background`).
- Direct commands: `mount <path.tape>` , `run <path.tape>` , `step <pid>` , `rolls <pid>` , `mem` , `tracks` .
- Evaluates live OmniCode rolls directly in silicon memory without subprocess overhead, maintaining persistent `Environment` bindings.

4. CLI Execution & End-to-End Functional Verification

- Upgraded CLI `omnicode run <path.tape>` to mount and step process tapes roll-by-roll.
- Added `test_genesis_boot_tape_execution` and `test_terminal_session_boot_and_tape_commands` in `tests/functional/main.rs` .
- **Test Suite Status:** All 47 tests passing (16 in `lib.rs` , 7 functional, 14 integrated, 10 unit) with zero warnings and zero regressions.

[2026-10-10] Milestone 13: AIMemory Store, Biblical Ground Truth & Thought Formulation Engine

1. Dedicated AI Memory Store (`src/ai/model/memory.rs`)

- **The Architecture:** Created `AIMemory` as the dedicated repository for the internal AI model's cognitive state:
- **Primordial Ground Truth:** Anchored in `BiblicalLexicon` (KJV/WEB public domain scriptures across 6 ontological domains: Cosmology, LivingWorld, HumanCondition, MoralVirtue, MoralVice, CausalLaw).
- **Associative Concept Graph:** Seeds foundational connections (`light` , `storehouse` , `cell` , `restoration` , `tape` , `wisdom` , `measure`) and allows dynamic association learning.
- **Episodic Scrolls Store:** Ingests historical `.scroll` files from `runtime/scrolls/` and inscribes newly formulated thought scrolls.
- **Working Thought Space:** Records active thoughts formulated by the AI model.

2. Native Thought Formulation in OmniCode Syntax

- The internal AI model formulates thoughts directly in OmniCode syntax rather than unstructured natural language or opaque neural embeddings:
- Employs the **6 Cardinal Classification Clauses**: `WHO` (Internal_AI), `WHAT` (CognitiveAction), `WHEN` (Lifecycle phase), `WHERE` (Target Vessel), `WHY` (Scripture / Invariant anchor), `HOW` (Balanced Ternary Quad-State + Guidance).
- Synthesizes executable **Thought Tapes**:

`omnicode [Thought.th_001] : WHO {Internal_AI} --> DELIBERATE {CognitiveThought_light} WHEN {Opening} WHERE {Cortex_Memory} = [light] WHY (Genesis 1:3 - 'And God said, Let there be light: and there was light.') HOW [PROCEED : Walk in the light; transparent and lawful boundaries.] :: < manifest > (illumination)`

- Enforces moral evaluation: Constructive prompts deliberate `+1: PROCEED`; malicious prompts (e.g., deceit, theft, harm) deliberate `-1: ROLLBACK` or `NULL: HALT`.

3. DiagnosticLogger Upgrade: Sensory Bridge to Memory & Scrolls

- Upgraded `DiagnosticLogger`:
- `log_thought()`: Automatically structures thoughts into episodic `CognitiveNote` records tagged `#thought` and `#ai_cortex`.
- `log_memory_association()`: Inscribes associative links between entities and scriptural/scroll anchors.
- `log_scroll_inscribed()`: Logs physical `.scroll` generation.
- `inscribe_canonical_scroll()`: Directly generates and writes valid 4-Block `.scroll` files to disk.

4. AICortex Integration & Test Verification

- Integrated `AIMemory` directly into `AICortex`:
- Deliberates prompt through `formulate_thought()` prior to semantic transduction.
- Automatically refreshes episodic memory via `digest_scrolls()` after saving execution scrolls.
- Added `test_ai_memory_and_thought_formulation` in `tests/unit/main.rs`.
- **Test Suite Status:** All 44 tests passing (15 in `lib.rs`, 5 functional, 14 integrated, 10 unit) with zero warnings.

[2026-10-10] Milestone 12: The AI Cognitive Loop, Episodic Scrolls & Native Thought Formulation

1. The Native Linguistic Identity of the AI Model

- **The Breakthrough:** Traditional operating systems treat AI as an external userland Python process, creating an irreconcilable semantic translation tax. In OmniCode, **the language of the internal AI model is the exact same language as the Operating System kernel.**
- Both speak the 8 parts of speech, 8-slot canonical sentences, and balanced ternary physics (`PROCEED`, `AWAIT`, `ROLLBACK`, `HALT`).
- When the internal AI formulates a thought, it writes it in **native OmniCode**, meaning an AI thought can be scheduled directly onto hardware as a **Tape**!

2. The Role of the Scrolls (`.scroll`) as Episodic Memory

- A `.scroll` is an **Episodic Memory Chronicle**:
- Records the session prompt, executed rolls, quad-state outcomes, failpoints, applied repairs, and first-class Asides `(...)`.
- Preserves **WHY** code was written, so the AI accumulates experiential memory over time.
- `AILearner` digests `.scroll` files to build training memories anchored to the `BiblicalLexicon`.
- The AI formulates new deductions directly from its past experiences and memory scrolls.

3. The Triune Diagnostic Nervous System

- **Logger (Sensory Organ):** Observes every vector mutation and records episodic scrolls into `scrolls/`.
- **Debugger (Conscious Auditor):** Monitors invariant boundaries (`{...}`) and enforces the 7 Invariant Covenant Laws.
- **Restorator (Self-Healing Cortex):** Reads the Tape Head and past memory scrolls to synthesize autonomous fixes when a `-1: ROLLBACK` occurs, preventing kernel crashes.

[2026-10-10] Milestone 11: Cardinal Classification Clauses, Spatial Vector Modulations & `v--` Partitioning

1. The 6 Cardinal Classification Clauses (The 5 W's and 1 H)

- **The Problem:** Program statements often lack clear dimensionality (who runs it, when it executes, where it is stored, why it exists, how it mutates).
- **The Solution:** Codified `WHO`, `WHAT`, `WHEN`, `WHERE`, `WHY`, and `HOW` as first-class classification clauses in both the Long Form and Sentence Form:
- `WHO`: Agent / Caller / Security Principal (`WHO self`).
- `WHAT`: Entity / Vessel / Payload (`WHAT [Cell]`).
- `WHEN`: Cadence / Trigger / Lifecycle phase (`WHEN Opening`).
- `WHERE`: Spatial site / Memory address / Invariant bounds (`WHERE {unit}`).
- `WHY`: Teleological intent / Axiom link (`WHY ( Safe placement )`).
- `HOW`: Causal mechanism / Transducer cycle (`HOW [ 8 bit --> 5 trit --> 8 bit ]`).

2. Spatial Directional Vector Geometry & 2D Quadrants

- **The Problem:** Horizontal arrows (`-->` , `<--`) only capture 1D data movement. Real spatial computation requires multi-scale elevation, destructuring, and 2D/3D coordinate motion.
- **The Solution:** Added 8 new directional vectors to `src/core/operators.rs` :
- Scale Modulations: `/-^` (Major), `\-v` (Minor), `\-^` (Great), `/-v` (Small).
- 2D Quadrant / Compass Vectors: `/^` (Quadrant I: $+X, +Y$), `\v` (Quadrant IV: $+X, -Y$), `^\` (Quadrant II: $-X, +Y$), `v/` (Quadrant III: $-X, -Y$).
- Range Span Operator: `..` (`[ 0 .. 256 ]`).

3. Runtime Storehouse Partitioning (`v--`)

- Wired the `BreaksDownFrom` (`v--`) operator in `src/runtime/evaluator.rs` to directly carve child memory partitions from parent storehouses:
- `Storehouse v-- Scratchpad [ 256 ]` calls `Storehouse::partition("Scratchpad", 256)` , tracking origins and preventing out-of-bounds leaks.
- Added `test_cardinal_clauses_and_spatial_directionals_and_partitioning` in `tests/unit/main.rs` .
- All 43 automated unit, functional, and integrated tests pass with zero errors.

[2026-10-10] Milestone 10: 8-Slot Canonical Sentence Architecture & ASCII Operator Specialization

1. The 8-Slot Canonical Sentence Architecture

- **The Problem:** An arbitrary sentence length or rigid 3-part imperative statement (`subj verb obj`) cannot naturally house invariants, execution cadences, spatial prepositions, or error traps without clumsy decorators, macros, or ad-hoc keywords.
- **The Solution:** Formalized the **8-Slot Canonical Sentence Architecture** matching 1:1 with the 8 parts of speech:
- **Slot 1 (Interjection):** Alert / Trap / Teleological Aside (`Oh!` , `( Intent )` , `HALT`).
- **Slot 2 (Conjunction):** Pipeline transition / Linkage / Consensus (`::` , `&`).
- **Slot 3 (Pronoun):** Context handle / Scoped caller identity (`self` , `it` , `origin`).
- **Slot 4 (Adjective):** Invariant contract / Axiom qualifier (`{ Pure }` , `< Axiom >` , `PROCEED`).
- **Slot 5 (Noun):** Subject substance / Vessel container / Data operand (`Gold` , `Storehouse` , `Cell`).
- **Slot 6 (Verb):** Causal vector / Action / State transition (`-->` , `<--` , `v--` , `plus` , `=`).
- **Slot 7 (Adverb):** Execution cadence / Policy / Negation (`--` , `atomic` , `yield`).
- **Slot 8 (Preposition):** Spatial address / Memory site / Index enclosure (`@ Vault_Offset` , `[0 .. 256]`).

2. ASCII Operator Specialization & Distinction

- `@` **— Spatial Location (Preposition):**
- Specifies memory site, address, hardware MMIO offset, or network endpoint: `Storehouse @ Vault_Offset` .
- `%` **— Percentage & Capacity Fullness (Adjective / Noun):**
- Expresses relative ratios and fullness thresholds without arithmetic boilerplate: `Storehouse % 80` , `Throttle % 50` .
- `&` **vs.** `::` **— Inseparable Bond vs. Cooperative Junction:**
- `&` **(Hand-in-Hand Inseparable Binding):** Binds twin entities into an indivisible covenant (`Jack&Jill` , `Key&Value` , `Addr&Offset` , `Tx&Rx`). `A` cannot be referenced without `B` , nor `B` without `A` .
- `::` **(Cooperative Junction):** Joins independent entities together in sequence, pipeline chaining, or namespace consensus (`A :: B :: C`). Entities maintain independent existence and can be used on their own.

3. Engine Implementation & Lexer Expansion

- Added `TokenType::At` , `TokenType::Percent` , and `TokenType::BoundPair` in `src/parser/lexer.rs` .
- Updated `src/core/linguistics.rs` with primary and secondary grammatical classifications for all new tokens.
- Updated `docs/01_foundations/03_eight_parts_of_speech_matrix.md` .
- All 42 tests passing with zero errors.

[2026-10-10] Milestone 9: Vessel Subsystem Codification & `lib/core/memory.omni` Library

1. Rust Engine Implementation: Bounded Vessels & Storehouses

- **The Problem:** The language lacked runtime data structures for contiguous, bounded memory spaces, and evaluating SVO directional vector operations against indexed memory (`Value --> Vessel[Offset]`).
- **The Solution:** Engineered `src/core/vessel.rs` and integrated it into the runtime:
- Implemented `Vessel` , `Storehouse` , and `Partition` with strict invariant boundary enforcement.
- An out-of-bounds access (`Offset >= Capacity`) does not segfault; it naturally returns `-1: ROLLBACK` via balanced ternary physics.
- Added `RuntimeValue::Cell` and `RuntimeValue::Vessel` to `src/runtime/environment.rs` .
- Upgraded `src/runtime/evaluator.rs` to support vessel declarations, spatial index reads (`Vessel[Offset]`), forward vector stores (`Value --> Vessel[Offset]`), and feedback reads (`Dest <-- Vessel[Offset]`).

2. Standard Memory Library Reel (`lib/core/memory.omni`)

- Authored the foundational memory management module in 4-Block architecture:
- **Block 1 (Metadata):** Declares invariant macros for `TRIT_NOUN` , `CELL_NOUN` , `VESSEL_NOUN` , `STOREHOUSE_NOUN` , `PARTITION_NOUN` , `OFFSET_NOUN` , `CAPACITY_NOUN` , `COUNT_NOUN` , `VALUE_NOUN` .
- **Block 2 (Opening):** Allocates a root `Main_Storehouse [ 1024 ]` and an entrusted `Task_Partition [ 256 ]` .
- **Block 3 (Body):** Executes natural number storage (`42 --> Task_Partition[0]`), retrieval (`read_val <-- Task_Partition[0]`), and affirmative state inscription (`PROCEED --> Task_Partition[1]`).
- **Block 4 (Closing):** Verifies all memory operations and seals with `commit_status [ PROCEED ]` .

3. Verification & Test Suite Expansion

- Added `test_lib_core_memory` integration test in `tests/integrated/main.rs` .
- All 42 automated tests across unit, functional, and integrated suites pass with zero errors.

[2026-10-10] Milestone 8: 8-Bit to 5-Trit Hardware Transducer & Core Noun Taxonomy

1. Mathematical Grounding: 8 Bits <--> 5 Trits Bijective Transducer

- **The Problem:** In traditional C, an 8-bit byte ($2^8 = 256$ states) has zero space to store `NULL` , `AWAIT (0)` , `ROLLBACK (-1)` , or `PROCEED (+1)` without in-band signaling collisions or ballooning struct wrapper bloat.
- **The Solution:** Engineered a bijective hardware codec in `src/core/cell.rs` :
- **243 Data States ($0 \dots 242$):** Bijectively maps to 5 balanced trits spanning $[-121 \dots +121]$ with Radix-3 place values ($81, 27, 9, 3, 1$). State is implicitly `PROCEED` .
- **13 Out-of-Band Control States ($243 \dots 255$):**
  - `243` : `NULL / HALT` (Void / unmapped memory / terminal trap).
  - `244` : `AWAIT` (Superposition / pending / non-blocking lock).
  - `245` : `ROLLBACK` (Veto / tainted / unwound transaction cell).
  - `246` : `PROCEED` (Affirmative sealed landmark).
  - `247..255` : System traps & physical hardware markers.
- **Dense Memory Efficiency:** 1 Cell = exactly 1 physical byte in RAM, but unpacks losslessly into 5-trit balanced ternary + Quad-State. Zero struct padding, zero alignment waste.

2. Noun Taxonomy Codified

- **Atomic Units:** `Trit` , `Cell` , `Word` .
- **Storage Vessels:** `Vessel` , `Storehouse` , `Partition` .
- **Coordinates:** `Offset` , `Origin` , `Landmark` .
- **Measures:** `Capacity` , `Count` .
- **Substance:** `Value` , `State` .

3. Transducer Verification

- Verified 100% round-trip conversion for all 243 natural values ($0 \dots 242$), all 243 signed values ($-121 \dots +121$), and all 4 out-of-band Quad-State markers.
- All 38 automated unit, functional, and integrated tests passing cleanly.

[2026-10-10] Milestone 7: Universal Linguistics & 8 Parts of Speech Formalization

1. Conceptual Breakthrough: Nouns are Defined, Verbs are Symbolized

- **The Problem:** Traditional programming languages clone imperative C/Rust jargon (`malloc` , `free` , `alloc` , `deref` , `peek` , `poke` , `mut` , `fn`). This creates a severe cognitive translation penalty: human and AI reasoning in natural grammar must be disassembled into mechanical idioms.
- **The Discovery:** OmniCode is not an imperative clone. In OmniCode:
- **Nouns are defined:** Entities, storage containers, measures, types, and values are expressed in natural human vocabulary (`Vessel` , `Storehouse` , `Measure` , `Offset` , `Bound`).
- **Verbs are symbolized:** Actions, state transitions, and relations are expressed through universal directional vectors (`-->` , `<--` , `^--` , `v--`) and relational operators (`=` , `!=` , `~=`).

2. The 8-Word Grammatical Proof

- Demonstrated that a single, coherent, 8-word sentence cleanly incorporates all 8 parts of speech: $$\text{“}\underbrace{\text{Oh!}}*{\text{Interjection}}\quad\underbrace{\text{But}}*{\text{Conjunction}}\quad\underbrace{\text{my}}*{\text{Pronoun}}\quad\underbrace{\text{old}}*{\text{Adjective}}\quad\underbrace{\text{man}}*{\text{Noun}}\quad\underbrace{\text{walked}}*{\text{Verb}}\quad\underbrace{\text{straight}}*{\text{Adverb}}\quad\underbrace{\text{ahead}}*{\text{Preposition}}\text{.”}$$

- Mapped each part of speech directly to its operational counterpart in computer science and operating systems:
- Nouns $\to$ Data, Types, Memory Containers, Hardware Devices.
- Pronouns $\to$ References, Context Handles, Iterators (`self` , `this` , `_`).
- Verbs $\to$ Instructions, State Transitions, Mutations (`-->` , `<--` , `=`).
- Adjectives $\to$ Type Invariants, Contracts, Bounds (`{ Bounded, Fixed }`).
- Adverbs $\to$ Execution Policies, Scheduler Priorities, Negation (`--`).
- Prepositions $\to$ Memory Offsets, Array Indexing (`[offset]`), Projections (`:`).
- Conjunctions $\to$ Logic Connectives, Pipeline Guards, Caveats (`!` , `::` , `\|`).
- Interjections $\to$ Traps, Interrupts, Halts, Panics, Asides (`(...)` , `NULL` , `-1`).

3. Implementation Upgrades

- Implemented `src/core/linguistics.rs` defining `PartOfSpeech` , `GrammaticalClassification` , and `GrammaticalEntity` .
- Classified all tokens, boundaries, directional vectors, relational operators, and quad-states with **Primary**, **Secondary**, and **Tertiary** grammatical roles.
- Enriched standard library macros in `lib/core/vectors.omni` with grammatical role metadata.
- Authored formal specification: `docs/01_foundations/03_eight_parts_of_speech_matrix.md` .
- Verified: All 26 automated unit, functional, and integration tests passing.

[2026-10-10] Milestone 6: Cloud Deployment & Oracle VM Synchronization

- Installed `oci-cli` v3.94.2 at `/usr/local/bin/oci` on remote Oracle Cloud VM (`147.224.147.30`).
- Configured `iptables` Rule #1 for authorized IP `71.11.99.168` and opened port 8080 (persisted via `netfilter-persistent`).
- Synced clean directory layout to Oracle VM via `infra/scripts/sync_oracle_vm.sh` .
- Verified all 9 public HTTP endpoints returning HTTP 200 OK (`/api/health` , `/terminal` , `/download` , `/specs` , `/model` , `/lexicon` , `/scrolls` , `/cloud` , `/api/scheduler`).
- DNS synchronization confirmed via `omnicodestudio.duckdns.org` .

[2026-10-10] Milestone 5: Workspace Reorganization & Playwright Validation

- Reorganized repository root into 8 clean, logical directories:
- `apps/` — Native GUI and web editor interfaces.
- `infra/` — DNS scripts, Docker configurations, systemd services, sync scripts.
- `runtime/` — Executable scrolls, test logs, episodic traces.
- `src/` — Rust compiler and runtime source code.
- `lib/` — Standard library modules (`lib/core/`).
- `examples/` — Canonical showcases.
- `tests/` — Unit, functional, and integrated test suites.
- `docs/` — Architectural and foundational specifications.
- `dist/` — Production releases and tarballs.
- Bootstrapped headless Playwright browser driver for automated end-to-end browser GUI testing; captured high-resolution verification screenshots of the live web terminal HUD.

[2026-10-09] Milestone 4: Cooperative OmniScheduler & Interactive Terminal REPL

- Built `OmniScheduler` with 4 priority tiers (`System` , `High` , `Normal` , `Idle`) and tick quantum enforcement.
- Integrated cooperative yielding via balanced ternary `0: AWAIT` .
- Created interactive REPL (`TerminalSession`) supporting multi-line input, transaction rollback, and live quad-state feedback.

[2026-10-09] Milestone 3: AI Cognitive Cortex & Episodic Scrolls

- Integrated 100% offline `BiblicalLexicon` derived from public domain KJV 1611 and WEB scriptures across 6 domains (`Cosmology` , `LivingWorld` , `HumanCondition` , `MoralVirtue` , `MoralVice` , `CausalLaw`).
- Created `AICortex` deliberation engine evaluating natural language prompts against 7 Invariant Covenant Laws.
- Built dual-logging pipeline: `.scroll` executable reels and `.log` machine telemetry.

[2026-10-08] Milestone 2: Compiler Pipeline & Universal 4-Block Architecture

- Implemented lexer, parser, and AST with first-class Asides `(...)` .
- Formatted source architecture into 4 deterministic blocks: `Metadata` , `Opening` , `Body` , `Closing` .

- Built `OmniLinter` and autonomous self-healing `Restorator` capable of repairing boundary leaks and missing scaffolding.

[2026-10-08] Milestone 1: Algebraic Foundation & Balanced Ternary Physics

- Initialized core physics: Balanced Ternary Quad-State (`+1: PROCEED` , `0: AWAIT` , `-1: ROLLBACK` , `NULL: HALT`).
- Engineered 6 morphological boundary delimiters ranked by ontological hierarchy (`<...>` , `[...]` , `{...}` , `[...}` , `{...]` , `(...)`).
- Established 3D coordinate operators along Being (Y), Dynamics (X), and Scale (Z) axes.

`<TRACK 5> : {Infrastructure, Operations & Archives}/docs/04_cognitive_and_autonomous_systems/01_ai_cortex_and_neural_symbolic_loop.md` MARKDOWN / DOC | 118 LINES | 607 WORDS | 5.4 KB

Volume 4: Cognitive & Autonomous Systems

Document 01: The AI Cortex & Neural-Symbolic Transduction Loop

> ***Bridging Freeform Human Discourse and Deterministic Machine Execution via Dual-Readable 4-Block Turing Reels.***

In traditional computing, natural language processing and compiler execution exist in disjoint worlds: LLMs generate unverified code strings that are prone to hallucinated syntax, while compilers reject anything that deviates from rigid grammars.

OmniCode solves this duality by embedding a **Neural-Symbolic Cognitive Loop** directly within the language engine:

$$\text{Natural Prompt} \xrightarrow{\text{Transducer}} \text{Canonical 4-Block AST} \xrightarrow{\text{Restorator}} \text{Self-Healed Code} \xrightarrow{\text{Evaluator}} \text{Episodic Trace} \xrightarrow{\text{Logger}} \text{Dual Inscription}$$

1. Architectural Foundation & Preconditions

The `AICortex` orchestrates five specialized subsystems within an integrated cognitive loop:

```
flowchart TD
    Prompt["Human Prompt (Natural Discourse)"] --> Transducer["AITransducer (Entity & SVO Parser)"]
    Transducer --> RawOmni["Draft 4-Block OmniCode"]
    RawOmni --> Restorator["Restorator (Autonomous Self-Healing)"]
    Restorator --> HealedOmni["Sanitized Executable OmniCode"]
    HealedOmni --> Evaluator["Evaluator & OmniDebugger (Quad-State Physics)"]
    Evaluator --> ExecResult["Runtime Result & Telemetry"]
    ExecResult --> ScrollLogger["ScrollLogger (Episodic Inscriber)"]
    ScrollLogger --> ScrollFile[".scroll (Native Executable Reel)"]
    ScrollLogger --> LogFile[".log (Microsecond Telemetry)"]
```

Precondition Invariants

1. **Deterministic Transduction:** All inferred entity names, numeric literals, and relational directions must be captured in explicit 4-Block sections.
2. **Intent Preservation:** Conceptual commentary and reasoning traces are preserved as first-class Asides `(...)`.
3. **Transactional Safety:** If evaluation triggers a `-1:ROLLBACK` or caveat veto, state changes are rolled back automatically.

2. Transduction Mechanics & Operational Pipeline

The `AITransducer` executes a four-stage cognitive transformation:

1. **Entity & Numeric Extraction:**
2. Scans natural discourse for quantities (e.g., `5 units`, `12 items`) and targets (e.g., `inventory A`, `group B`).
3. Distinguishes initial baseline quantities from incremental operands.
4. **Subject-Verb-Object (SVO) Resolution:**
5. Maps operational verbs (`add`, `combine`, `transfer`, `mutate`, `verify`) to directional vectors:
    - Addition / Transfer $\to$ `-->` (Act Upon / Forward Causality).
    - Retroactive Feedback $\to$ `<--` (Feedback / Reverse Causality).
    - Invariant Layering $\to$ `^--` (Builds to) / `v--` (Breaks down from).
6. **Ontological Type Inference:**
7. Dynamically resolves domain nouns into ontological categories:
    - Discrete counts $\to$ `Entity?`
    - Aggregations $\to$ `Set?`
    - Contextual domains $\to$ `Domain?`
8. **Canonical 4-Block Assembly:**
9. Wraps inferred logic into standard `Module`, `Opening`, `Body`, and `Closing` blocks.

Concrete Transduction Walkthrough

Input Natural Language Prompt:

```
"Add 5 units to inventory A which contains 12 units"
```

Inferred 4-Block OmniCode Reel:

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ================================================================================
Module < NaturalReel.Addition > ( Add 5 units to inventory A which contains 12 units ) {
    version: < "1.0.0" >,
    system: < OmniCode.Core >,
    semantics: < TrackPhysics :: MultiTape :: AIInference >
}

Units < Entity? > ( Inferred discrete unit: resolved from 'Units' )
Inventory < Set? > ( Inferred collection container: resolved from 'Inventory' )

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    import: [ Core.Arithmetic :: Core.Logic :: Core.Vectors ],
    preconditions: [ +1 ]
}

inventory_a [ 12 ] ( Initial baseline state of target container )
transfer_units [ 5 ] ( Incremental operand value )

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    result_inventory_a [ inventory_a + transfer_units ]
    inventory_a --> result_inventory_a ( Directional causal update )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [ result_inventory_a = 17 ],
    commit_status: [ +1 ]
}
```

---

3. Verification, Telemetry & Invariant Guarantees

1. **State Certification:** Transduced programs are executed through the `Evaluator` before any output is returned to the user.
2. **Dual Log Persistence:** Both `.scroll` and `.log` artifacts are inscribed in `scrolls/` and verified to be valid on disk.
3. **Conversational Synthesis:** A natural-language explanation is generated alongside the machine state, guaranteeing dual human-machine transparency.

Volume 4: Cognitive & Autonomous Systems

Document 02: Episodic Scrolls & Experiential Learning Archive

> ***Homoiconic Trace Recording and Continuous Machine Learning from Past Execution Experiences.***

OmniCode introduces an episodic learning model where every execution—whether initiated through the interactive CLI, the AI Cortex, or test harnesses—is permanently inscribed into a dual-format episodic memory bank:

1. **Native OmniCode Scroll (`.scroll`):** A fully homoiconic, valid 4-Block Turing reel recording the initial prompt, inferred types, synthesized AST, self-healing repairs, and postconditions.
2. **Traditional Telemetry Log (`.log`):** Microsecond-resolution execution timings, structured channel events, and trapped diagnostic failpoints.

---

1. Inscription Topology & File Naming Invariants

All cognitive sessions persist artifacts into the `scrolls/` directory:

```
scrolls/
├── <timestamp>_<seq>_<session_id>.scroll    # Native 4-Block Turing Reel
└── <timestamp>_<seq>_<session_id>.log       # Microsecond Telemetry & Diagnostics
```

Naming Convention Invariants

- **Timestamp Prefix:** Microseconds since UNIX epoch (e.g., `1791492898991747`).
- **Sequence Counter:** 4-digit hexadecimal counter (`0000` to `FFFF`).
- **Session Identifier:** Unique alphanumeric identifier (`sess_...`).

---

2. The Homoiconic Anatomy of an OmniCode `.scroll` File

An OmniCode `.scroll` is not passive JSON or XML; it is executable OmniCode code:

```
# ================================================================================
# BLOCK 1: METADATA BLOCK (THE COGNITIVE SCROLL)
# ================================================================================
Scroll < EpistemicTrace.sess_1791492898991747_0000 > ( Add 5 units to inventory A which contains 12 units ) {
    version: < "1.0.0" >,
    system: < OmniCode.AI.Model >,
    timestamp: < "1791492898991747_0000" >,
    session_id: < "sess_1791492898991747_0000" >,
    execution_duration_micros: < 5804 >,
    cognitive_mode: < NeuralSymbolicTransduction >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK (EXPERIENCE INITIALIZATION)
# ================================================================================
Opening {
    import: [ Core.Arithmetic :: Core.Logic :: Core.Vectors ],
    preconditions: [ +1 ]
}

user_prompt < "Add 5 units to inventory A which contains 12 units" >

# ================================================================================
# BLOCK 3: BODY BLOCK (CHRONICLE OF EXECUTION & RESTORATION)
# ================================================================================
Body [
```

```
    # Immune System Interventions (Restorator):
    repair_entry [ L1: AddImport --> "Injected library imports into Opening block: import: [ Core.Vectors ]" ]

    # Executed Turing Reel Synthesized by AI Model:
    synthesized_reel [
        # ... executable 4-Block module ...
    ]
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK (EPISODIC COMMIT & LEARNING STATUS)
# ================================================================================
Closing {
    commit_status: [ <PROCEED> ],
    verification_proof: [ +1 ],
    learning_ready: [ +1 ]
}
```

3. Episodic Memory Digestion: `AILearner`

The `AILearner` scans the `scrolls/` directory and digests past experiences:

```
let learner = AILearner::new("scrolls");
let summary = learner.digest_experiences()?;
println!("Total Inscribed Episodes: {}", summary.total_episodes);
println!("QuadState Distribution: Proceed: {}, Await: {}, Rollback: {}, Halt: {}",
    summary.proceed_count, summary.await_count, summary.rollback_count, summary.halt_count);
println!("Learned Repairs: {}", summary.total_repairs_learned);
```

CLI Inspection

Developers can inspect the episodic archive at any time from the command line:

```
cargo run -- scrolls
```

Output:

```
================================================================================
  OmniCode AI: Episodic Memory Bank & Experiential Learning Archive
================================================================================

EPISODIC LEARNING SUMMARY:
  • Total Inscribed Episodes:     1
  • State Distribution:           Proceed (+1): 1 | Halt (NULL): 0 | Rollback (-1): 0 | Await (0): 0
  • Restorator Repairs Learned:   1

RECENT EPISODES:
  [ 1] Session: sess_1791492898991747_0000 | State: +1:PROCEED  | Repairs:  1 | Failpoints:  1 | .log: yes
       Prompt: "Add 5 units to inventory A which contains 12 units"
       Scroll: "scrolls/1791492898991747_0000_sess_1791492898991747_0000.scroll"
```

4. Continuous Improvement & Invariant Guarantees

1. **Zero Data Loss:** Every run produces verifiable records; intermittent crashes or rollbacks are fully chronicled for diagnostic autopsy.
2. **Replayability:** A `.scroll` can be fed directly back into `omnicode run` to deterministically reproduce any historical computation.
3. **Model Refinement:** Inscribed repairs provide ground truth training pairs for the next generation of neural-symbolic transducers.

Volume 4: Cognitive & Autonomous Systems

Document 03: The Restorator & Autonomous Self-Healing Diagnostics

> ***An Autonomous Linguistic Immune System That Repairs Syntax, Delimiter Morphology, and Structural Anomalies.***

Conventional compilers are passive and punitive: upon encountering a missing delimiter or an unclosed block, they stop immediately with an error message and exit.

In contrast, OmniCode includes an **active diagnostic immune system**:

1. `Restorator` : Proactively repairs broken source files, balances mismatched delimiters, and scaffolds missing architectural blocks.
2. `OmniDebugger` : Maintains execution snapshots and rolls back environment state when `-1:ROLLBACK` or caveat vetoes are triggered.
3. `DiagnosticLogger` : Tracks failpoints with structured taxonomy (`FP_BOUNDARY_LEAK` , `FP_IMMUTABLE_MUTATION` , `FP_ROLLBACK_TRIGGERED` , etc.).

---

1. Repair Hierarchy & Morphological Preconditions

The Restorator evaluates problems along a 4-level hierarchy:

Level	Kind	Trigger Condition	Autonomous Action
L0	`FormatOnly`	Inconsistent whitespace or line endings	Normalizes indentation and block separation.
L1	`ImportInjection`	Standard keywords used without module imports	Injects `import: [ Core.Arithmetic :: Core.Logic :: Core.Vectors ]` .
L2	`BoundaryBalance`	Unclosed `[` , `<` , `{` delimiters or rank crossing	Closes unclosed delimiters respecting morphological hierarchy.
L3	`BlockScaffold`	Missing `Metadata` , `Opening` , `Body` , or `Closing`	Auto-scaffolds missing 4-block headers and wraps expressions.

---

2. Repair Algorithms & Operational Mechanics

Autonomous Boundary Balancing Algorithm

When a file contains unclosed delimiters (e.g., `state [ +1`), the Restorator scans the token stream:

1. Pushes opening delimiters onto a morphological balance stack.
2. If the stack is non-empty at EOF or before a closing block boundary, pops unclosed delimiters in reverse order.
3. Appends the corresponding closing delimiter (`]` , `>` , `}`) with appropriate rank preservation.

Autonomous Structural 4-Block Scaffolding

If raw code is provided without block demarcations:

```
x [ 10 ]
y [ 20 ]
z [ x + y ]
```

The Restorator automatically transforms it into:

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ================================================================================
Module < AutoRestored > ( Autonomously healed module ) {
    version: < "1.0.0" >,
    system: < OmniCode.Core >
}

# ================================================================================
```

```
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    import: [ Core.Arithmetic :: Core.Logic :: Core.Vectors ],
    preconditions: [ +1 ]
}

x [ 10 ]
y [ 20 ]

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    z [ x + y ]
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [ +1 ],
    commit_status: [ +1 ]
}
```

CLI Command: `omnicode restore`

```
# Preview repairs (dry-run mode)
cargo run -- restore examples/draft_file.omni

# Apply repairs directly to the file
cargo run -- restore examples/draft_file.omni --write
```

Transactional Rollback: `OmniDebugger`

During execution, mutating statements take an environment snapshot before performing writes:

- If the result evaluates to `-1:ROLLBACK` or a caveat `! [ condition ]` evaluates to false, the debugger restores the snapshot.
- **Cyclic Retroaction Protection:** Tracks repeated transitions back to the same snapshot. If cycles exceed the threshold (default: 10), halts execution with `FP_CYCLIC_RETROACTION` to prevent infinite loops.

---

3. Verification & Invariant Proofs

1. **Non-Destructive Guarantee:** In dry-run mode, files are never modified without the `--write` flag.
2. **Linter Re-Verification:** After a file is repaired, the Restorator passes it through `OmniLinter` to prove that the resulting code is 100% compliant before committing to disk.

Volume 4: Cognitive & Autonomous Systems

Document 04: Bare-Metal OS & Cross-Platform Toolchains

> ***Targeting Bare-Metal Silicon, LLVM Compilation, and Hardware Emulators from First Principles.***

OmniCode is designed to function as an Operating System kernel and bare-metal systems language without relying on external C runtimes, libc, or garbage collectors.

1. Supported Bare-Metal Target Architectures

OmniCode's bare-metal layer targets `no_std` environments across three major CPU architectures:

Target Triple	Architecture	Instruction Set	Emulation Platform
`x86_64-unknown-none`	Intel / AMD 64-bit	x86_64 Long Mode	`qemu-system-x86_64`
`aarch64-unknown-none`	ARMv8-A 64-bit	AArch64	`qemu-system-aarch64`
`riscv64gc-unknown-none-elf`	RISC-V 64-bit	RV64GC	`qemu-system-riscv64`

2. Containerized Emulation & Toolchain Infrastructure

To provide a fully reproducible toolchain across Linux, macOS, and Windows without manual host dependency installation, OmniCode provides a standardized container environment via `Dockerfile` and `docker-compose.yml`:

```
# Build and launch the container
docker compose up -d

# Open interactive bash session in the container
docker compose exec omnicode-toolchain bash
```

Container Capabilities

- **Compilers & Assemblers:** `rustc 1.85+`, `clang`, `lld`, `llvm`, `nasm`, `yasm`.
- **Hardware Emulation:** `qemu-system-x86`, `qemu-system-arm`, `qemu-system-misc`.
- **Bootloader Utilities:** `grub-pc-bin`, `xorriso`, `mtools` for generating bootable ISO images.
- **Remote Debugging:** `gdb-multiarch` wired to QEMU GDB stub on port `1234`.

Toolchain Verification Script

The `scripts/check_toolchain.sh` script verifies the status of all compilers, assemblers, and bare-metal targets:

```
./scripts/check_toolchain.sh
```

3. Bare-Metal Memory Model & Boundary Mapping

In bare-metal mode, OmniCode's morphological boundaries map directly to physical hardware memory structures:

- `[...]` (Rigid): Fixed hardware registers (e.g., MMIO ports, control registers `CR0-CR4`), physical page frames, and static arrays.
- `<...>` (Dynamic): Thread stacks, heap allocators, and volatile registers.
- `{...}` (Connotation): CPU privilege rings (`Ring 0` kernel vs `Ring 3` userland), page table permissions (`RWX`), and virtual address space namespaces.

4. Hardware Verification & Safety Invariants

1. **Memory Isolation:** Boundary crossings between `{...}` universes verify page privilege levels before memory is addressed.
2. **Interrupt Safety:** Asymmetric delimiters `{...]` and `[...}` model hardware interrupt service routines:
3. `{...]` : Absorbs hardware interrupts from the external physical environment into a fixed kernel status block.

4. `[...}` : Emits fixed command words out to peripheral device buses.

<TRACK 5> : {Infrastructure, Operations & Archives}/docs/04_cognitive_and_autonomous_systems/05_core_standard_libraries.md MARKDOWN / DOC 64 LINES 333 WORDS 2.7 KB

Volume 4: Cognitive & Autonomous Systems

Document 05: Core Standard Library Architecture

> ***Axiomatic Modules Grounded in Balanced Ternary Logic, Safe Arithmetic, and Directional Vectors.***

The OmniCode Standard Library lives under `lib/core/` and provides the core algebraic foundation for the language. Every standard library file is written in canonical 4-Block OmniCode.

1. Standard Library Modules & Topology

Module Name	Source File	Description
`Core.Arithmetic`	`lib/core/arithmetic.omni`	Balanced ternary arithmetic, addition, subtraction, multiplication, and non-zero division.
`Core.Logic`	`lib/core/logic.omni`	Ternary Kleene and Lukasiewicz logic gates (AND, OR, NOT, XOR, and Consensual Agreement).
`Core.Vectors`	`lib/core/vectors.omni`	Directional coordinate vectors across Being, Causality, and Scale axes with caveat checks.

2. Core Mathematical & Logical Specifications

`Core.Arithmetic`

Guarantees quad-state mathematical correctness:

- **Addition:** $a + b \implies$ produces sum when both operands are valid; returns `NULL` on overflow or trap.
- **Safe Division:** Division by zero returns `NULL:HALT` without raising an exception:

```
omni divisor != 0 ! [ divisor = 0 --> NULL ]
```

`Core.Logic`

Implements full balanced ternary logic gates:

- **Consensual AND (`::`):** $$\text{AND}(a, b) = \min(a, b)$$
- **Ternary OR (`|`):** $$\text{OR}(a, b) = \max(a, b)$$
- **Ternary NOT (`--`):** $$\text{NOT}(x) = -x$$
- **Consensual Agreement:** Returns `+1` if both agree, `-1` if in contradiction, and `0` if either is awaiting consensus.

`Core.Vectors`

Implements 3D directional physics:

- **Causality (`-->` , `<--`):** Drives causal transformations forward and retroactively verifies feedback.
- **Scale (`^--` , `v--`):** Manages abstraction transitions between primitive memory buffers and high-level types.
- **Caveat Pipeline (`!`):** Evaluates guard conditions prior to committing state transitions:

```
omni a --> b ! [ precondition = +1 ]
```

3. Invariant Proofs & Resolution Guarantees

1. **Self-Contained Execution:** All standard library functions are verified in `tests/test_failpoints.rs` (`test_lib_core_vectors` , `test_svo_arithmetic_keywords_and_infer_operator`).
2. **Deterministic Resolution:** The compiler automatically resolves `import: [ Core.Arithmetic :: Core.Logic :: Core.Vectors ]` directly to `lib/core/` .

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/systemd/omnicode-web.service

SERVICE | 22 LINES | 34 WORDS | 0.5 KB

```
[Unit]
Description=OmniCode Cognitive Studio Web & REST API Server (CreativeWorkzStudio LLC)
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/OmniCode
ExecStart=/home/ubuntu/OmniCode/target/release/omnicode gui --port 8080
Restart=always
RestartSec=5s
Environment=RUST_LOG=info
Environment=OMNICODE_OFFLINE=1
Environment=OMNICODE_HOST=0.0.0.0

# Security and resource sandboxing
LimitNOFILE=65536
NoNewPrivileges=true

[Install]
WantedBy=multi-user.target
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/docker/Dockerfile TEXT 178 LINES 947 WORDS 7.8 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( OmniCode Bare-Metal OS & Multi-Architecture Toolchain Specification )
# ================================================================================
# Image:       omnicode-toolchain:latest
# Base OS:     Debian 12 (Bookworm) via official Rust 1.85 slim image
# Maintainer:  ProfessorSeanEX
# Architecture: Multi-target (x86_64, aarch64, riscv64gc)
#
# PURPOSE & COGNITIVE RATIONALE:
#   This container establishes a completely reproducible, hermetic compilation,
#   emulation, and cognitive execution environment for the OmniCode language.
#   It serves two distinct computational tiers:
#     1. Low-Level Bare-Metal Systems: no_std cross-compilation, bootloader ISO
#        packaging, NASM assembly, and multi-architecture QEMU hardware emulation.
#     2. High-Level Cognitive & Language Runtime: The Rust 2024 core engine,
#        interactive REPL, AI Cortex, Episodic Scroll logger, and self-healing Restorator.
#
# WHY EXTENSIVE COMMENTING MATTERS FOR AI & DEVELOPERS:
#   - For AI Agents (e.g., AICortex, Pair-Programming Assistants):
#     Code comments serve as semantic invariant anchors. When an AI parses this
#     Dockerfile, comments provide explicit causal rationales ("Why is xorriso here?
#     Because GRUB El Torito ISOs require it without loop mounts"). This prevents
#     the AI from hallucinating dependencies or accidentally stripping essential
#     cross-compilers during automated refactoring.
#   - For Human Engineers:
#     Systems programming toolchains conflate dozens of specialized CLI utilities.
#     Detailed comments eliminate guesswork, clarify port assignments (e.g., 1234
#     for QEMU GDB), and explain target triples for bare-metal silicon.
# ================================================================================

FROM rust:1.85-slim-bookworm AS toolchain-base

LABEL maintainer="ProfessorSeanEX"
LABEL description="OmniCode Bare-Metal Toolchain: Rust Core, LLVM/Clang, QEMU Emulation, and ISO Bootloaders"
LABEL version="0.1.0"

# Non-interactive frontend prevents apt from prompting for timezone/keyboard input during build
ENV DEBIAN_FRONTEND=noninteractive

# Standardize Cargo and Rustup directories into global system locations
ENV CARGO_HOME=/usr/local/cargo
ENV RUSTUP_HOME=/usr/local/rustup
ENV PATH="${CARGO_HOME}/bin:${PATH}"

# OmniCode runtime environment defaults
ENV RUST_BACKTRACE=1
ENV OMNICODE_SCROLL_DIR=/workspace/scrolls
ENV OMNICODE_LIB_DIR=/workspace/lib


# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( System Dependencies, Cross-Compilers, Assemblers & Emulation Prerequisites )
# ================================================================================
# RATIONALE FOR PACKAGE GROUPINGS:
#   1. build-essential, clang, lld, llvm:
#      - Clang/LLVM provides the primary C ABI transpilation and LLVM IR compilation backend.
#      - LLD is LLVM's ultra-fast linker, indispensable for bare-metal custom memory linker scripts.
#   2. nasm, yasm:
#      - Assemblers for 16-bit real mode, 32-bit protected mode bootloaders, and 64-bit entry stubs.
#   3. qemu-system-x86, qemu-system-arm, qemu-system-misc:
#      - Hardware emulators covering all three tier-1 OmniCode targets:
#        * qemu-system-x86: Intel/AMD x86_64 PC hardware (Q35/i440FX chipsets).
#        * qemu-system-arm: ARMv8-A Cortex bare-metal boards (virt, raspi).
#        * qemu-system-misc: RISC-V 64-bit hardware (virt board, RV64GC).
#   4. xorriso, grub-pc-bin, grub-common, mtools:
#      - Bootloader ISO image authoring. Allows generating bootable hybrid El Torito ISOs
#        and Multiboot2/UEFI media without needing root loop-device privileges.
#   5. gdb-multiarch:
#      - Cross-architecture debugger capable of stepping through x86_64, aarch64, and
#        riscv64 instructions when attached to QEMU's GDB stub on port 1234.
#   6. ripgrep, jq, file, tree, less:
#      - Diagnostic inspection tools. `file` inspects ELF kernel binaries; `jq` and `ripgrep`
#        parse JSON AST dumps and episodic trace telemetry.
#   7. nodejs, npm:
#      - Required for building, linting, and packaging the OmniCode VS Code extension
```

```
#        via `@vscode/vsce` in editors/vscode/.
# ==============================================================================

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    clang \
    lld \
    llvm \
    nasm \
    yasm \
    qemu-system-x86 \
    qemu-system-arm \
    qemu-system-misc \
    xorriso \
    grub-pc-bin \
    grub-common \
    mtools \
    gdb-multiarch \
    curl \
    git \
    pkg-config \
    ca-certificates \
    ripgrep \
    jq \
    file \
    tree \
    less \
    nodejs \
    npm \
    && rm -rf /var/lib/apt/lists/*


# ==============================================================================
# BLOCK 3: BODY BLOCK
# ( Bare-Metal Rust Targets, LLVM Tools, and Global Binary Compilation )
# ==============================================================================
# RATIONALE FOR RUSTUP COMPONENTS & TARGETS:
#   1. rust-src:
#      - Contains the source code of the Rust standard library (core, alloc, compiler_builtins).
#      - Required by cargo `-Z build-std` when compiling custom bare-metal OS kernels.
#   2. llvm-tools-preview:
#      - Ships llvm-objdump, llvm-objcopy, llvm-nm, and llvm-size for binary introspection.
#   3. Bare-Metal Target Triples:
#      - `x86_64-unknown-none`: 64-bit x86 bare-metal kernel target (no OS, no libc).
#      - `aarch64-unknown-none`: 64-bit ARM bare-metal target (no OS, no libc).
#      - `riscv64gc-unknown-none-elf`: 64-bit RISC-V with General Purpose + Compressed instructions.
# ==============================================================================

RUN rustup component add rust-src llvm-tools-preview rustfmt clippy && \
    rustup target add x86_64-unknown-none && \
    rustup target add aarch64-unknown-none && \
    rustup target add riscv64gc-unknown-none-elf

# Establish standardized container workspace directory
WORKDIR /workspace

# Copy manifest, source tree, libraries, and tests to pre-compile the release binary
# into /usr/local/cargo/bin so `omnicode` is immediately available in the system PATH
COPY Cargo.toml Cargo.lock /workspace/
COPY src/ /workspace/src/
COPY lib/ /workspace/lib/
COPY examples/ /workspace/examples/
COPY tests/ /workspace/tests/

# Build and install the canonical OmniCode binary globally into the image
RUN cargo build --release && \
    cargo test --release && \
    cp /workspace/target/release/omnicode /usr/local/bin/omnicode && \
    chmod +x /usr/local/bin/omnicode

# Comprehensive toolchain verification during build
RUN echo "=== Verifying OmniCode Multi-Target Toolchain ===" && \
    omnicode --version && \
    rustc --version && \
    cargo --version && \
    clang --version && \
    lld --version && \
    nasm -v && \
    qemu-system-x86_64 --version && \
    qemu-system-aarch64 --version && \
    qemu-system-riscv64 --version && \
```

```
    gdb-multiarch --version && \
    node --version && \
    npm --version


# ============================================================================
# BLOCK 4: CLOSING BLOCK
# ( Runtime Verification, Environment Seal & Default Entrypoint )
# ============================================================================
# ENTRYPOINT & INVARIANTS:
#   - Default command drops into an interactive bash shell where developers or
#     AI subagents can immediately invoke `omnicode repl`, `omnicode run`,
#     `omnicode ai`, `omnicode scrolls`, or run QEMU bare-metal kernel sessions.
#   - In docker-compose, this workspace is overlaid with the host volume mount
#     to provide live two-way code synchronization.
# ============================================================================

# Default command: interactive bash shell with full toolchain availability
CMD ["bash"]
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/docker/docker-compose.yml YML | 113 LINES | 650 WORDS | 5.7 KB

```yaml
# ==============================================================================
# BLOCK 1: METADATA BLOCK
# ( Container Orchestration Identity, Infrastructure Manifest & Cognitive Intent )
# ==============================================================================
# Project:      OmniCode Language Engine & Bare-Metal OS
# Compose File: docker-compose.yml
# Service:      omnicode-toolchain
# Purpose:      Multi-tier development, bare-metal hardware emulation, and cognitive execution
#
# WHY EXTENSIVE COMMENTING MATTERS FOR AI & DEVELOPERS:
#   - For AI Autonomous Agents:
#     When an AI inspects orchestration files, bare configuration directives (e.g.
#     `1234:1234`) lack intent. Detailed comments explain the semantic contract:
#     Port 1234 connects to QEMU's GDB stub; modifying it breaks kernel debugging.
#     Named volume `target-cache` prevents cross-compilation thrashing between the
#     host and Linux container. These comments serve as immutable guardrails.
#   - For Human Collaborators:
#     Eliminates onboarding friction. Developers immediately know how to attach an
#     interactive REPL, connect a debugger to a bootloader, or run the AI cognitive loop.
# ==============================================================================

# ==============================================================================
# BLOCK 2: OPENING BLOCK
# ( Service Topology, Container Preconditions & Interactive Shell Configuration )
# ==============================================================================
services:
  omnicode-toolchain:
    build:
      context: .
      dockerfile: Dockerfile
    image: omnicode-toolchain:latest
    container_name: omnicode-toolchain

    # Interactive TTY configuration:
    # `stdin_open: true` (-i) and `tty: true` (-t) keep STDIN open and allocate
    # a pseudo-TTY, enabling interactive `omnicode repl` and terminal debugging sessions.
    stdin_open: true
    tty: true

    # Execution root directory inside the container
    working_dir: /workspace

# ==============================================================================
# BLOCK 3: BODY BLOCK
# ( Volume Topology, Network Port Routing & Environment Invariant Configuration )
# ==============================================================================
    # VOLUME MOUNTING STRATEGY & EXPLANATION:
    #   1. `.:/workspace`:
    #      Two-way live bind mount between host project root and container.
    #      Edits in host IDEs (VS Code, Cursor, Antigravity) appear immediately inside
    #      the container, and generated `.scroll` episodic memory logs are saved to host.
    #   2. `cargo-cache:/usr/local/cargo/registry`:
    #      Named persistent volume caching downloaded crates.io index and packages,
    #      preventing redundant downloads across container restarts.
    #   3. `cargo-git:/usr/local/cargo/git`:
    #      Named persistent volume caching git dependencies fetched by Cargo.
    #   4. `target-cache:/workspace/target`:
    #      Named persistent volume isolating Linux ELF build artifacts from the host OS.
    #      CRITICAL: Prevents target directory collisions if the host machine is macOS
    #      or Windows, and dramatically speeds up compilation by avoiding 9p/FUSE overhead.
    volumes:
      - .:/workspace
      - cargo-cache:/usr/local/cargo/registry
      - cargo-git:/usr/local/cargo/git
      - target-cache:/workspace/target

    # PORT MAPPING TOPOLOGY:
    #   - 1234: QEMU Remote GDB Debugging Stub.
    #           When launching a bare-metal kernel with `qemu-system-x86_64 -s -S`,
    #           QEMU halts execution at the first instruction and listens on TCP port 1234.
    #           Attach from host or container with `gdb-multiarch -ex "target remote localhost:1234"`.
    #   - 5900: QEMU VNC Graphical Framebuffer.
    #           Bare-metal graphical displays rendered via VESA/GOP or virtio-gpu are
    #           streamed over VNC. Connect any VNC viewer to `localhost:5900`.
    #   - 8080: OmniCode Cognitive Daemon & Language Server.
```

```
    #           Reserved for future HTTP/WebSocket language server or telemetry visualization.
    ports:
      - "1234:1234" # QEMU GDB remote debugging socket
      - "5900:5900" # QEMU VNC graphical framebuffer display
      - "8080:8080" # OmniCode Cognitive Daemon / Web Inspector

    # RUNTIME ENVIRONMENT INVARIANTS:
    #   - RUST_BACKTRACE=1: Produces full symbolic stack traces upon error or panic.
    #   - OMNICODE_SCROLL_DIR: Directs episodic cognitive logs to the shared workspace.
    #   - OMNICODE_LIB_DIR: Resolves built-in standard library modules (Core.Arithmetic, etc.).
    environment:
      - RUST_BACKTRACE=1
      - OMNICODE_SCROLL_DIR=/workspace/scrolls
      - OMNICODE_LIB_DIR=/workspace/lib

    # Default command drops into an interactive bash shell ready for commands:
    # e.g., `omnicode repl`, `omnicode run <file>`, `omnicode ai "<prompt>"`, or `./scripts/check_toolchain.sh`
    command: bash

# ==============================================================================
# BLOCK 4: CLOSING BLOCK
# ( Volume Persistence Declarations & Invariant Storage Lifecycle )
# ==============================================================================
# NAMED VOLUME DECLARATIONS:
#   These volumes persist independently of container destruction:
#   - Run `docker compose down` to stop the container without losing caches.
#   - Run `docker compose down -v` to explicitly purge build caches if clean builds are required.
volumes:
  cargo-cache:
    labels:
      description: "Persistent cache for Cargo crates.io package registry"
  cargo-git:
    labels:
      description: "Persistent cache for Cargo git repositories"
  target-cache:
    labels:
      description: "Isolated persistent build cache for target/ binary compilation"
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/check_toolchain.sh

SH | 57 LINES | 227 WORDS | 2.5 KB

```sh
#!/usr/bin/env bash
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Toolchain Diagnostics Identity & Environment Manifest )
# ================================================================================
set -e

echo -e "\x1b[1;36m=============================================================================================\x1b[0m"
echo -e "\x1b[1;36m  OmniCode Toolchain Diagnostics & Verification Harness\x1b[0m"
echo -e "\x1b[1;36m=============================================================================================\x1b[0m\n"

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Dependency Check Function & Invariant Preconditions )
# ================================================================================
check_tool() {
    local name="$1"
    local cmd="$2"
    if command -v "$cmd" >/dev/null 2>&1; then
        echo -e "  \x1b[1;32m✓\x1b[0m $name: \x1b[33m$($cmd --version 2>&1 | head -n 1)\x1b[0m"
    else
        echo -e "  \x1b[1;31m✗\x1b[0m $name is NOT installed ($cmd missing)"
        return 1
    fi
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Core Compilers, Assemblers, Emulators & Bare-Metal Verification )
# ================================================================================
echo -e "\x1b[1;34m[1/3] Core Compilers & Assemblers:\x1b[0m"
check_tool "Rust Compiler" rustc
check_tool "Cargo" cargo
check_tool "Clang" clang
check_tool "LLD Linker" lld
check_tool "NASM Assembler" nasm

echo -e "\n\x1b[1;34m[2/3] Hardware Emulation & Bootloaders:\x1b[0m"
check_tool "QEMU x86_64" qemu-system-x86_64
check_tool "Xorriso" xorriso

echo -e "\n\x1b[1;34m[3/3] Rust Bare-Metal Target Verification:\x1b[0m"
TARGETS=$(rustup target list --installed)
for target in "x86_64-unknown-none" "aarch64-unknown-none" "riscv64gc-unknown-none-elf"; do
    if echo "$TARGETS" | grep -q "$target"; then
        echo -e "  \x1b[1;32m✓\x1b[0m Installed target: \x1b[33m$target\x1b[0m"
    else
        echo -e "  \x1b[1;33m!\x1b[0m Missing bare-metal target: $target (Install with 'rustup target add $target')"
    fi
done

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Final Status Summary & Exit Verification )
# ================================================================================
echo -e "\n\x1b[1;32m✓ Toolchain verification complete.\x1b[0m"
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/deploy_oracle_cloud.sh

SH | 160 LINES | 721 WORDS | 7.0 KB

```sh
#!/usr/bin/env bash
# ==============================================================================
# CreativeWorkzStudio LLC - OmniCode Platform
# Oracle Cloud Ubuntu VM Turnkey Provisioning & Deployment Script
# ==============================================================================
# Turnkey setup script for deploying OmniCode 24/7 on an Oracle Cloud Compute Instance:
# 1. Installs system build prerequisites, Rust/Cargo compiler, and git.
# 2. Configures OS firewall & iptables to allow Port 8080 (Web Studio & Cognitive API).
# 3. Builds the optimized OmniCode release binary (`cargo build --release`).
# 4. Installs and registers the systemd 24/7 service (`omnicode-web.service`).
# 5. Verifies service health and edge availability.
#
# Usage (run as root or with sudo on the Oracle Cloud VM):
#   sudo ./scripts/deploy_oracle_cloud.sh
# ==============================================================================

set -e

# Visual styling
BOLD="\033[1m"
GREEN="\033[0;32m"
BLUE="\033[0;34m"
CYAN="\033[0;36m"
YELLOW="\033[1;33m"
RED="\033[0;31m"
NC="\033[0m"

echo -e "${BOLD}${CYAN}================================================================================${NC}"
echo -e "${BOLD}${CYAN}  🌌 CreativeWorkzStudio LLC — OmniCode Platform                              ${NC}"
echo -e "${BOLD}${CYAN}  Oracle Cloud VM Turnkey Deployment & Systemd Provisioner                    ${NC}"
echo -e "${BOLD}${CYAN}================================================================================${NC}"

if [ "$EUID" -ne 0 ]; then
    echo -e "${RED}[!] Error: This script must be run as root or via sudo.${NC}"
    echo -e "    Please run: sudo $0"
    exit 1
fi

REAL_USER="${SUDO_USER:-$USER}"
REAL_HOME="$(eval echo "~$REAL_USER")"
SCRIPT_DIR="$(cd -P "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
BASE_DIR="$(cd "$SCRIPT_DIR/.." && pwd)"

echo -e "${BLUE}[*] Provisioning for host user: ${GREEN}$REAL_USER${NC}"
echo -e "${BLUE}[*] OmniCode base directory:    ${GREEN}$BASE_DIR${NC}"

# Detect Architecture
ARCH="$(uname -m)"
echo -e "${BLUE}[*] Target CPU architecture:    ${GREEN}$ARCH${NC}"

# ------------------------------------------------------------------------------
# STEP 1: System Packages and Build Essentials
# ------------------------------------------------------------------------------
echo ""
echo -e "${BOLD}${BLUE}[1/5] Updating packages and installing build prerequisites...${NC}"
export DEBIAN_FRONTEND=noninteractive
apt-get update -y
apt-get install -y \
    build-essential \
    curl \
    git \
    pkg-config \
    libssl-dev \
    iptables \
    iptables-persistent \
    netfilter-persistent \
    ca-certificates

# Ensure Rust & Cargo are installed for the target user
if ! sudo -u "$REAL_USER" command -v cargo >/dev/null 2>&1; then
    echo -e "${BLUE}[*] Rust compiler not found. Installing Rust toolchain via rustup...${NC}"
    sudo -u "$REAL_USER" curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sudo -u "$REAL_USER" sh -s -- -y
    echo -e "${GREEN}[+] Rust toolchain installed.${NC}"
else
    echo -e "${GREEN}[+] Rust toolchain already available: $(sudo -u "$REAL_USER" cargo --version)${NC}"
```

```
fi

# ------------------------------------------------------------------------------
# STEP 2: Configure Oracle Cloud Firewall & iptables
# ------------------------------------------------------------------------------
echo ""
echo -e "${BOLD}${BLUE}[2/5] Configuring OS Firewall & iptables for Port 8080...${NC}"
PORTS=(22 8080)

for PORT in "${PORTS[@]}"; do
    if ! iptables -C INPUT -p tcp --dport "$PORT" -j ACCEPT 2>/dev/null; then
        echo -e "${BLUE}[*] Opening TCP Port $PORT in iptables...${NC}"
        iptables -I INPUT 1 -p tcp --dport "$PORT" -j ACCEPT
    else
        echo -e "${GREEN}[+] TCP Port $PORT already allowed in iptables.${NC}"
    fi
done

# Persist firewall rules
netfilter-persistent save || true

# Check UFW if active
if command -v ufw >/dev/null 2>&1 && ufw status 2>/dev/null | grep -q "Status: active"; then
    echo -e "${BLUE}[*] UFW firewall active. Permitting port 8080...${NC}"
    ufw allow 22/tcp
    ufw allow 8080/tcp
    ufw reload
fi
echo -e "${GREEN}[+] OS firewall configured for OmniCode Web & Cognitive Studio (Port 8080).${NC}"

# ------------------------------------------------------------------------------
# STEP 3: Compile OmniCode Release Binary
# ------------------------------------------------------------------------------
echo ""
echo -e "${BOLD}${BLUE}[3/5] Compiling OmniCode release binary with Cargo...${NC}"
export PATH="$REAL_HOME/.cargo/bin:$PATH"
sudo -u "$REAL_USER" env PATH="$REAL_HOME/.cargo/bin:$PATH" cargo build --release --manifest-path "$BASE_DIR/Cargo.toml"
echo -e "${GREEN}[+] OmniCode release binary built at: $BASE_DIR/target/release/omnicode${NC}"

# ------------------------------------------------------------------------------
# STEP 4: Install and Enable Systemd Service
# ------------------------------------------------------------------------------
echo ""
echo -e "${BOLD}${BLUE}[4/5] Registering and enabling systemd 24/7 service...${NC}"
SERVICE_SRC="$BASE_DIR/systemd/omnicode-web.service"
SERVICE_DEST="/etc/systemd/system/omnicode-web.service"

sed -e "s|/home/ubuntu/OmniCode|$BASE_DIR|g" \
    -e "s|User=ubuntu|User=$REAL_USER|g" \
    "$SERVICE_SRC" > "$SERVICE_DEST"

systemctl daemon-reload
systemctl enable omnicode-web.service
systemctl restart omnicode-web.service
echo -e "${GREEN}[+] omnicode-web.service installed and restarted.${NC}"

# ------------------------------------------------------------------------------
# STEP 5: Verification and Endpoint Summary
# ------------------------------------------------------------------------------
echo ""
echo -e "${BOLD}${BLUE}[5/5] Verifying service health and endpoints...${NC}"
sleep 2

STATUS=$(systemctl is-active omnicode-web.service || echo "failed")
PUBLIC_IP="$(curl -s -4 https://ifconfig.me || curl -s -4 https://api.ipify.org || echo "ORACLE_VM_PUBLIC_IP")"

echo ""
echo -e "${BOLD}${GREEN}================================================================================${NC}"
echo -e "${BOLD}${GREEN}  🎉 OMNICODE ORACLE CLOUD 24/7 DEPLOYMENT COMPLETE!                            ${NC}"
echo -e "${BOLD}${GREEN}  CreativeWorkzStudio LLC                                                       ${NC}"
echo -e "${BOLD}${GREEN}================================================================================${NC}"
echo -e "Service Status:         ${BOLD}${GREEN}$STATUS${NC}"
echo -e "Public Server IP:       ${BOLD}${YELLOW}$PUBLIC_IP${NC}"
echo -e "Web Studio & API:       ${BOLD}${CYAN}http://$PUBLIC_IP:8080${NC}"
echo ""
echo -e "${BOLD}Management Commands:${NC}"
echo -e "  Check service status:  ${BLUE}sudo systemctl status omnicode-web${NC}"
echo -e "  Restart service:       ${BLUE}sudo systemctl restart omnicode-web${NC}"
echo -e "  View live logs:        ${BLUE}sudo journalctl -u omnicode-web -f${NC}"
echo ""
echo -e "${BOLD}Cloudflare Integration:${NC}"
```

```
echo -e "  To route edge traffic through Cloudflare, launch the tunnel connector:"
echo -e "  ${BLUE}$BASE_DIR/scripts/setup_cloudflare_tunnel.sh --quick${NC}"
echo -e "${BOLD}${GREEN}================================================================================${NC}"
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/oci_security_list_helper.sh

SH | 87 LINES | 474 WORDS | 4.1 KB

```sh
#!/usr/bin/env bash
# ==============================================================================
# CreativeWorkzStudio LLC - OmniCode Platform
# Oracle Cloud VCN Ingress/Egress & Firewall Configuration Helper
# ==============================================================================
# Helper script to manage VM OS firewall (iptables) and OCI VCN Security List
# for OmniCode Studio (port 8080).
#
# Requirements:
#   1. VM OS Firewall: iptables + netfilter-persistent (Already configured)
#   2. OCI Cloud Firewall: OCI CLI (installed at /usr/local/bin/oci) or OCI Console
# ==============================================================================

set -e

BOLD="\033[1m"
GREEN="\033[0;32m"
BLUE="\033[0;34m"
CYAN="\033[0;36m"
YELLOW="\033[1;33m"
NC="\033[0m"

echo -e "${BOLD}${CYAN}================================================================================${NC}"
echo -e "${BOLD}${CYAN}  🛡️  Oracle Cloud VM Firewall & Ingress/Egress Helper                ${NC}"
echo -e "${BOLD}${CYAN}  CreativeWorkzStudio LLC — OmniCode Platform (Port 8080)             ${NC}"
echo -e "${BOLD}${CYAN}================================================================================${NC}"

# 1. Verify OS-level iptables rules
echo -e "\n${BLUE}[1/3] Checking OS-level iptables firewall rules for Port 8080...${NC}"
if sudo iptables -L INPUT -n | grep -q "dpt:8080"; then
    echo -e "${GREEN}[✓] Port 8080 is ACCEPT in iptables INPUT chain.${NC}"
else
    echo -e "${YELLOW}[*] Adding Port 8080 to iptables INPUT chain...${NC}"
    sudo iptables -I INPUT 6 -p tcp --dport 8080 -j ACCEPT
fi

# Ensure rules are saved persistently
if command -v netfilter-persistent >/dev/null 2>&1; then
    sudo netfilter-persistent save >/dev/null 2>&1
    echo -e "${GREEN}[✓] iptables rules saved persistently via netfilter-persistent.${NC}"
fi

# 2. Check OCI CLI installation
echo -e "\n${BLUE}[2/3] Checking Oracle Cloud Infrastructure CLI (oci-cli)...${NC}"
if command -v oci >/dev/null 2>&1; then
    OCI_VERSION=$(oci --version)
    echo -e "${GREEN}[✓] OCI CLI is installed at: $(which oci) (version: $OCI_VERSION)${NC}"
else
    echo -e "${YELLOW}[!] OCI CLI not found. Run setup on VM: sudo pip3 install oci-cli${NC}"
fi

# 3. Guidance for OCI VCN Security List (Cloud Layer)
echo -e "\n${BLUE}[3/3] Oracle Cloud VCN Ingress Rule Instructions (Cloud Hypervisor Layer):${NC}"
cat << 'EOF'

In Oracle Cloud Infrastructure (OCI), incoming traffic must pass through TWO layers:
  Layer 1: Linux OS Firewall (iptables) -> ALREADY CONFIGURED & PERSISTED (Port 8080 OPEN)
  Layer 2: OCI VCN Security List        -> Requires Cloud Ingress Rule

Option A: Via OCI Web Console (1-Minute Setup)
  1. Open Oracle Cloud Console (https://cloud.oracle.com)
  2. Navigate to: Networking > Virtual Cloud Networks > Select your VCN
  3. Under Resources, click "Security Lists" > Click "Default Security List for [VCN]"
  4. Click "Add Ingress Rules":
     - Source CIDR:            0.0.0.0/0
     - IP Protocol:            TCP
     - Destination Port Range: 8080
     - Description:           OmniCode Studio & Interactive Terminal GUI
  5. Click "Add Ingress Rules" to save.

Option B: Via OCI CLI (Command Line on VM)
  If you have configured OCI API credentials (`oci setup config`):
  Run:
    oci network security-list update \
      --security-list-id <YOUR_SECURITY_LIST_OCID> \
```

```
       --ingress-security-rules file://<(echo '[{"source":"0.0.0.0/0","protocol":"6","tcpOptions":{"destinationPortRange":
{"max":8080,"min":8080}},"description":"OmniCode Studio"}]')

Option C: Via Cloudflare Zero Trust Tunnel (Zero Open Ports Required)
  Because cloudflared is already running on the VM, you can route:
  - Hostname: studio.omnicodestudio.com -> HTTP://localhost:8080
  This routes traffic securely via Cloudflare edge without needing any open ports in OCI VCN!
EOF

echo -e "\n${BOLD}${GREEN}================================================================================${NC}"
echo -e "${BOLD}${GREEN}  ✓ Firewall verification & guidance completed.                        ${NC}"
echo -e "${BOLD}${GREEN}================================================================================${NC}"
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/omnicode_gui.sh SH 16 LINES 53 WORDS 0.5 KB

```sh
#!/usr/bin/env bash
# ============================================================================
# CreativeWorkzStudio LLC — OmniCode Desktop GUI Studio Launcher
# ============================================================================
set -e

SCRIPT_DIR="$(cd -P "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
BASE_DIR="$(cd "$SCRIPT_DIR/.." && pwd)"

if command -v python3 >/dev/null 2>&1; then
    exec python3 "$BASE_DIR/gui/desktop_app.py" "$@"
else
    echo "Python 3 is required to launch the OmniCode Desktop GUI."
    exit 1
fi
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/package_native.sh

SH | 125 LINES | 395 WORDS | 4.2 KB

```
#!/usr/bin/env bash
# ================================================================================
# CreativeWorkzStudio LLC - OmniCode Platform
# Automated Native Terminal & Language OS Distribution Packager
# ================================================================================
# Compiles the optimized release binary and bundles a standalone native
# distribution archive with zero runtime dependencies.
#
# Output:
#   dist/omnicode-terminal-linux-x86_64.tar.gz
#   dist/omnicode-terminal-latest.tar.gz
# ================================================================================

set -e

SCRIPT_DIR="$(cd -P "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
if [ -f "$SCRIPT_DIR/../../Cargo.toml" ]; then
    BASE_DIR="$(cd "$SCRIPT_DIR/../.." && pwd)"
else
    BASE_DIR="$(cd "$SCRIPT_DIR/.." && pwd)"
fi
DIST_DIR="$BASE_DIR/infra/dist"
STAGING_DIR="/tmp/omnicode_dist_staging"

# ANSI styling
BOLD="\033[1m"
GREEN="\033[0;32m"
BLUE="\033[0;34m"
CYAN="\033[0;36m"
NC="\033[0m"

echo -e "${BOLD}${CYAN}================================================================================${NC}"
echo -e "${BOLD}${CYAN}  📦 OmniCode Native Distribution Packager                                    ${NC}"
echo -e "${BOLD}${CYAN}  CreativeWorkzStudio LLC — Standalone Native OS Shell                        ${NC}"
echo -e "${BOLD}${CYAN}================================================================================${NC}"

# 1. Compile release binary locally
echo -e "\n${BLUE}[1/4] Compiling optimized release binary with cargo...${NC}"
(cd "$BASE_DIR" && cargo build --release)

# 2. Prepare staging directory
echo -e "\n${BLUE}[2/4] Assembling standalone distribution files...${NC}"
rm -rf "$STAGING_DIR"
mkdir -p "$STAGING_DIR/bin"
mkdir -p "$STAGING_DIR/lib/core"
mkdir -p "$STAGING_DIR/examples"
mkdir -p "$STAGING_DIR/editors"
mkdir -p "$DIST_DIR"

# Copy binary
cp "$BASE_DIR/target/release/omnicode" "$STAGING_DIR/bin/omnicode"
chmod +x "$STAGING_DIR/bin/omnicode"

# Copy standard libraries and manifest
cp -r "$BASE_DIR/lib/core/"*.omni "$STAGING_DIR/lib/core/"
cp "$BASE_DIR/manifest.omni" "$STAGING_DIR/"
cp -r "$BASE_DIR/examples/"*.omni "$STAGING_DIR/examples/"

# Copy VS Code extension if available
if [ -d "$BASE_DIR/apps/editors/vscode" ]; then
    cp "$BASE_DIR/apps/editors/vscode/"*.vsix "$STAGING_DIR/editors/" 2>/dev/null || true
fi

# Create turnkey install script
cat << 'EOF' > "$STAGING_DIR/install.sh"
#!/usr/bin/env bash
# CreativeWorkzStudio LLC - OmniCode Standalone Installer
set -e

DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

echo "Installing OmniCode Native Terminal..."

# Target install directory
if [ "$EUID" -eq 0 ]; then
```

```
    INSTALL_DIR="/usr/local/bin"
else
    INSTALL_DIR="$HOME/.local/bin"
    mkdir -p "$INSTALL_DIR"
fi

cp "$DIR/bin/omnicode" "$INSTALL_DIR/omnicode"
chmod +x "$INSTALL_DIR/omnicode"

echo "✓ OmniCode installed successfully to $INSTALL_DIR/omnicode"
echo "Testing installation:"
"$INSTALL_DIR/omnicode" demo

echo ""
echo "🎉 OmniCode Native Shell is ready! Type 'omnicode' to enter interactive terminal."
EOF
chmod +x "$STAGING_DIR/install.sh"

# Create Distribution README
cat << 'EOF' > "$STAGING_DIR/README.md"
# OmniCode Native Terminal & Language Core
CreativeWorkzStudio LLC — Autonomous Neural-Symbolic Intelligence

## Quick Start
Run turnkey installer:
  ./install.sh

Or run binary directly:
  ./bin/omnicode

Launch interactive userspace shell:
  ./bin/omnicode terminal
EOF

# 3. Create compressed archives
echo -e "\n${BLUE}[3/4] Creating standalone distribution archives in dist/...${NC}"
ARCHIVE_NAME="omnicode-terminal-linux-x86_64.tar.gz"
(cd "/tmp" && tar -czf "$DIST_DIR/$ARCHIVE_NAME" -C "$STAGING_DIR" .)
cp "$DIST_DIR/$ARCHIVE_NAME" "$DIST_DIR/omnicode-terminal-latest.tar.gz"

# 4. Report
echo -e "\n${BLUE}[4/4] Package Verification:${NC}"
ls -lh "$DIST_DIR/$ARCHIVE_NAME"
sha256sum "$DIST_DIR/$ARCHIVE_NAME"

echo -e "\n${BOLD}${GREEN}================================================================================${NC}"
echo -e "${BOLD}${GREEN}  ✅ Standalone Native Distribution Package Created Successfully!     ${NC}"
echo -e "${BOLD}${GREEN}  Output: $DIST_DIR/$ARCHIVE_NAME                                     ${NC}"
echo -e "${BOLD}${GREEN}================================================================================${NC}"
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/setup_cloudflare_tunnel.sh

SH | 194 LINES | 713 WORDS | 6.8 KB

```sh
#!/usr/bin/env bash
# ==============================================================================
# CreativeWorkzStudio LLC - OmniCode Platform
# Cloudflare Zero Trust Secure HTTPS Tunnel Connector
# ==============================================================================
# Provides secure edge HTTPS routing to the local OmniCode Studio:
# 1. Quick Tunnel (Free, zero-config ephemeral HTTPS endpoint via trycloudflare.com):
#    Useful for local staging, development, and remote access.
# 2. Named Tunnel (Custom domain via Cloudflare Zero Trust Tunnel Token):
#    Production 24/7 tunnel mapped to your custom domain.
#
# Usage:
#   ./scripts/setup_cloudflare_tunnel.sh            # Auto-detects token from .env or starts quick
#   ./scripts/setup_cloudflare_tunnel.sh --quick    # Starts instant ephemeral quick tunnel
#   ./scripts/setup_cloudflare_tunnel.sh --token <TOKEN> # Starts named tunnel with specified token
#   ./scripts/setup_cloudflare_tunnel.sh --port <PORT>   # Specify local port (default: 8080)
# ==============================================================================

set -e

# Visual formatting constants
BOLD="\033[1m"
GREEN="\033[0;32m"
BLUE="\033[0;34m"
CYAN="\033[0;36m"
YELLOW="\033[1;33m"
RED="\033[0;31m"
NC="\033[0m"

# Canonical directory resolution
SCRIPT_DIR="$(cd -P "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
if [ -f "$SCRIPT_DIR/../../Cargo.toml" ]; then
    BASE_DIR="$(cd "$SCRIPT_DIR/../.." && pwd)"
else
    BASE_DIR="$(cd "$SCRIPT_DIR/.." && pwd)"
fi
ENV_FILE="$BASE_DIR/.env"

PORT="8080"
MODE="auto"
CLI_TOKEN=""

echo -e "${BOLD}${CYAN}================================================================================${NC}"
echo -e "${BOLD}${CYAN}  🌐 OmniCode Cloudflare Tunnel Connector                                       ${NC}"
echo -e "${BOLD}${CYAN}  CreativeWorkzStudio LLC — Edge Security & Gateway                             ${NC}"
echo -e "${BOLD}${CYAN}================================================================================${NC}"

# Ensure cloudflared is installed
ensure_cloudflared() {
    if command -v cloudflared >/dev/null 2>&1; then
        echo -e "${GREEN}[+] cloudflared binary verified: $(cloudflared --version | head -n 1)${NC}"
        return 0
    fi

    echo -e "${BLUE}[*] cloudflared not found in PATH. Checking system architecture...${NC}"
    ARCH="$(uname -m)"
    case "$ARCH" in
        x86_64)
            CLOUDFLARED_URL="https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64"
            ;;
        aarch64|arm64)
            CLOUDFLARED_URL="https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-arm64"
            ;;
        armv7l)
            CLOUDFLARED_URL="https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-arm"
            ;;
        *)
            echo -e "${RED}[-] Unsupported architecture: $ARCH. Please install cloudflared manually.${NC}"
            exit 1
            ;;
    esac

    echo -e "${BLUE}[*] Downloading cloudflared binary for $ARCH...${NC}"
    TMP_BIN="/tmp/cloudflared"
    curl -fsSL "$CLOUDFLARED_URL" -o "$TMP_BIN"
```

```
    chmod +x "$TMP_BIN"

    if [ "$EUID" -eq 0 ]; then
        mv "$TMP_BIN" /usr/local/bin/cloudflared
    elif sudo -n true 2>/dev/null; then
        sudo mv "$TMP_BIN" /usr/local/bin/cloudflared
    else
        mkdir -p "$BASE_DIR/bin"
        mv "$TMP_BIN" "$BASE_DIR/bin/cloudflared"
        export PATH="$BASE_DIR/bin:$PATH"
    fi

    echo -e "${GREEN}[+] cloudflared successfully installed.${NC}"
}

# Parse CLI arguments
while [ $# -gt 0 ]; do
    case "$1" in
        --quick)
            MODE="quick"
            shift
            ;;
        --token)
            MODE="token"
            CLI_TOKEN="$2"
            shift 2
            ;;
        --port)
            PORT="$2"
            shift 2
            ;;
        *)
            shift
            ;;
    esac
done

ensure_cloudflared

# Resolve Token from .env if auto
ENV_TOKEN=""
if [ -f "$ENV_FILE" ]; then
    ENV_TOKEN=$(grep -E '^CLOUDFLARE_TUNNEL_TOKEN=' "$ENV_FILE" 2>/dev/null | cut -d '=' -f 2- | tr -d '"' | tr -d "'" ||
true)
fi

# Fallback: check yugioh-server .env if available
if [ -z "$ENV_TOKEN" ] && [ -f "/home/professorseanex/yugioh-server/.env" ]; then
    ENV_TOKEN=$(grep -E '^CLOUDFLARE_TUNNEL_TOKEN=' "/home/professorseanex/yugioh-server/.env" 2>/dev/null | cut -d '=' -f 2-
| tr -d '"' | tr -d "'" || true)
fi

if [ "$MODE" = "token" ] && [ -n "$CLI_TOKEN" ]; then
    ACTIVE_TOKEN="$CLI_TOKEN"
elif [ "$MODE" = "auto" ] && [ -n "$ENV_TOKEN" ] && [ "$ENV_TOKEN" != "your_cloudflare_tunnel_token_here" ]; then
    ACTIVE_TOKEN="$ENV_TOKEN"
    MODE="token"
else
    MODE="quick"
fi

# Execution
if [ "$MODE" = "token" ]; then
    echo -e "${BLUE}[*] Starting Cloudflare Named Tunnel using configured token...${NC}"
    echo -e "    Local Target  : http://localhost:$PORT (OmniCode Studio & API)"
    echo -e "    Token         : ${ACTIVE_TOKEN:0:15}... (Truncated for security)"
    echo ""
    echo -e "${YELLOW}[!] Tunnel running in foreground. Press Ctrl+C to terminate.${NC}"
    echo ""
    exec cloudflared tunnel --no-autoupdate run --token "$ACTIVE_TOKEN"
else
    echo -e "${BLUE}[*] Starting Cloudflare Quick Tunnel (Free, zero-config HTTPS edge)...${NC}"
    echo -e "    Local Target : http://localhost:$PORT"
    echo ""

    LOG_FILE="/tmp/omnicode_cloudflared_quick.log"
    rm -f "$LOG_FILE"

    cloudflared tunnel --url "http://localhost:$PORT" > "$LOG_FILE" 2>&1 &
    CF_PID=$!

```

```
    cleanup() {
        echo ""
        echo -e "${YELLOW}[*] Shutting down Cloudflare tunnel (PID: $CF_PID)...${NC}"
        kill "$CF_PID" 2>/dev/null || true
        exit 0
    }
    trap cleanup SIGINT SIGTERM EXIT

    echo -e "${BLUE}[*] Provisioning edge tunnel and acquiring public HTTPS URL...${NC}"
    URL=""
    for i in {1..30}; do
        if [ -f "$LOG_FILE" ]; then
            URL=$(grep -o 'https://[-a-zA-Z0-9\.]*\.trycloudflare\.com' "$LOG_FILE" | head -n 1 || true)
            if [ -n "$URL" ]; then
                break
            fi
        fi
        sleep 1
    done

    if [ -z "$URL" ]; then
        echo -e "${RED}[-] Timeout: Failed to obtain public tunnel URL within 30 seconds.${NC}"
        echo -e "    Log output:"
        cat "$LOG_FILE"
        exit 1
    fi

    echo ""
    echo -e "${BOLD}${GREEN}================================================================================${NC}"
    echo -e "${BOLD}${GREEN}  🎉 OMNICODE CLOUDFLARE HTTPS TUNNEL IS LIVE!                                  ${NC}"
    echo -e "${BOLD}${GREEN}================================================================================${NC}"
    echo -e "  Public HTTPS URL:  ${BOLD}${CYAN}${URL}${NC}"
    echo -e "  Local Destination: http://localhost:$PORT"
    echo -e "  Organization:      CreativeWorkzStudio LLC"
    echo ""
    echo -e "${YELLOW}[!] Keep this terminal open. Tunnel will close when stopped.${NC}"
    echo -e "    Press Ctrl+C to terminate."
    wait "$CF_PID"
fi
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/sync_oracle_vm.sh | SH | 133 LINES | 577 WORDS | 5.6 KB

```
#!/usr/bin/env bash
# ================================================================================
# CreativeWorkzStudio LLC - OmniCode Platform
# Oracle Cloud VM Automated Synchronization Script
# ================================================================================
# Synchronizes the local OmniCode repository to the Oracle Cloud VM:
# 1. Connects to Oracle Cloud VM (147.224.147.30 / Host 'oracle') via SSH.
# 2. Synchronizes repository source files to /home/ubuntu/OmniCode using rsync.
# 3. Compiles the optimized release binary with cargo on the remote host.
# 4. Restarts systemd service: omnicode-web.service.
# 5. Queries and reports remote service health.
#
# Usage:
#   ./scripts/sync_oracle_vm.sh [optional commit or sync note]
# ================================================================================

set -e

# ANSI Styling
GREEN="\033[0;32m"
BLUE="\033[0;34m"
CYAN="\033[0;36m"
YELLOW="\033[1;33m"
RED="\033[0;31m"
BOLD="\033[1m"
NC="\033[0m"

SCRIPT_DIR="$(cd -P "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
if [ -f "$SCRIPT_DIR/../../Cargo.toml" ]; then
    BASE_DIR="$(cd "$SCRIPT_DIR/../.." && pwd)"
else
    BASE_DIR="$(cd "$SCRIPT_DIR/.." && pwd)"
fi

# Target Configuration
ORACLE_HOST="${ORACLE_VM_HOST:-147.224.147.30}"
ORACLE_USER="${ORACLE_VM_USER:-ubuntu}"
SSH_KEY="${ORACLE_SSH_KEY:-$HOME/.ssh/oracle_key.pem}"
REMOTE_DIR="/home/ubuntu/OmniCode"

echo -e "${BOLD}${CYAN}================================================================================${NC}"
echo -e "${BOLD}${CYAN}  🌌 CreativeWorkzStudio LLC — OmniCode Platform                              ${NC}"
echo -e "${BOLD}${CYAN}  Oracle Cloud VM Automated Synchronization Gateway                            ${NC}"
echo -e "${BOLD}${CYAN}  Target: ${ORACLE_USER}@${ORACLE_HOST} (${REMOTE_DIR})                        ${NC}"
echo -e "${BOLD}${CYAN}================================================================================${NC}"

# Check SSH Key
if [ ! -f "$SSH_KEY" ]; then
    echo -e "${RED}[!] SSH Private Key not found at: ${SSH_KEY}${NC}"
    echo -e "    Please specify valid key path via ORACLE_SSH_KEY env var."
    exit 1
fi
chmod 600 "$SSH_KEY" 2>/dev/null || true

# Test SSH Connection
echo -e "\n${BLUE}[1/4] Verifying SSH connectivity to Oracle VM...${NC}"
if ! ssh -i "$SSH_KEY" -o ConnectTimeout=8 -o StrictHostKeyChecking=accept-new "${ORACLE_USER}@${ORACLE_HOST}" "echo 'Connected successfully'" >/dev/null 2>&1; then
    echo -e "${RED}[!] Failed to establish SSH connection to ${ORACLE_USER}@${ORACLE_HOST}.${NC}"
    exit 1
fi
echo -e "${GREEN}[+] Remote Oracle VM host reachable.${NC}"

# Ensure Remote Directory Exists
echo -e "\n${BLUE}[2/4] Ensuring remote workspace directory exists...${NC}"
ssh -i "$SSH_KEY" "${ORACLE_USER}@${ORACLE_HOST}" "mkdir -p $REMOTE_DIR"

# Rsync Files to Oracle VM (excluding target/ and .git/ large artifacts for speed)
echo -e "\n${BLUE}[3/4] Synchronizing repository files via rsync...${NC}"
rsync -avz --delete \
    -e "ssh -i $SSH_KEY -o StrictHostKeyChecking=accept-new" \
    --exclude "target/" \
    --exclude ".git/" \
    --exclude "node_modules/" \
    --exclude "*.log" \
```

```
    "$BASE_DIR/" "${ORACLE_USER}@${ORACLE_HOST}:${REMOTE_DIR}/"
echo -e "${GREEN}[+] Repository source synchronization complete.${NC}"

# Ensure release binary exists locally and transfer it
if [ ! -f "$BASE_DIR/target/release/omnicode" ]; then
    echo -e "${BLUE}[*] Compiling local release binary with cargo...${NC}"
    (cd "$BASE_DIR" && cargo build --release)
fi

echo -e "\n${BLUE}[*] Deploying optimized release binary to Oracle VM...${NC}"
ssh -i "$SSH_KEY" "${ORACLE_USER}@${ORACLE_HOST}" "mkdir -p ${REMOTE_DIR}/target/release"
scp -i "$SSH_KEY" "$BASE_DIR/target/release/omnicode" "${ORACLE_USER}@${ORACLE_HOST}:/tmp/omnicode.new"
ssh -i "$SSH_KEY" "${ORACLE_USER}@${ORACLE_HOST}" "mv -f /tmp/omnicode.new ${REMOTE_DIR}/target/release/omnicode && chmod +x ${REMOTE_DIR}/target/release/omnicode"
echo -e "${GREEN}[+] Binary deployment complete.${NC}"

# Remote Service Configuration & Restart
echo -e "\n${BLUE}[4/4] Configuring firewall and restarting omnicode-web.service on Oracle VM...${NC}"
SSH_CMD="ssh -i \"$SSH_KEY\" \"${ORACLE_USER}@${ORACLE_HOST}\""

eval $SSH_CMD << 'EOF'
set -e
GREEN="\033[0;32m"
BLUE="\033[0;34m"
NC="\033[0m"

cd /home/ubuntu/OmniCode

# Install / update systemd service
SERVICE_FILE="infra/systemd/omnicode-web.service"
if [ ! -f "$SERVICE_FILE" ]; then
    SERVICE_FILE="systemd/omnicode-web.service"
fi
if [ -f "$SERVICE_FILE" ]; then
    echo -e "${BLUE}[*] Installing/updating omnicode-web.service from $SERVICE_FILE...${NC}"
    sudo cp "$SERVICE_FILE" /etc/systemd/system/omnicode-web.service
    sudo systemctl daemon-reload
    sudo systemctl enable omnicode-web.service
    sudo systemctl restart omnicode-web.service
fi

# Ensure TCP port 8080 is permitted in iptables
if ! sudo iptables -L INPUT -n | grep -q "dpt:8080"; then
    echo -e "${BLUE}[*] Opening port 8080 in iptables...${NC}"
    sudo iptables -I INPUT 6 -p tcp --dport 8080 -j ACCEPT
    if command -v netfilter-persistent >/dev/null 2>&1; then
        sudo netfilter-persistent save >/dev/null 2>&1 || true
    fi
fi

sleep 2
STATUS=$(sudo systemctl is-active omnicode-web.service || echo "inactive")
echo -e " • omnicode-web.service status: [${STATUS}]"
EOF

echo -e "\n${BOLD}${GREEN}================================================================================${NC}"
echo -e "${BOLD}${GREEN}  ✅ OmniCode Oracle Cloud VM Synchronization Complete!              ${NC}"
echo -e "${BOLD}${GREEN}  CreativeWorkzStudio LLC                                           ${NC}"
echo -e "${BOLD}${GREEN}================================================================================${NC}"
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/dns/README.md MARKDOWN / DOC 71 LINES 380 WORDS 2.9 KB

🌐 OmniCode DNS Zone Infrastructure

Authoritative BIND RFC 1035 DNS zone configuration files for **OmniCode Studio & CreativeWorkzStudio LLC**, designed for 1-click import into Cloudflare DNS.

📁 File Manifest

File	Standard	Description
`omnicodestudio.com.zone`	RFC 1035 / BIND 9	Master DNS zone file compatible with Cloudflare DNS Import
`zone_manager.py`	Python 3	CLI generator, syntax validator, and record inspector

🚀 How to Import into Cloudflare (Step-by-Step)

1. Log into your Cloudflare Dashboard.
2. Select your domain (e.g. `omnicodestudio.com` or your company domain).
3. In the left navigation sidebar, click **DNS → Records**.
4. Scroll to the **Import and Export** card and click **Import**.
5. Select `omnicodestudio.com.zone` .
6. Click **Import**. Cloudflare will parse and create all records automatically.

🗺️ Architectural Routing Map

```
Internet Request
       |
       ├── HTTPS Web Studio (Port 8080) ──→ Cloudflare Tunnel (c346aa27...) ──→ Oracle VM / Local Port 8080
       |     (studio.omnicodestudio.com)
       |
       ├── DuckDNS Dynamic Host ──────────→ omnicodestudio.duckdns.org ──────→ 71.11.99.168 (CWS Gateway)
       |     (duck.omnicodestudio.com)
       |
       └── Yu-Gi-Oh! Game Simulator (7911) → play.omnicodestudio.com ──────→ 147.224.147.30 (Oracle VM)
             (Raw TCP - Grey Cloud Only!)
```

⚙️ Proxy Status Rules (Crucial!)

Record	Type	Target	Cloudflare Proxy Status	Why
`@` , `www`	`CNAME`	`c346aa27...cfargotunnel.com.`	**Proxied (Orange Cloud)**	Free edge SSL/TLS & DDoS mitigation
`studio` , `api`	`CNAME`	`c346aa27...cfargotunnel.com.`	**Proxied (Orange Cloud)**	Direct edge routing to OmniCode
`local` , `gateway`	`A`	`71.11.99.168`	**DNS-Only (Grey Cloud)**	CreativeWorkzStudio LLC WAN IP
`cloud` , `vm`	`A`	`147.224.147.30`	**DNS-Only (Grey Cloud)**	Direct Oracle VM SSH / Administration
`duck` , `edge`	`CNAME`	`omnicodestudio.duckdns.org.`	**DNS-Only (Grey Cloud)**	Dynamic IP synchronization
`play` , `sim`	`A`	`147.224.147.30`	**DNS-Only (Grey Cloud)**	**MUST BE GREY CLOUD**: Raw TCP port 7911

🛠️ CLI Zone Management

```
cd /home/professorseanex/OmniCode

# Validate zone file syntax
python3 dns/zone_manager.py validate
```

```
# Inspect all configured records
python3 dns/zone_manager.py inspect

# Re-render with a custom domain
python3 dns/zone_manager.py render --domain yourcompany.com --out dns/yourcompany.com.zone
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/dns/omnicodestudio.com.zone

ZONE | 49 LINES | 249 WORDS | 2.6 KB

```
; ================================================================================
; BIND RFC 1035 Zone File for omnicodestudio.com
; CreativeWorkzStudio LLC — OmniCode Platform Infrastructure
; Compatible with Cloudflare DNS Zone File Import (DNS -> Records -> Import)
; ================================================================================
$ORIGIN omnicodestudio.com.
$TTL 3600

; ================================================================================
; 1. PRIMARY WEB STUDIO & REST API (HTTP/HTTPS - Port 8080)
; ================================================================================
; Option A: Cloudflare Zero Trust Named Tunnel (Recommended)
; Tunnel ID: c346aa27-b4c1-41ba-815c-f66b2b84243b
; Benefits: Free SSL certificates, zero open firewall ports, DDoS mitigation.
@       IN  CNAME   c346aa27-b4c1-41ba-815c-f66b2b84243b.cfargotunnel.com.
www     IN  CNAME   c346aa27-b4c1-41ba-815c-f66b2b84243b.cfargotunnel.com.
studio  IN  CNAME   c346aa27-b4c1-41ba-815c-f66b2b84243b.cfargotunnel.com.
api     IN  CNAME   c346aa27-b4c1-41ba-815c-f66b2b84243b.cfargotunnel.com.

; Option B: Direct Oracle Cloud Compute 24/7 Instance (IP: 147.224.147.30)
; If routing directly to the Oracle VM rather than Cloudflare Tunnel, use these:
; @       IN  A       147.224.147.30
; www     IN  CNAME   omnicodestudio.com.
cloud   IN  A       147.224.147.30
vm      IN  A       147.224.147.30

; ================================================================================
; 2. CREATIVEWORKZSTUDIO LLC LOCAL GATEWAY & WORKSTATION (IP: 71.11.99.168)
; ================================================================================
; Points to CreativeWorkzStudio LLC primary gateway / local staging engine
local   IN  A       71.11.99.168
gateway IN  A       71.11.99.168

; ================================================================================
; 3. DYNAMIC DNS (DuckDNS) FALLBACK ALIASES
; ================================================================================
; Dynamic CNAME mappings to licensed DuckDNS host (omnicodestudio.duckdns.org)
duck    IN  CNAME   omnicodestudio.duckdns.org.
edge    IN  CNAME   omnicodestudio.duckdns.org.

; ================================================================================
; 4. UNIFIED PLATFORM CROSS-LINKS (Yu-Gi-Oh! Server & Simulator)
; ================================================================================
; Live duel simulator on Oracle VM (Raw TCP Port 7911 / HTTP Port 7922)
; NOTE: In Cloudflare Dashboard, play/sim MUST remain DNS-Only (Grey Cloud) for TCP
cards   IN  CNAME   c346aa27-b4c1-41ba-815c-f66b2b84243b.cfargotunnel.com.
play    IN  A       147.224.147.30
sim     IN  A       147.224.147.30
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/dns/zone_manager.py PY 269 LINES 925 WORDS 10.7 KB

```python
#!/usr/bin/env python3
# ==============================================================================
# BLOCK 1: METADATA BLOCK
# ==============================================================================
"""
Module: dns.zone_manager
Architecture: Systems Engineering & Network Edge Routing (OmniCode Platform)
Organization: CreativeWorkzStudio LLC
Description:
    Authoritative BIND RFC 1035 Zone File Generator, Validator, and Synchronizer.
    Manages DNS routing rules for Cloudflare Zero Trust Tunnels, Oracle Cloud
    Compute VM (147.224.147.30), Local Gateway (71.11.99.168), and DuckDNS endpoints.

Exported CLI & API:
    - python3 dns/zone_manager.py validate [file.zone]
    - python3 dns/zone_manager.py inspect [file.zone]
    - python3 dns/zone_manager.py render [--domain DOMAIN] [--out PATH]
"""

import os
import re
import sys
from dataclasses import dataclass
from typing import Dict, List, Optional, Tuple

_CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
_PROJECT_ROOT = os.path.dirname(_CURRENT_DIR)

DEFAULT_ZONE_FILE_PATH: str = os.path.join(_CURRENT_DIR, "omnicodestudio.com.zone")
ENV_FILE_PATH: str = os.path.join(_PROJECT_ROOT, ".env")


# ==============================================================================
# BLOCK 2: OPENING BLOCK (Data Structures & Models)
# ==============================================================================

@dataclass
class DNSZoneRecord:
    name: str
    record_type: str
    target: str
    ttl: int = 3600
    comment: str = ""

    def to_bind_line(self) -> str:
        base = f"{self.name:<8} IN  {self.record_type:<7} {self.target}"
        if self.comment:
            return f"{base:<50} ; {self.comment}"
        return base


def parse_env_file(path: str) -> Dict[str, str]:
    values = {}
    if not os.path.isfile(path):
        return values
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line or line.startswith("#"):
                continue
            if "=" in line:
                k, v = line.split("=", 1)
                values[k.strip()] = v.strip().strip("\"'")
    return values


# ==============================================================================
# BLOCK 3: BODY BLOCK (Renderer, Validator & Inspector)
# ==============================================================================

def render_zone_file(
    domain: str = "omnicodestudio.com",
    oracle_ip: str = "147.224.147.30",
    local_ip: str = "71.11.99.168",
    duckdns_host: str = "omnicodestudio.duckdns.org",
    tunnel_id: str = "c346aa27-b4c1-41ba-815c-f66b2b84243b",
    ttl: int = 3600,
```

```
) -> str:
    """Renders a standard RFC 1035 / BIND 9 zone file string for Cloudflare import."""
    tunnel_target = f"{tunnel_id}.cfargotunnel.com."
    duck_target = f"{duckdns_host.rstrip('.') }."

    lines = [
        "; ================================================================================",
        f"; BIND RFC 1035 Zone File for {domain}",
        "; CreativeWorkzStudio LLC — OmniCode Platform Infrastructure",
        "; Compatible with Cloudflare DNS Zone File Import (DNS -> Records -> Import)",
        "; ================================================================================",
        f"$ORIGIN {domain.rstrip('.')}.",
        f"$TTL {ttl}",
        "",
        "; ================================================================================",
        "; 1. PRIMARY WEB STUDIO & REST API (HTTP/HTTPS - Port 8080)",
        "; ================================================================================",
        "; Option A: Cloudflare Zero Trust Named Tunnel (Recommended)",
        f"; Tunnel ID: {tunnel_id}",
        DNSZoneRecord("@", "CNAME", tunnel_target, ttl, "Root Apex -> Cloudflare Tunnel").to_bind_line(),
        DNSZoneRecord("www", "CNAME", tunnel_target, ttl, "WWW Subdomain -> Cloudflare Tunnel").to_bind_line(),
        DNSZoneRecord("studio", "CNAME", tunnel_target, ttl, "OmniCode Studio Web Interface").to_bind_line(),
        DNSZoneRecord("api", "CNAME", tunnel_target, ttl, "OmniCode REST API Endpoint").to_bind_line(),
        "",
        "; Option B: Oracle Cloud Compute 24/7 Instance",
        DNSZoneRecord("cloud", "A", oracle_ip, ttl, "Oracle Cloud VM Direct Ingress").to_bind_line(),
        DNSZoneRecord("vm", "A", oracle_ip, ttl, "Oracle Cloud VM Alias").to_bind_line(),
        "",
        "; ================================================================================",
        "; 2. CREATIVEWORKZSTUDIO LLC LOCAL GATEWAY (WORKSTATION IP)",
        "; ================================================================================",
        DNSZoneRecord("local", "A", local_ip, ttl, "CreativeWorkzStudio LLC Workstation WAN").to_bind_line(),
        DNSZoneRecord("gateway", "A", local_ip, ttl, "CreativeWorkzStudio LLC Gateway IP").to_bind_line(),
        "",
        "; ================================================================================",
        "; 3. DYNAMIC DNS (DuckDNS) FALLBACK ALIASES",
        "; ================================================================================",
        DNSZoneRecord("duck", "CNAME", duck_target, ttl, "DuckDNS Dynamic Host Endpoint").to_bind_line(),
        DNSZoneRecord("edge", "CNAME", duck_target, ttl, "DuckDNS Edge Alias").to_bind_line(),
        "",
        "; ================================================================================",
        "; 4. UNIFIED PLATFORM CROSS-LINKS (Yu-Gi-Oh! Server & Simulator)",
        "; ================================================================================",
        DNSZoneRecord("cards", "CNAME", tunnel_target, ttl, "Card Catalog & Lore Web Dashboard").to_bind_line(),
        DNSZoneRecord("play", "A", oracle_ip, ttl, "Live Duel Simulator (Raw TCP 7911 - Grey Cloud)").to_bind_line(),
        DNSZoneRecord("sim", "A", oracle_ip, ttl, "Game Room Manager (Port 7922)").to_bind_line(),
        "",
    ]
    return "\n".join(lines)


def validate_zone_file(file_path: str) -> Tuple[bool, List[str]]:
    """Validates zone file syntax and record definitions."""
    errors = []
    if not os.path.isfile(file_path):
        return False, [f"File not found: {file_path}"]

    with open(file_path, "r", encoding="utf-8") as f:
        content = f.read()

    has_origin = bool(re.search(r"^\$ORIGIN\s+[\w\.\-]+\.", content, re.MULTILINE))
    if not has_origin:
        errors.append("Missing required directive: $ORIGIN <domain>.")

    has_ttl = bool(re.search(r"^\$TTL\s+\d+", content, re.MULTILINE))
    if not has_ttl:
        errors.append("Missing required directive: $TTL <seconds>")

    record_pattern = re.compile(r"^\s*([\@\w\.\-]+)\s+(?:IN\s+)?(A|AAAA|CNAME|TXT|MX|NS)\s+([^\s;]+)", re.MULTILINE)
    matches = record_pattern.findall(content)
    if not matches:
        errors.append("No valid DNS resource records found in file.")

    return len(errors) == 0, errors


def inspect_zone_file(file_path: str):
    """Parses and prints a formatted summary table of the DNS zone records."""
    if not os.path.isfile(file_path):
        print(f"[!] Error: File '{file_path}' does not exist.")
        sys.exit(1)
```

```
    print("=" * 70)
    print(f"  🌐 DNS Zone Inspector: {os.path.basename(file_path)}")
    print("  CreativeWorkzStudio LLC — OmniCode Platform")
    print("=" * 70)

    with open(file_path, "r", encoding="utf-8") as f:
        lines = f.readlines()

    origin = "unknown"
    ttl = "unknown"
    records = []

    record_re = re.compile(r"^\s*([\@\w\.\-]+)\s+(?:IN\s+)?(A|AAAA|CNAME|TXT|MX|NS)\s+([^\s;]+)(?:\s*;\s*(.*))?")

    for line in lines:
        stripped = line.strip()
        if stripped.startswith("$ORIGIN"):
            origin = stripped.split()[1]
        elif stripped.startswith("$TTL"):
            ttl = stripped.split()[1]
        elif stripped and not stripped.startswith(";"):
            m = record_re.match(line)
            if m:
                records.append({
                    "name": m.group(1),
                    "type": m.group(2),
                    "target": m.group(3),
                    "comment": m.group(4) or ""
                })

    print(f"• Origin Domain : {origin}")
    print(f"• Default TTL   : {ttl} seconds")
    print(f"• Record Count  : {len(records)}\n")
    print(f"{'NAME':<10} {'TYPE':<8} {'TARGET':<45} {'NOTE'}")
    print("-" * 75)
    for r in records:
        print(f"{r['name']:<10} {r['type']:<8} {r['target']:<45} {r['comment']}")
    print("-" * 75)


# ============================================================================
# BLOCK 4: CLOSING BLOCK (CLI Dispatcher)
# ============================================================================

def main():
    args = sys.argv[1:]
    command = args[0] if args else "inspect"

    target_file = DEFAULT_ZONE_FILE_PATH
    env = parse_env_file(ENV_FILE_PATH)

    if command == "inspect":
        file_to_inspect = args[1] if len(args) > 1 else target_file
        inspect_zone_file(file_to_inspect)

    elif command == "validate":
        file_to_val = args[1] if len(args) > 1 else target_file
        valid, errors = validate_zone_file(file_to_val)
        if valid:
            print(f"[✓] Zone file '{file_to_val}' is syntactically valid RFC 1035 format.")
            sys.exit(0)
        else:
            print(f"[✗] Zone file validation failed with {len(errors)} error(s):")
            for err in errors:
                print(f"    - {err}")
            sys.exit(1)

    elif command == "render":
        domain = "omnicodestudio.com"
        out_file = target_file

        if "--domain" in args:
            idx = args.index("--domain")
            if idx + 1 < len(args):
                domain = args[idx + 1]

        if "--out" in args:
            idx = args.index("--out")
            if idx + 1 < len(args):
                out_file = args[idx + 1]
```

```
        oracle_ip = env.get("ORACLE_VM_HOST", "147.224.147.30")
        local_ip = env.get("PUBLIC_WAN_IP", "71.11.99.168")
        duckdns = env.get("PUBLIC_DOMAIN", "omnicodestudio.duckdns.org")
        tunnel_id = env.get("CLOUDFLARE_TUNNEL_ID", "c346aa27-b4c1-41ba-815c-f66b2b84243b")

        rendered = render_zone_file(
            domain=domain,
            oracle_ip=oracle_ip,
            local_ip=local_ip,
            duckdns_host=duckdns,
            tunnel_id=tunnel_id,
        )

        with open(out_file, "w", encoding="utf-8") as f:
            f.write(rendered)

        print(f"[✓] Rendered RFC 1035 zone file to: {out_file}")
        inspect_zone_file(out_file)

    else:
        print("Usage:")
        print("  python3 dns/zone_manager.py inspect [path.zone]")
        print("  python3 dns/zone_manager.py validate [path.zone]")
        print("  python3 dns/zone_manager.py render [--domain DOMAIN] [--out PATH]")


if __name__ == "__main__":
    main()
```

scripts/build_notebooklm_pdf.sh SH 82 LINES 393 WORDS 3.6 KB

```
#!/usr/bin/env bash
# ==============================================================================
# BLOCK 1: METADATA BLOCK
# ( OmniCode Google NotebookLM Multi-Volume & Chapter Manual Runner )
# ==============================================================================
# ARCHITECTURAL TOPOLOGY:
#   Orchestrates track-by-track Volume Manuals (Vol 00 - Vol 06) and bounded
#   Chapter PDFs (Ch 00 - Ch 24) for Google NotebookLM, human study manuals,
#   and AI model training. Renders markdown documentation with rich typography
#   and source code with line-numbered verification cards.
#
# SCRIPTURAL WITNESSES:
#   - Exodus 31:18: "written with the finger of God"
#   - Proverbs 24:27: "Prepare thy work without, and make it fit for thyself"
#   - Proverbs 22:28: "Remove not the ancient landmark"
#   - Colossians 2:19: "From whom the whole body fitly joined together"
# ==============================================================================

# ==============================================================================
# BLOCK 2: OPENING BLOCK
# ( Environment Validation, Toolchain Verification & Path Setup )
# ==============================================================================
set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)"
OUTPUT_DIR="${REPO_ROOT}/docs/notebooklm"

echo "==============================================================================="
echo "==> OMNICODE NOTEBOOKLM MULTI-VOLUME MANUAL RUNNER (4-BLOCK RUNNER)"
echo "==============================================================================="

# Validate Python 3
if ! command -v python3 >/dev/null 2>&1; then
    echo "[-] Error: python3 is required but not installed." >&2
    exit 1
fi

# Validate Headless Browser
if ! command -v google-chrome >/dev/null 2>&1 && \
   ! command -v chromium >/dev/null 2>&1; then
    echo "    HTML artifacts created, but PDF conversion may fail." >&2
fi

# Ensure output directory exists
mkdir -p "${OUTPUT_DIR}"

# ==============================================================================
# BLOCK 3: BODY BLOCK
# ( Pipeline Execution & Chapter Generation )
# ==============================================================================
echo "==> Invoking Multi-Volume & Chapter Manual Pipeline..."
python3 "${REPO_ROOT}/scripts/export_notebooklm_pdf.py" "$@"

# ==============================================================================
# BLOCK 4: CLOSING BLOCK
# ( Artifact Verification, Summary & Affirmative Sealing )
# ==============================================================================
echo ""
echo "==============================================================================="
echo "==> MANUAL ARTIFACTS VERIFICATION"
echo "==============================================================================="

VOL_COUNT=$(find "${OUTPUT_DIR}" -maxdepth 1 -name "OmniCode_Vol*.pdf" | wc -l)
CH_COUNT=$(find "${OUTPUT_DIR}" -maxdepth 1 -name "OmniCode_Ch*.pdf" | wc -l)
MASTER_COUNT=$(find "${OUTPUT_DIR}" -maxdepth 1 -name "OmniCode_Master_Omnibus_Manual.pdf" | wc -l)
MD_COUNT=$(find "${OUTPUT_DIR}" -maxdepth 1 -name "*.md" | wc -l)

echo "[+] Total Volume Manual PDFs Generated:  ${VOL_COUNT}"
echo "[+] Total Chapter PDFs Generated:        ${CH_COUNT}"
echo "[+] Total Master Omnibus Manuals:        ${MASTER_COUNT}"
echo "[+] Ingestion Manifests Generated:       ${MD_COUNT}"
echo "[+] Ingestion Manifest Location: ${OUTPUT_DIR}/NOTEBOOKLM_INGESTION_MANIFEST.md"

if [ "${CH_COUNT}" -gt 0 ]; then
    echo "[√] Status: PROCEED (+1) - Multi-Volume documentation ready for ingestion."
    exit 0
```

```
else
    echo "[-] Status: ROLLBACK (-1) - No PDF artifacts generated." >&2
    exit 1
fi
```

scripts/export_notebooklm_pdf.py PY 1515 LINES 5570 WORDS 58.9 KB

```python
#!/usr/bin/env python3
# ==============================================================================
# BLOCK 1: METADATA BLOCK
# ( OmniCode Multi-Volume Repository & Chapter PDF Manual Generator )
# ==============================================================================
# SCRIPTURE ANCHORS & EPISTEMIC WITNESSES:
#   - Exodus 31:18: "written with the finger of God"
#     Inscription of eternal moral truth upon stone.
#   - Proverbs 24:27: "Prepare thy work without, and make it fit for thyself"
#     Systematic modularization, track-by-track volumes, and chapter-based order.
#   - Proverbs 22:28: "Remove not the ancient landmark"
#     Preservation of invariant boundaries across all tracks and manual volumes.
#   - Colossians 2:19: "From whom the whole body fitly joined together"
#     Harmonious integration of systems, subsystems, and documentation manuals.
#
# ARCHITECTURAL STRATEGY:
#   Organizes the entire OmniCode repository into a two-tier publication hierarchy:
#     1. VOLUMES: Track-by-track standalone operational manuals (Vol 00 to Vol 06).
#     2. CHAPTERS: Thematic, bounded subject modules (Ch 00 to Ch 24) optimized
#        for Google NotebookLM (10-35 pages) to prevent token dilution.
#     3. MASTER OMNIBUS: Unified master reference uniting all volumes.
#
# VOLUME MAPPING:
#   - Volume 00: Foundations, Manifest & Ontological Bedrock (Root & Ontological)
#   - Volume 01: Track 0 - Silicon & Bare-Metal Hardware Kernel Manual
#   - Volume 02: Track 1 - Host Engine & Virtual Machine Manual
#   - Volume 03: Track 2 - OmniCode Language & Standard Library Manual
#   - Volume 04: Track 3 - Applications & OmniStudio Interfaces Manual
#   - Volume 05: Track 4 - Variable Storage & Episodic Memory Manual
#   - Volume 06: Track 5 - Infrastructure, Operations & Archives Manual
# ==============================================================================

import os
import sys
import html
import json
import time
import shutil
import argparse
import subprocess
from datetime import datetime
from pathlib import Path

# Optional markdown library for rich rendering of READMEs and design documents
try:
    import markdown
    HAS_MARKDOWN = True
except ImportError:
    HAS_MARKDOWN = False

# ==============================================================================
# BLOCK 2: OPENING BLOCK
# ( Configuration Constants, Volume & Chapter Definitions & CSS Styles )
# ==============================================================================

REPO_ROOT = Path("/home/professorseanex/OmniCode").resolve()
OUTPUT_DIR = REPO_ROOT / "docs" / "notebooklm"

EXCLUDE_DIRS = {
    ".git", "target", ".vscode", ".gemini", "node_modules",
    "__pycache__", ".cargo", "dist", "notebooklm", "scrolls"
}

BINARY_EXTENSIONS = {
    ".pdf", ".img", ".elf", ".efi", ".bin", ".iso", ".png", ".jpg", ".jpeg",
    ".gif", ".ico", ".woff", ".woff2", ".ttf", ".eot", ".tar", ".gz",
    ".zip", ".pyc", ".o", ".a", ".so", ".dylib", ".rlib"
}

VOLUME_DEFINITIONS = [
    {
        "id": "VOL 00",
        "slug": "Vol00_Foundations_and_Manifest",
        "filename": "OmniCode_Vol00_Foundations_and_Manifest.pdf",
        "title": "Volume 00: Foundations, Manifest & Ontological Bedrock",
        "track": "Root & Ontological Foundations",
        "subtitle": "Moral Bedrock, Exodus 20 Decalogue, Manifest & Ontological Principles",
```

```
        "witness": "Exodus 31:18 (KJV) - Written with the finger of God",
        "description": "Foundational bedrock documents, root system manifests, packaging configurations, and ontological boundaries of computation.",
        "chapters": ["CH 00", "CH 01"],
    },
    {
        "id": "VOL 01",
        "slug": "Vol01_Track0_Silicon_Kernel_Manual",
        "filename": "OmniCode_Vol01_Track0_Silicon_Kernel_Manual.pdf",
        "title": "Volume 01: Track 0 - Silicon & Bare-Metal Hardware Kernel Manual",
        "track": "<TRACK 0> : {Silicon & Bare-Metal Kernel}",
        "subtitle": "Bare-Metal Boot, BIOS MBR, UEFI Long Mode, Multiboot & Alternate States",
        "witness": "Job 38:5 (KJV) - Who hath laid the measures thereof",
        "description": "Hardware inception ladder, 16-bit real mode BIOS, 32-bit protected mode CPI-SI substrate, 64-bit native UEFI PE32+, multiboot hypervisor, and safe mode cascades.",
        "chapters": ["CH 02", "CH 03"],
    },
    {
        "id": "VOL 02",
        "slug": "Vol02_Track1_Host_Engine_Manual",
        "filename": "OmniCode_Vol02_Track1_Host_Engine_Manual.pdf",
        "title": "Volume 02: Track 1 - Host Engine & Virtual Machine Manual",
        "track": "<TRACK 1> : {Host Engine & Virtual Machine}",
        "subtitle": "Host Runtime, AST, AI Cortex, Causal Clock, Bytecode Lowering & Diagnostics",
        "witness": "Colossians 2:19 (KJV) - The whole body fitly joined together",
        "description": "Complete implementation of Track 1: Abstract Syntax Tree, process tape ribbon, neural-symbolic AI cortex, runtime evaluator, bytecode assembler, linter, logger, restorator, terminal, and verification suites.",
        "chapters": ["CH 04", "CH 05", "CH 06", "CH 07", "CH 08", "CH 09", "CH 10", "CH 11", "CH 12", "CH 13"],
    },
    {
        "id": "VOL 03",
        "slug": "Vol03_Track2_Language_and_StdLib_Manual",
        "filename": "OmniCode_Vol03_Track2_Language_and_StdLib_Manual.pdf",
        "title": "Volume 03: Track 2 - OmniCode Language & Standard Library Manual",
        "track": "<TRACK 2> : {OmniCode Language & Standard Library}",
        "subtitle": "Grammar, Parts of Speech, Primitive Lexicons & Core Standard Libraries",
        "witness": "1 Corinthians 14:40 (KJV) - Let all things be done decently and in order",
        "description": "Formal language specification: Character, Number, State, Containers, Systems, Verbs, Modifiers, Connectives, and Standard Library modules.",
        "chapters": ["CH 14", "CH 15", "CH 16", "CH 17", "CH 18", "CH 19", "CH 20"],
    },
    {
        "id": "VOL 04",
        "slug": "Vol04_Track3_Applications_Manual",
        "filename": "OmniCode_Vol04_Track3_Applications_Manual.pdf",
        "title": "Volume 04: Track 3 - Applications & OmniStudio Interfaces Manual",
        "track": "<TRACK 3> : {Applications & OmniStudio Interfaces}",
        "subtitle": "Language Bridges, C99/Lua Environments, IDE Extensions & Studio HUD",
        "witness": "Proverbs 24:27 (KJV) - Prepare thy work without, and make it fit for thyself",
        "description": "User-facing applications, language bridges to C and Lua, VSCode editor integrations, and the OmniStudio bare-metal graphical projection HUD.",
        "chapters": ["CH 21"],
    },
    {
        "id": "VOL 05",
        "slug": "Vol05_Track4_Variable_Storage_Manual",
        "filename": "OmniCode_Vol05_Track4_Variable_Storage_Manual.pdf",
        "title": "Volume 05: Track 4 - Variable Storage & Episodic Memory Manual",
        "track": "<TRACK 4> : {Variable Storage & Episodic Memory}",
        "subtitle": "Immutable Bibles, Episodic Scrolls, Neural Weights & Persistent Storage",
        "witness": "Psalm 119:89 (KJV) - For ever, O LORD, thy word is settled in heaven",
        "description": "Data storage layer: immutable scriptural canons, episodic event scrolls, long-term neural memory weights, and variable state persistence.",
        "chapters": ["CH 22"],
    },
    {
        "id": "VOL 06",
        "slug": "Vol06_Track5_Infrastructure_Manual",
        "filename": "OmniCode_Vol06_Track5_Infrastructure_Manual.pdf",
        "title": "Volume 06: Track 5 - Infrastructure, Operations & Archives Manual",
        "track": "<TRACK 5> : {Infrastructure, Operations & Archives}",
        "subtitle": "Systems Architecture, Logic & Algebra, CI/CD, Containerization & Operations",
        "witness": "Proverbs 3:19 (KJV) - The LORD by wisdom hath founded the earth",
        "description": "Infrastructure engineering: logic and algebraic specifications, four-block architectures, autonomous restorative agents, build toolchains, and deployment archives.",
        "chapters": ["CH 23", "CH 24"],
    },
]

CHAPTER_DEFINITIONS = [
    {
```

```
            "id": "CH 00",
            "volume_id": "VOL 00",
            "slug": "Ch00_Decalogue_and_Root_Manifest",
            "filename": "OmniCode_Ch00_Decalogue_and_Root_Manifest.pdf",
            "title": "Chapter 0: Foundations, Root Manifest & Decalogue Bedrock",
            "subtitle": "Moral Bedrock, Exodus 20 Decalogue, Root Manifest & System Configuration",
            "prompt": "How does the Exodus 20 Decalogue govern the AI cortex and immutable constraints of OmniCode?",
        },
        {
            "id": "CH 01",
            "volume_id": "VOL 00",
            "slug": "Ch01_Ontological_Principles_and_Boundaries",
            "filename": "OmniCode_Ch01_Ontological_Principles_and_Boundaries.pdf",
            "title": "Chapter 1: Ontological Principles, Boundaries & Parts of Speech",
            "subtitle": "Ontological Foundations, Asymmetric Boundary Algebra & Parts of Speech Matrix",
            "prompt": "Explain the philosophical and mathematical difference between BottomUp emergence [ } and TopDown
imposition { ].",
        },
        {
            "id": "CH 02",
            "volume_id": "VOL 01",
            "slug": "Ch02_Track0_Bare_Metal_Substrate_and_BIOS",
            "filename": "OmniCode_Ch02_Track0_Bare_Metal_Substrate_and_BIOS.pdf",
            "title": "Chapter 2: Track 0 - Bare-Metal Substrate & BIOS Inception",
            "subtitle": "16-Bit Real Mode MBR, Stage 2 Protected Mode, CPI-SI Substrate & Alternate States",
            "prompt": "Trace how the Genesis Boot sequence establishes protected mode and mounts the CPI-SI substrate.",
        },
        {
            "id": "CH 03",
            "volume_id": "VOL 01",
            "slug": "Ch03_Track0_UEFI_and_Multiboot_Hypervisor",
            "filename": "OmniCode_Ch03_Track0_UEFI_and_Multiboot_Hypervisor.pdf",
            "title": "Chapter 3: Track 0 - UEFI Long Mode & Multiboot Hypervisor",
            "subtitle": "64-Bit Native UEFI PE32+, OVMF Firmware, Direct Hypervisor Handshake & Telemetry",
            "prompt": "How does the native 64-bit UEFI application verify the balanced (+1)+(-1)=0 equilibrium in bare-metal
RAM?",
        },
        {
            "id": "CH 04",
            "volume_id": "VOL 02",
            "slug": "Ch04_Track1_Core_and_Process_Tape_Ribbon",
            "filename": "OmniCode_Ch04_Track1_Core_and_Process_Tape_Ribbon.pdf",
            "title": "Chapter 4: Track 1 - Core Types & Process Tape Ribbon",
            "subtitle": "TapeHead 5-Ws Ribbon, Cell, Word, Reels, Registers & System Entry Points",
            "prompt": "Explain how the TapeHead coordinates Who, What, Where, When, Why, and How across causal time.",
        },
        {
            "id": "CH 05",
            "volume_id": "VOL 02",
            "slug": "Ch05_Track1_Parser_and_AST",
            "filename": "OmniCode_Ch05_Track1_Parser_and_AST.pdf",
            "title": "Chapter 5: Track 1 - Abstract Syntax Tree & Parser",
            "subtitle": "Lexical Tokens, Grammar Rules, AST Entities, Invariant Asides & Span Tracking",
            "prompt": "How does the OmniCode parser preserve first-class intent asides without polluting execution semantics?",
        },
        {
            "id": "CH 06",
            "volume_id": "VOL 02",
            "slug": "Ch06_Track1_AI_Cortex_and_Neural_Engine",
            "filename": "OmniCode_Ch06_Track1_AI_Cortex_and_Neural_Engine.pdf",
            "title": "Chapter 6: Track 1 - AI Cortex & Neural Learning Engine",
            "subtitle": "Cognitive Loop, Synapses, Episodic Learner, Transducer & Non-Hallucinatory Reflection",
            "prompt": "How does the AI Cortex reconcile probabilistic deliberations with immutable biblical truth?",
        },
        {
            "id": "CH 07",
            "volume_id": "VOL 02",
            "slug": "Ch07_Track1_Runtime_and_Evaluator",
            "filename": "OmniCode_Ch07_Track1_Runtime_and_Evaluator.pdf",
            "title": "Chapter 7: Track 1 - Runtime Environment & Evaluator",
            "subtitle": "SVO Verbs, Causal Clock, Monotonic Time, Transactions & Rollback Semantics",
            "prompt": "How does the runtime environment guarantee that causal clock ticks move strictly forward during
rollback?",
        },
        {
            "id": "CH 08",
            "volume_id": "VOL 02",
            "slug": "Ch08_Track1_Bytecode_Assembler",
            "filename": "OmniCode_Ch08_Track1_Bytecode_Assembler.pdf",
            "title": "Chapter 8: Track 1 - Bytecode Assembler & Lowering",
            "subtitle": "Instruction Encodings, Lowering Lexicon Tapes to Machine Words, Baton Passes",
```

```
        "prompt": "Trace the lowering pipeline from high-level process tape rolls to 64-bit machine instruction words.",
    },
    {
        "id": "CH 09",
        "volume_id": "VOL 02",
        "slug": "Ch09_Track1_Diagnostics_Linter",
        "filename": "OmniCode_Ch09_Track1_Diagnostics_Linter.pdf",
        "title": "Chapter 9: Track 1 - Diagnostics & 4-Block Linter Subsystem",
        "subtitle": "Preflight Boundary Audits, 4-Block Slicing, SVO Syntax, Scaffolding & QuadState Gates",
        "prompt": "Why is sensory telemetry logging inside the linter essential for the autonomic DAR self-healing loop?",
    },
    {
        "id": "CH 10",
        "volume_id": "VOL 02",
        "slug": "Ch10_Track1_Diagnostics_Logger_and_Query",
        "filename": "OmniCode_Ch10_Track1_Diagnostics_Logger_and_Query.pdf",
        "title": "Chapter 10: Track 1 - Diagnostics Logger & Query Engine",
        "subtitle": "Universal Bus, Dual-Rail Failpoints, Inscriber Scrolls, Flight Recorder & Query Engine",
        "prompt": "How does the query engine index episodic memory notes to guide automated developer diagnostics?",
    },
    {
        "id": "CH 11",
        "volume_id": "VOL 02",
        "slug": "Ch11_Track1_Restorator_and_Debugger",
        "filename": "OmniCode_Ch11_Track1_Restorator_and_Debugger.pdf",
        "title": "Chapter 11: Track 1 - Restorator & Debugger Autonomic Self-Healing",
        "subtitle": "DAR (Detect, Assess, Restore) Closed Loop, Test Oracles, Patch Synthesis & Auto-Commit",
        "prompt": "Explain how the Restorator consumes failpoint remediations to heal broken source delimiters
autonomously.",
    },
    {
        "id": "CH 12",
        "volume_id": "VOL 02",
        "slug": "Ch12_Track1_Terminal_Studio_and_Hover",
        "filename": "OmniCode_Ch12_Track1_Terminal_Studio_and_Hover.pdf",
        "title": "Chapter 12: Track 1 - Terminal, REPL & Hover Inspection",
        "subtitle": "Interactive REPL, Task Scheduler, OmniStudio HUD & 3-Tier Hover Cards",
        "prompt": "How does the hover engine bridge tape execution rolls back to original .omni source specifications?",
    },
    {
        "id": "CH 13",
        "volume_id": "VOL 02",
        "slug": "Ch13_Track1_Verification_and_Tripwires",
        "filename": "OmniCode_Ch13_Track1_Verification_and_Tripwires.pdf",
        "title": "Chapter 13: Track 1 - Verification & Functional Test Suites",
        "subtitle": "Tripwires, Boot Verification, Assemble Invariants, Functional Tests & Self-Audit",
        "prompt": "How do assemble tripwire tests mathematically prove zero-disk-mutation on dry run invocations?",
    },
    {
        "id": "CH 14",
        "volume_id": "VOL 03",
        "slug": "Ch14_Track2_Lexicon_Character_Primitive",
        "filename": "OmniCode_Ch14_Track2_Lexicon_Character_Primitive.pdf",
        "title": "Chapter 14: Track 2 - Lexicon: Character Primitive Type",
        "subtitle": "Single-Cell Enclosure, Bit Rails (Bit_0..Bit_7), 5-Trit / 8-Bit ASCII Bijective Invariance",
        "prompt": "Explain the bijective mapping of character 'A' (ASCII 65) to balanced ternary trits and bit rails.",
    },
    {
        "id": "CH 15",
        "volume_id": "VOL 03",
        "slug": "Ch15_Track2_Lexicon_Number_Primitive",
        "filename": "OmniCode_Ch15_Track2_Lexicon_Number_Primitive.pdf",
        "title": "Chapter 15: Track 2 - Lexicon: Number Primitive Type",
        "subtitle": "Dual-Rail Polar Planes, 32-Trit Word, 0b Binary Literals & Zero-Sum Balanced ALU",
        "prompt": "How does the dual-rail balanced ternary number representation enable O(1) rail-swap negation?",
    },
    {
        "id": "CH 16",
        "volume_id": "VOL 03",
        "slug": "Ch16_Track2_Lexicon_State_and_Containers",
        "filename": "OmniCode_Ch16_Track2_Lexicon_State_and_Containers.pdf",
        "title": "Chapter 16: Track 2 - Lexicon: State, Containers & Primitive Types",
        "subtitle": "Quad-State T4 Clock, Memory Envelopes, Bounded Sites, Text Records & Types",
        "prompt": "How do IT, BOUND, and TURN govern canonical loop cadence within bounded containers?",
    },
    {
        "id": "CH 17",
        "volume_id": "VOL 03",
        "slug": "Ch17_Track2_Lexicon_Nouns_and_Vessels",
        "filename": "OmniCode_Ch17_Track2_Lexicon_Nouns_and_Vessels.pdf",
        "title": "Chapter 17: Track 2 - Lexicon: Systems, Places & Covenant Vessels",
```

```
        "subtitle": "Operators, Supervisors, Storehouses, Partitions, Physical Places & Stewards",
        "prompt": "Trace how the Covenant Storehouse partitions memory vaults via the v-- operator.",
    },
    {
        "id": "CH 18",
        "volume_id": "VOL 03",
        "slug": "Ch18_Track2_Lexicon_Verbs_and_Operators",
        "filename": "OmniCode_Ch18_Track2_Lexicon_Verbs_and_Operators.pdf",
        "title": "Chapter 18: Track 2 - Lexicon: Verbs, Pronouns & Directional Operators",
        "subtitle": "Directional SVO Verbs, Scale Vectors, Polarity Operators, Ascension & Descension",
        "prompt": "Explain the functional difference between ascension (/-^) and descension (\\-v) across ontological
scales.",
    },
    {
        "id": "CH 19",
        "volume_id": "VOL 03",
        "slug": "Ch19_Track2_Lexicon_Modifiers_and_Connectives",
        "filename": "OmniCode_Ch19_Track2_Lexicon_Modifiers_and_Connectives.pdf",
        "title": "Chapter 19: Track 2 - Modifiers, Conjunctions & Guard Traps",
        "subtitle": "Invariants, Quad-States, Cadence, Pipelines, Conjunctions & Adversative Caveats (!)",
        "prompt": "How does the adversative caveat operator (!) guard ontological truth against semantic drift?",
    },
    {
        "id": "CH 20",
        "volume_id": "VOL 03",
        "slug": "Ch20_Track2_Standard_Library_Modules",
        "filename": "OmniCode_Ch20_Track2_Standard_Library_Modules.pdf",
        "title": "Chapter 20: Track 2 - Standard Library Modules",
        "subtitle": "Core Logic, Vector Mathematics, High-Precision Arithmetic & Memory Management",
        "prompt": "How do the standard library modules export pure algebraic invariants without side effects?",
    },
    {
        "id": "CH 21",
        "volume_id": "VOL 04",
        "slug": "Ch21_Track3_Applications_and_Studio",
        "filename": "OmniCode_Ch21_Track3_Applications_and_Studio.pdf",
        "title": "Chapter 21: Track 3 - Applications, Bridges & OmniStudio",
        "subtitle": "C99 Bare-Metal Bridge, Lua Coroutine Fibers, VSCode Extensions & Studio HUD",
        "prompt": "How does the Lua coroutine bridge execute process tape rolls as non-blocking fiber threads?",
    },
    {
        "id": "CH 22",
        "volume_id": "VOL 05",
        "slug": "Ch22_Track4_Storage_and_Episodic_Scrolls",
        "filename": "OmniCode_Ch22_Track4_Storage_and_Episodic_Scrolls.pdf",
        "title": "Chapter 22: Track 4 - Variable Storage & Episodic Scrolls",
        "subtitle": "Canonical Bibles, Invariant Truth Tables, Episodic Memory Logs & Variable States",
        "prompt": "How does Track 4 preserve immutable scriptural anchors across distributed variable updates?",
    },
    {
        "id": "CH 23",
        "volume_id": "VOL 06",
        "slug": "Ch23_Track5_Architecture_and_Logic",
        "filename": "OmniCode_Ch23_Track5_Architecture_and_Logic.pdf",
        "title": "Chapter 23: Track 5 - Systems Architecture & Balanced Logic",
        "subtitle": "Coordinate Computation, Execution Pipelines, 4-Block Architecture & Quad-State Loops",
        "prompt": "Explain the architectural synergy between the 4-block layout and balanced ternary arithmetic.",
    },
    {
        "id": "CH 24",
        "volume_id": "VOL 06",
        "slug": "Ch24_Track5_Cognitive_Systems_and_Infra",
        "filename": "OmniCode_Ch24_Track5_Cognitive_Systems_and_Infra.pdf",
        "title": "Chapter 24: Track 5 - Cognitive Systems, Infrastructure & CI/CD",
        "subtitle": "Neural-Symbolic Feedback, Autonomous Restorator, Build Scripts & DNS Infrastructure",
        "prompt": "How does the autonomous restorator interface with cross-platform toolchains to certify zero-defect
builds?",
    },
]

CSS_STYLES = """
@page {
    size: letter portrait;
    margin: 1.5cm 1.2cm 1.5cm 1.2cm;
    @top-center {
        content: "OmniCode Framework & AI OS | Official Reference Manual";
        font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
        font-size: 7pt;
        color: #64748b;
    }
    @bottom-left {
```

```
        content: "Witnesses: Exodus 31:18, Proverbs 24:27, Proverbs 22:28, Colossians 2:19";
        font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
        font-size: 7pt;
        color: #64748b;
    }
    @bottom-right {
        content: "Page " counter(page);
        font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
        font-size: 7pt;
        color: #475569;
    }
}

body {
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
    color: #0f172a;
    background: #ffffff;
    line-height: 1.45;
    font-size: 8.5pt;
    margin: 0;
    padding: 0;
}

.page-break {
    page-break-before: always;
    break-before: page;
}

/* Cover Page */
.cover-page {
    min-height: 90vh;
    display: flex;
    flex-direction: column;
    justify-content: center;
    border: 3px double #1e3a8a;
    padding: 35px;
    background: linear-gradient(135deg, #f8fafc 0%, #edf2f7 100%);
    box-sizing: border-box;
}

.cover-volume-badge {
    display: inline-block;
    background: #1e3a8a;
    color: #ffffff;
    font-size: 8pt;
    font-weight: 700;
    letter-spacing: 1px;
    text-transform: uppercase;
    padding: 3px 10px;
    border-radius: 4px;
    margin-bottom: 12px;
}

.cover-title {
    font-size: 20pt;
    font-weight: 800;
    color: #0f172a;
    margin-bottom: 6px;
    line-height: 1.15;
}

.cover-subtitle {
    font-size: 11pt;
    font-weight: 500;
    color: #2563eb;
    margin-bottom: 24px;
    line-height: 1.35;
}

.cover-scripture {
    border-left: 3px solid #f59e0b;
    background: #fffbeb;
    padding: 10px 14px;
    font-style: italic;
    color: #78350f;
    font-size: 8pt;
    margin-bottom: 24px;
    border-radius: 0 4px 4px 0;
}

.cover-metadata-grid {
```

```
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 12px;
    border-top: 1px solid #cbd5e1;
    padding-top: 16px;
    margin-bottom: 20px;
}

.metadata-item {
    display: flex;
    flex-direction: column;
}

.metadata-label {
    font-size: 6.5pt;
    text-transform: uppercase;
    color: #64748b;
    font-weight: 700;
    letter-spacing: 0.5px;
}

.metadata-value {
    font-size: 8.5pt;
    color: #1e293b;
    font-weight: 600;
}

.cover-prompt-box {
    background: #f1f5f9;
    border: 1px solid #94a3b8;
    border-radius: 4px;
    padding: 10px 12px;
    margin-top: 10px;
}

.cover-prompt-label {
    font-size: 7pt;
    font-weight: 700;
    text-transform: uppercase;
    color: #334155;
    margin-bottom: 3px;
}

.cover-prompt-text {
    font-size: 8pt;
    color: #0f172a;
    font-style: italic;
}

/* TOC */
.toc-container {
    margin-top: 20px;
    margin-bottom: 25px;
}

.toc-header {
    font-size: 13pt;
    font-weight: 700;
    color: #1e3a8a;
    border-bottom: 2px solid #1e3a8a;
    padding-bottom: 5px;
    margin-bottom: 12px;
    text-transform: uppercase;
    letter-spacing: 0.5px;
}

.toc-table {
    width: 100%;
    border-collapse: collapse;
    font-size: 7.5pt;
}

.toc-table th {
    background: #f1f5f9;
    color: #334155;
    text-align: left;
    padding: 5px 8px;
    border-bottom: 2px solid #94a3b8;
    text-transform: uppercase;
    font-size: 7pt;
}
```

```
551
552 .toc-table td {
553     padding: 4px 8px;
554     border-bottom: 1px solid #e2e8f0;
555 }
556
557 .toc-table tr:nth-child(even) {
558     background: #f8fafc;
559 }
560
561 /* File Cards */
562 .file-card {
563     margin-top: 14px;
564     margin-bottom: 20px;
565 }
566
567 .file-header-box {
568     background: #0f172a;
569     color: #ffffff;
570     padding: 7px 10px;
571     border-radius: 4px 4px 0 0;
572     display: flex;
573     justify-content: space-between;
574     align-items: center;
575     page-break-after: avoid;
576     break-after: avoid;
577 }
578
579 .file-path {
580     font-family: ui-monospace, SFMono-Regular, "SF Mono", Menlo, Consolas, monospace;
581     font-size: 8pt;
582     font-weight: 700;
583     color: #38bdf8;
584     word-break: break-all;
585 }
586
587 .file-meta-badges {
588     display: flex;
589     gap: 6px;
590 }
591
592 .badge {
593     background: #1e293b;
594     border: 1px solid #334155;
595     padding: 1px 5px;
596     border-radius: 3px;
597     font-size: 6.5pt;
598     color: #cbd5e1;
599     font-family: monospace;
600     text-transform: uppercase;
601 }
602
603 .badge-doc {
604     background: #065f46;
605     border-color: #047857;
606     color: #a7f3d0;
607 }
608
609 .file-content-box {
610     background: #f8fafc;
611     border: 1px solid #cbd5e1;
612     border-top: none;
613     padding: 8px 10px;
614     border-radius: 0 0 4px 4px;
615     overflow-x: auto;
616 }
617
618 /* Code Display */
619 pre.code-block {
620     margin: 0;
621     font-family: ui-monospace, SFMono-Regular, "SF Mono", Menlo, Consolas, monospace;
622     font-size: 6.8pt;
623     line-height: 1.32;
624     color: #0f172a;
625     tab-size: 4;
626     white-space: pre-wrap;
627     word-break: break-word;
628 }
629
630 .line-num {
631     color: #94a3b8;
```

```
    user-select: none;
    display: inline-block;
    width: 32px;
    text-align: right;
    margin-right: 8px;
    border-right: 1px solid #cbd5e1;
    padding-right: 5px;
}

/* Rendered Markdown Typography */
.markdown-doc-container {
    padding: 4px 8px;
    color: #1e293b;
    font-size: 8.2pt;
    line-height: 1.45;
}

.markdown-doc-container h1 {
    font-size: 13pt;
    font-weight: 800;
    color: #0f172a;
    border-bottom: 1.5px solid #cbd5e1;
    padding-bottom: 4px;
    margin-top: 8px;
    margin-bottom: 8px;
}

.markdown-doc-container h2 {
    font-size: 11pt;
    font-weight: 700;
    color: #1e3a8a;
    border-bottom: 1px solid #e2e8f0;
    padding-bottom: 3px;
    margin-top: 10px;
    margin-bottom: 6px;
}

.markdown-doc-container h3 {
    font-size: 9.5pt;
    font-weight: 700;
    color: #0f172a;
    margin-top: 8px;
    margin-bottom: 4px;
}

.markdown-doc-container h4 {
    font-size: 8.5pt;
    font-weight: 600;
    color: #334155;
    margin-top: 6px;
    margin-bottom: 3px;
}

.markdown-doc-container p {
    margin: 5px 0;
}

.markdown-doc-container blockquote {
    border-left: 3.5px solid #2563eb;
    background: #f0f7ff;
    padding: 6px 12px;
    margin: 6px 0;
    font-style: italic;
    color: #1e3a8a;
    border-radius: 0 3px 3px 0;
}

.markdown-doc-container table {
    width: 100%;
    border-collapse: collapse;
    margin: 8px 0;
    font-size: 7.2pt;
}

.markdown-doc-container th {
    background: #f1f5f9;
    color: #0f172a;
    font-weight: 700;
    padding: 4px 7px;
    border: 1px solid #cbd5e1;
    text-align: left;
```

```
}

.markdown-doc-container td {
    padding: 3px 7px;
    border: 1px solid #e2e8f0;
}

.markdown-doc-container tr:nth-child(even) {
    background: #f8fafc;
}

.markdown-doc-container code {
    background: #f1f5f9;
    color: #0f172a;
    padding: 1px 4px;
    border-radius: 3px;
    font-family: ui-monospace, SFMono-Regular, "SF Mono", monospace;
    font-size: 7.5pt;
}

.markdown-doc-container pre {
    background: #0f172a;
    color: #f8fafc;
    padding: 8px 10px;
    border-radius: 4px;
    overflow-x: auto;
    margin: 6px 0;
}

.markdown-doc-container pre code {
    background: transparent;
    color: #f8fafc;
    padding: 0;
    font-size: 6.8pt;
    line-height: 1.35;
}

.markdown-doc-container ul, .markdown-doc-container ol {
    margin: 4px 0;
    padding-left: 20px;
}

.markdown-doc-container hr {
    border: none;
    border-top: 1px solid #cbd5e1;
    margin: 10px 0;
}
"""

def is_binary_file(filepath: Path) -> bool:
    """Detects whether a file is binary via extension or byte inspection."""
    if filepath.suffix.lower() in BINARY_EXTENSIONS:
        return True
    try:
        with open(filepath, "rb") as f:
            chunk = f.read(1024)
            if b"\x00" in chunk:
                return True
    except Exception:
        return True
    return False

# ==============================================================================
# BLOCK 3: BODY BLOCK
# ( Routing, Discovery, Publication Cards, Volumes & Chapter Lowering )
# ==============================================================================

def categorize_file(rel_path: str) -> str:
    """Categorizes any file in the repository into its canonical Chapter ID."""
    p = rel_path.lower()

    # CH 00: Foundations, Root Manifests, Cargo, Decalogue & Environment
    if '/' not in rel_path or any(
        rel_path.startswith(x) for x in ['README', 'manifest', 'Cargo', '.env', '.markdownlint']
    ):
        return "CH 00"

    # CH 01: Ontological Principles and Boundaries (Track 5 / docs / foundations)
    if 'track 5' in p and '01_foundations' in p:
        return "CH 01"

```

```
    # CH 02 & CH 03: Track 0 Silicon & Bare-Metal Kernel
    if 'track 0' in p:
        if any(x in p for x in ['uefi', 'multiboot', 'bootx64']):
            return "CH 03"
        return "CH 02"

    # Track 1: Host Engine & Virtual Machine (CH 04 through CH 13)
    if 'track 1' in p:
        if '/tests/' in p:
            return "CH 13"
        if '/src/parser/' in p or '/src/ast/' in p:
            return "CH 05"
        if '/src/ai/' in p:
            return "CH 06"
        if '/src/runtime/environment/' in p or '/src/runtime/evaluator/' in p:
            return "CH 07"
        if '/src/runtime/assembler/' in p:
            return "CH 08"
        if '/src/diagnostics/linter/' in p:
            return "CH 09"
        if '/src/diagnostics/logger/' in p:
            return "CH 10"
        if '/src/diagnostics/debugger/' in p or '/src/diagnostics/restorator/' in p or 'dar_detect' in p:
            return "CH 11"
        if '/src/terminal/' in p or '/src/cli/' in p or 'hover' in p:
            return "CH 12"
        return "CH 04"  # Default Track 1: Core, Cogs, Ribbon, Lib, Main

    # Track 2: OmniCode Language & Standard Library (CH 14 through CH 20)
    if 'track 2' in p:
        if '01_nouns/ideas/types/character' in p:
            return "CH 14"
        if '01_nouns/ideas/types/number' in p:
            return "CH 15"
        if '01_nouns/ideas/types' in p:
            return "CH 16"
        if '01_nouns' in p:
            return "CH 17"
        if '02_pronouns' in p or '03_verbs' in p or '/operators/' in p:
            return "CH 18"
        if any(x in p for x in [
            '04_adjectives', '05_adverbs', '06_prepositions',
            '07_conjunctions', '08_interjections'
        ]):
            return "CH 19"
        if '/lib/' in p:
            return "CH 20"
        return "CH 16"

    # Track 3: Applications & Studio (CH 21)
    if 'track 3' in p:
        return "CH 21"

    # Track 4: Variable Storage & Bibles (CH 22)
    if 'track 4' in p:
        return "CH 22"

    # Track 5: Infrastructure, Operations & Archives (CH 23 & CH 24)
    if 'track 5' in p or 'scripts' in p:
        if '02_logic_and_algebra' in p or '03_architecture' in p:
            return "CH 23"
        return "CH 24"

    return "CH 00"

def discover_repository_files():
    """Scans and catalogs text, code, documentation, and manifest files."""
    all_files = []

    for root, dirs, files in os.walk(REPO_ROOT):
        dirs[:] = [
            d for d in dirs
            if d not in EXCLUDE_DIRS and not d.startswith(".")
        ]

        rel_root = Path(root).relative_to(REPO_ROOT)
        if any(x in rel_root.parts for x in ["notebooklm", "scrolls"]):
            continue

        top_component = rel_root.parts[0] if rel_root.parts else ""

```

```
        for file in sorted(files):
            if (
                file.startswith(".")
                or file.endswith("~")
                or file.endswith(".tmp")
                or file.endswith(".tmp.html")
            ):
                continue

            full_path = Path(root) / file
            rel_path = full_path.relative_to(REPO_ROOT)
            rel_str = str(rel_path)

            if any(x in rel_path.parts for x in ["notebooklm", "scrolls"]):
                continue

            is_bin = is_binary_file(full_path)
            file_bytes = full_path.stat().st_size
            lines_count = 0
            words_count = 0
            content = ""

            if not is_bin:
                try:
                    with open(full_path, "r", encoding="utf-8", errors="replace") as f:
                        content = f.read()
                        lines_count = content.count("\n") + (1 if content else 0)
                        words_count = len(content.split())
                except Exception as e:
                    content = f"[Error reading file: {e}]"
            else:
                if full_path.suffix.lower() == ".pdf":
                    size_mb = file_bytes / (1024 * 1024)
                    content = f"[Immutable Scripture Reference: {file} ({size_mb:.2f} MB)]"
                else:
                    size_kb = file_bytes / 1024.0
                    content = f"[Binary Artifact: {file} ({size_kb:.1f} KB)]"
                lines_count = 1
                words_count = len(content.split())

            ch_id = categorize_file(rel_str)

            entry = {
                "rel_path": rel_str,
                "full_path": str(full_path),
                "filename": file,
                "extension": full_path.suffix.lower(),
                "top_dir": top_component,
                "chapter_id": ch_id,
                "is_binary": is_bin,
                "lines": lines_count,
                "words": words_count,
                "bytes": file_bytes,
                "content": content,
            }
            all_files.append(entry)

    return all_files

def render_file_card_html(entry):
    """Renders a single file into a publication card (rich Markdown or code block)."""
    rel_path = entry["rel_path"]
    ext = entry["extension"]
    is_bin = entry["is_binary"]
    lines = entry["lines"]
    words = entry["words"]
    size_kb = entry["bytes"] / 1024.0
    content = entry["content"]

    is_markdown = ext in [".md", ".markdown"]

    badge_type = ext.replace(".", "").upper() if ext else "TEXT"
    if is_bin:
        badge_type = "DOC/BINARY"
    elif is_markdown:
        badge_type = "MARKDOWN / DOC"

    badge_class = "badge badge-doc" if is_markdown else "badge"

    out = []
    out.append('<div class="file-card page-break">')
```

```
    out.append('  <div class="file-header-box">')
    out.append(f'    <div class="file-path">{html.escape(rel_path)}</div>')
    out.append('    <div class="file-meta-badges">')
    out.append(f'      <span class="{badge_class}">{badge_type}</span>')
    out.append(f'      <span class="badge">{lines} Lines</span>')
    out.append(f'      <span class="badge">{words} Words</span>')
    out.append(f'      <span class="badge">{size_kb:.1f} KB</span>')
    out.append('    </div>')
    out.append('  </div>')
    out.append('  <div class="file-content-box">')

    if is_bin:
        out.append('    <p style="font-style:italic; color:#475569; margin:8px 0; font-size:8pt;">')
        out.append(f'      {html.escape(content)}')
        out.append('    </p>')
    elif is_markdown and HAS_MARKDOWN:
        # Beautifully rendered Markdown typography for manuals and specifications
        rendered_md = markdown.markdown(
            content,
            extensions=['tables', 'fenced_code', 'toc']
        )
        out.append('    <div class="markdown-doc-container">')
        out.append(rendered_md)
        out.append('    </div>')
    else:
        # Monospace code block with line numbers for source code
        out.append('    <pre class="code-block">')
        raw_lines = content.splitlines()
        for i, l in enumerate(raw_lines, 1):
            escaped = html.escape(l)
            out.append(f'<span class="line-num">{i}</span>{escaped}')
        out.append('</pre>')

    out.append('  </div>')
    out.append('</div>\n')
    return "\n".join(out)

def build_chapter_html(chapter_def, volume_def, files):
    """Assembles a publication-grade HTML document for a single chapter."""
    ch_id = chapter_def["id"]
    ch_title = chapter_def["title"]
    ch_sub = chapter_def["subtitle"]
    ch_prompt = chapter_def["prompt"]
    vol_title = volume_def["title"]
    vol_track = volume_def["track"]
    vol_witness = volume_def["witness"]

    ch_lines = sum(f["lines"] for f in files)
    ch_words = sum(f["words"] for f in files)
    gen_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    html_parts = []
    html_parts.append('<!DOCTYPE html>')
    html_parts.append('<html lang="en">')
    html_parts.append('<head>')
    html_parts.append('  <meta charset="UTF-8">')
    html_parts.append(f'  <title>{html.escape(ch_title)}</title>')
    html_parts.append(f'  <style>{CSS_STYLES}</style>')
    html_parts.append('</head>')
    html_parts.append('<body>')

    # 1. Chapter Cover Page
    html_parts.append('<div class="cover-page">')
    html_parts.append(f'  <div class="cover-volume-badge">{html.escape(vol_title)}</div>')
    html_parts.append(f'  <div class="cover-title">{html.escape(ch_title)}</div>')
    html_parts.append(f'  <div class="cover-subtitle">{html.escape(ch_sub)}</div>')

    html_parts.append('  <div class="cover-scripture">')
    html_parts.append(f'    <strong>Witness:</strong> {html.escape(vol_witness)}')
    html_parts.append('  </div>')

    html_parts.append('  <div class="cover-metadata-grid">')
    html_parts.append('    <div class="metadata-item">')
    html_parts.append('      <span class="metadata-label">System Track</span>')
    html_parts.append(f'      <span class="metadata-value">{html.escape(vol_track)}</span>')
    html_parts.append('    </div>')
    html_parts.append('    <div class="metadata-item">')
    html_parts.append('      <span class="metadata-label">Chapter Identifier</span>')
    html_parts.append(f'      <span class="metadata-value">{html.escape(ch_id)}</span>')
    html_parts.append('    </div>')
    html_parts.append('    <div class="metadata-item">')
```

```
    html_parts.append('      <span class="metadata-label">NotebookLM Word Count</span>')
    html_parts.append(f'      <span class="metadata-value">{ch_words:,} words</span>')
    html_parts.append('    </div>')
    html_parts.append('    <div class="metadata-item">')
    html_parts.append('      <span class="metadata-label">Cataloged Chapter Files</span>')
    html_parts.append(f'      <span class="metadata-value">{len(files)} files ({ch_lines:,} lines)</span>')
    html_parts.append('    </div>')
    html_parts.append('  </div>')

    html_parts.append('  <div class="cover-prompt-box">')
    html_parts.append('    <div class="cover-prompt-label">Recommended NotebookLM Cognitive Query</div>')
    html_parts.append(f'    <div class="cover-prompt-text">"{html.escape(ch_prompt)}"</div>')
    html_parts.append('  </div>')
    html_parts.append('</div>')

    # 2. Table of Contents
    html_parts.append('<div class="toc-container page-break">')
    html_parts.append(f'  <div class="toc-header">Chapter Manifest & Index ({len(files)} files)</div>')
    html_parts.append('  <table class="toc-table">')
    html_parts.append('    <thead>')
    html_parts.append('      <tr>')
    html_parts.append('        <th>#</th>')
    html_parts.append('        <th>File Path</th>')
    html_parts.append('        <th>Type</th>')
    html_parts.append('        <th>Lines</th>')
    html_parts.append('        <th>Words</th>')
    html_parts.append('        <th>Size (KB)</th>')
    html_parts.append('      </tr>')
    html_parts.append('    </thead>')
    html_parts.append('    <tbody>')

    for i, f in enumerate(files, 1):
        ext_clean = f["extension"].replace(".", "").upper() if f["extension"] else "TEXT"
        size_kb = f["bytes"] / 1024.0
        html_parts.append('      <tr>')
        html_parts.append(f'        <td>{i}</td>')
        html_parts.append(f'        <td><strong>{html.escape(f["rel_path"])}</strong></td>')
        html_parts.append(f'        <td>{ext_clean}</td>')
        html_parts.append(f'        <td>{f["lines"]:,}</td>')
        html_parts.append(f'        <td>{f["words"]:,}</td>')
        html_parts.append(f'        <td>{size_kb:.1f}</td>')
        html_parts.append('      </tr>')

    html_parts.append('    </tbody>')
    html_parts.append('  </table>')
    html_parts.append('</div>')

    # 3. Publication File Cards
    for f in files:
        html_parts.append(render_file_card_html(f))

    html_parts.append('</body>')
    html_parts.append('</html>')

    return "\n".join(html_parts)

def build_volume_cover_html(volume_def, chapters, total_files, total_lines, total_words):
    """Assembles a formal volume cover page for uniting multiple chapter PDFs into a Volume Manual."""
    vol_title = volume_def["title"]
    vol_track = volume_def["track"]
    vol_sub = volume_def["subtitle"]
    vol_witness = volume_def["witness"]
    vol_desc = volume_def["description"]

    html_parts = []
    html_parts.append('<!DOCTYPE html>')
    html_parts.append('<html lang="en">')
    html_parts.append('<head>')
    html_parts.append('  <meta charset="UTF-8">')
    html_parts.append(f'  <title>{html.escape(vol_title)}</title>')
    html_parts.append(f'  <style>{CSS_STYLES}</style>')
    html_parts.append('</head>')
    html_parts.append('<body>')

    html_parts.append('<div class="cover-page">')
    html_parts.append(f'  <div class="cover-volume-badge">Official OmniCode System Manual</div>')
    html_parts.append(f'  <div class="cover-title">{html.escape(vol_title)}</div>')
    html_parts.append(f'  <div class="cover-subtitle">{html.escape(vol_sub)}</div>')

    html_parts.append('  <div class="cover-scripture">')
    html_parts.append(f'    <strong>Foundational Scripture Anchor:</strong> {html.escape(vol_witness)}')
```

```
    html_parts.append('  </div>')

    html_parts.append('  <p style="font-size:9pt; color:#334155; line-height:1.5; margin-bottom:20px;">')
    html_parts.append(f'    {html.escape(vol_desc)}')
    html_parts.append('  </p>')

    html_parts.append('  <div class="cover-metadata-grid">')
    html_parts.append('    <div class="metadata-item">')
    html_parts.append('      <span class="metadata-label">System Track</span>')
    html_parts.append(f'      <span class="metadata-value">{html.escape(vol_track)}</span>')
    html_parts.append('    </div>')
    html_parts.append('    <div class="metadata-item">')
    html_parts.append('      <span class="metadata-label">Total Volume Chapters</span>')
    html_parts.append(f'      <span class="metadata-value">{len(chapters)} Chapters</span>')
    html_parts.append('    </div>')
    html_parts.append('    <div class="metadata-item">')
    html_parts.append('      <span class="metadata-label">Cataloged Source Files</span>')
    html_parts.append(f'      <span class="metadata-value">{total_files} files ({total_lines:,} lines)</span>')
    html_parts.append('    </div>')
    html_parts.append('    <div class="metadata-item">')
    html_parts.append('      <span class="metadata-label">Total Word Count</span>')
    html_parts.append(f'      <span class="metadata-value">{total_words:,} words</span>')
    html_parts.append('    </div>')
    html_parts.append('  </div>')

    html_parts.append('  <div class="toc-container">')
    html_parts.append('    <div class="toc-header">Volume Chapters Catalog</div>')
    html_parts.append('    <table class="toc-table">')
    html_parts.append('      <thead>')
    html_parts.append('        <tr>')
    html_parts.append('          <th>Chapter</th>')
    html_parts.append('          <th>Title</th>')
    html_parts.append('          <th>Subsystem Coverage</th>')
    html_parts.append('        </tr>')
    html_parts.append('      </thead>')
    html_parts.append('      <tbody>')
    for ch in chapters:
        html_parts.append('        <tr>')
        html_parts.append(f'          <td><strong>{html.escape(ch["id"])}</strong></td>')
        html_parts.append(f'          <td>{html.escape(ch["title"])}</td>')
        html_parts.append(f'          <td>{html.escape(ch["subtitle"])}</td>')
        html_parts.append('        </tr>')
    html_parts.append('      </tbody>')
    html_parts.append('    </table>')
    html_parts.append('  </div>')

    html_parts.append('</div>')
    html_parts.append('</body>')
    html_parts.append('</html>')
    return "\n".join(html_parts)

def convert_html_to_pdf(html_content, output_pdf_path):
    """Renders HTML directly to PDF using Google Chrome headless print."""
    tmp_html = output_pdf_path.with_suffix(".tmp.html")
    with open(tmp_html, "w", encoding="utf-8") as f:
        f.write(html_content)

    browser_bin = shutil.which("google-chrome") or shutil.which("chromium")
    if not browser_bin:
        raise RuntimeError("No headless Chrome browser found in PATH.")

    cmd = [
        browser_bin,
        "--headless",
        "--disable-gpu",
        "--no-sandbox",
        "--hide-scrollbars",
        f"--print-to-pdf={output_pdf_path}",
        f"file://{tmp_html}"
    ]

    t0 = time.time()
    res = subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE)
    t1 = time.time()

    if tmp_html.exists():
        tmp_html.unlink()

    if res.returncode != 0:
        err = res.stderr.decode('utf-8', errors='ignore')
        raise RuntimeError(f"Chrome PDF generation failed (code {res.returncode}): {err}")
```

```
    pdf_size_kb = output_pdf_path.stat().st_size / 1024.0
    return t1 - t0, pdf_size_kb

def unite_pdfs(input_pdfs, output_pdf_path):
    """Assembles multiple PDF pieces into a unified Master or Volume PDF."""
    t0 = time.time()
    if shutil.which("pdfunite"):
        cmd = ["pdfunite"] + [str(p) for p in input_pdfs] + [str(output_pdf_path)]
        res = subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE)
        if res.returncode == 0:
            t1 = time.time()
            return t1 - t0, output_pdf_path.stat().st_size / 1024.0

    if shutil.which("gs"):
        cmd = [
            "gs", "-dBATCH", "-dNOPAUSE", "-q",
            "-sDEVICE=pdfwrite",
            f"-sOutputFile={output_pdf_path}"
        ] + [str(p) for p in input_pdfs]
        res = subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE)
        if res.returncode == 0:
            t1 = time.time()
            return t1 - t0, output_pdf_path.stat().st_size / 1024.0

    raise RuntimeError("Neither pdfunite nor gs was able to unite PDF volumes.")

# ==============================================================================
# BLOCK 4: CLOSING BLOCK
# ( Orchestration, Volume Assembly, Master Omnibus & Ingestion Manifest )
# ==============================================================================

def main():
    parser = argparse.ArgumentParser(
        description="Export OmniCode repository into Track Volumes and Chapter PDFs."
    )
    parser.add_argument(
        "--output-dir",
        default=str(OUTPUT_DIR),
        help="Target directory for generated PDF files."
    )
    parser.add_argument(
        "--volume",
        default=None,
        help="Optional single volume ID or slug to generate (e.g. 'VOL 02' or 'Track1')."
    )
    parser.add_argument(
        "--chapter",
        default=None,
        help="Optional single chapter ID or slug to generate (e.g. 'CH 09' or 'Linter')."
    )
    parser.add_argument(
        "--no-volumes",
        action="store_true",
        help="Skip assembling track-by-track Volume PDFs."
    )
    parser.add_argument(
        "--no-master",
        action="store_true",
        help="Skip assembling the complete unified Master Omnibus PDF."
    )

    args = parser.parse_args()
    target_dir = Path(args.output_dir).resolve()
    target_dir.mkdir(parents=True, exist_ok=True)

    print("=" * 80)
    print("OMNICODE NOTEBOOKLM MULTI-VOLUME & CHAPTER MANUAL GENERATOR")
    print(f"Timestamp: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
    print(f"Target Directory: {target_dir}")
    print("=" * 80)

    print("\n[1/5] Discovering repository files across all tracks...")
    all_files = discover_repository_files()

    total_files = len(all_files)
    total_lines = sum(f["lines"] for f in all_files)
    total_words = sum(f["words"] for f in all_files)
    total_mb = sum(f["bytes"] for f in all_files) / (1024 * 1024)

    print(f"  • Total Cataloged Files: {total_files}")
```

```
    print(f"  • Total Source Lines:   {total_lines:,}")
    print(f"  • Total Word Count:     {total_words:,} words")
    print(f"  • Total Raw Volume:     {total_mb:.2f} MB")

    # Map files by chapter ID
    files_by_ch = {}
    for f in all_files:
        files_by_ch.setdefault(f["chapter_id"], []).append(f)

    # Map chapters and volumes
    vol_by_id = {v["id"]: v for v in VOLUME_DEFINITIONS}
    ch_by_id = {c["id"]: c for c in CHAPTER_DEFINITIONS}

    # Filter chapters if requested
    active_chapters = CHAPTER_DEFINITIONS
    if args.chapter:
        query = args.chapter.lower()
        active_chapters = [
            c for c in CHAPTER_DEFINITIONS
            if query in c["slug"].lower() or query in c["id"].lower() or query in c["title"].lower()
        ]
        if not active_chapters:
            print(f"[-] Error: No chapter matched query '{args.chapter}'.")
            sys.exit(1)

    if args.volume:
        v_query = args.volume.lower()
        matched_vols = [
            v for v in VOLUME_DEFINITIONS
            if v_query in v["id"].lower() or v_query in v["slug"].lower() or v_query in v["track"].lower()
        ]
        if not matched_vols:
            print(f"[-] Error: No volume matched query '{args.volume}'.")
            sys.exit(1)
        allowed_ch_ids = set()
        for v in matched_vols:
            allowed_ch_ids.update(v["chapters"])
        active_chapters = [c for c in active_chapters if c["id"] in allowed_ch_ids]

    print(f"\n[2/5] Rendering {len(active_chapters)} bounded Chapter PDFs...")
    chapter_manifest = []
    generated_chapter_pdfs = {}

    for idx, c in enumerate(active_chapters, 1):
        c_id = c["id"]
        c_file = c["filename"]
        v_id = c["volume_id"]
        v_def = vol_by_id.get(v_id, VOLUME_DEFINITIONS[0])

        c_files = files_by_ch.get(c_id, [])
        if not c_files:
            continue

        words = sum(f["words"] for f in c_files)
        lines = sum(f["lines"] for f in c_files)
        out_pdf = target_dir / c_file

        print(f"  [{idx:02d}/{len(active_chapters):02d}] {c_file} ({v_id})")
        print(f"       Files: {len(c_files)} | Lines: {lines:,} | Words: {words:,}")

        html_doc = build_chapter_html(c, v_def, c_files)
        duration, size_kb = convert_html_to_pdf(html_doc, out_pdf)
        generated_chapter_pdfs[c_id] = out_pdf

        print(f"       Rendered in {duration:.2f}s | Size: {size_kb:.1f} KB")

        chapter_manifest.append({
            "chapter_id": c_id,
            "volume_id": v_id,
            "volume_title": v_def["title"],
            "slug": c["slug"],
            "filename": c_file,
            "path": str(out_pdf),
            "title": c["title"],
            "subtitle": c["subtitle"],
            "prompt": c["prompt"],
            "file_count": len(c_files),
            "line_count": lines,
            "word_count": words,
            "size_kb": size_kb,
            "render_seconds": round(duration, 2),
```

```
        })

    # Assemble Track-by-Track Volume PDFs
    volume_manifest = []
    if not args.no_volumes and not args.chapter:
        print("\n[3/5] Assembling Track-by-Track Volume Manuals...")
        for v in VOLUME_DEFINITIONS:
            v_id = v["id"]
            v_filename = v["filename"]
            v_title = v["title"]
            out_vol_pdf = target_dir / v_filename

            # Find generated chapter PDFs in this volume
            ch_pdfs_in_vol = []
            vol_chapters_meta = []
            v_files_count = 0
            v_lines_count = 0
            v_words_count = 0

            for ch_id in v["chapters"]:
                if ch_id in generated_chapter_pdfs:
                    ch_pdfs_in_vol.append(generated_chapter_pdfs[ch_id])
                    ch_meta = ch_by_id[ch_id]
                    vol_chapters_meta.append(ch_meta)
                    ch_files = files_by_ch.get(ch_id, [])
                    v_files_count += len(ch_files)
                    v_lines_count += sum(f["lines"] for f in ch_files)
                    v_words_count += sum(f["words"] for f in ch_files)

            if not ch_pdfs_in_vol:
                continue

            print(f"  • Assembling {v_filename} ({len(ch_pdfs_in_vol)} chapters)...")

            # Build and render volume cover PDF
            vol_cover_html = build_volume_cover_html(
                v, vol_chapters_meta, v_files_count, v_lines_count, v_words_count
            )
            vol_cover_pdf = target_dir / f"{v['slug']}_Cover.tmp.pdf"
            convert_html_to_pdf(vol_cover_html, vol_cover_pdf)

            # Unite cover + chapter PDFs
            all_vol_pieces = [vol_cover_pdf] + ch_pdfs_in_vol
            u_sec, u_size_kb = unite_pdfs(all_vol_pieces, out_vol_pdf)

            if vol_cover_pdf.exists():
                vol_cover_pdf.unlink()

            print(f"       United in {u_sec:.2f}s | Size: {u_size_kb/1024.0:.2f} MB")

            volume_manifest.append({
                "volume_id": v_id,
                "title": v_title,
                "track": v["track"],
                "filename": v_filename,
                "path": str(out_vol_pdf),
                "chapter_count": len(ch_pdfs_in_vol),
                "file_count": v_files_count,
                "line_count": v_lines_count,
                "word_count": v_words_count,
                "size_mb": round(u_size_kb / 1024.0, 2),
            })
    else:
        print("\n[3/5] Skipping Volume PDF assembly (--no-volumes or single chapter requested).")

    # Assemble Unified Master Omnibus Manual
    master_manifest_entry = None
    if not args.no_master and not args.chapter and not args.volume:
        print("\n[4/5] Assembling Master Omnibus Manual (All Volumes & Tracks Combined)...")
        master_pdf = target_dir / "OmniCode_Master_Omnibus_Manual.pdf"
        try:
            # Order all chapter PDFs sequentially CH 00 through CH 24
            ordered_chapter_pdfs = [
                generated_chapter_pdfs[c["id"]]
                for c in CHAPTER_DEFINITIONS
                if c["id"] in generated_chapter_pdfs
            ]
            u_sec, u_size_kb = unite_pdfs(ordered_chapter_pdfs, master_pdf)
            print(f"  • Complete Master Omnibus united in {u_sec:.2f}s | Size: {u_size_kb/1024.0:.2f} MB")
            master_manifest_entry = {
                "title": "OmniCode Master Omnibus Manual",
```

```
                "filename": "OmniCode_Master_Omnibus_Manual.pdf",
                "path": str(master_pdf),
                "chapter_count": len(ordered_chapter_pdfs),
                "file_count": total_files,
                "line_count": total_lines,
                "word_count": total_words,
                "size_mb": round(u_size_kb / 1024.0, 2),
            }
        except Exception as e:
            print(f"  [!] Master assembly warning: {e}")
    else:
        print("\n[4/5] Skipping Master Omnibus assembly.")

    # Write Ingestion Manifests
    print("\n[5/5] Writing Ingestion Manifests for Google NotebookLM...")
    manifest_md = target_dir / "NOTEBOOKLM_INGESTION_MANIFEST.md"
    manifest_json = target_dir / "notebooklm_manifest.json"

    full_manifest = {
        "generated_at": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "repository": str(REPO_ROOT),
        "total_files": total_files,
        "total_lines": total_lines,
        "total_words": total_words,
        "master_omnibus": master_manifest_entry,
        "volumes": volume_manifest,
        "chapters": chapter_manifest,
    }

    with open(manifest_json, "w", encoding="utf-8") as f:
        json.dump(full_manifest, f, indent=2)

    with open(manifest_md, "w", encoding="utf-8") as f:
        f.write("# OmniCode: Google NotebookLM Multi-Volume Manual Ingestion Manifest\n\n")
        f.write(f"**Generated:** {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}  \n")
        f.write(f"**Repository:** `{REPO_ROOT}`  \n")
        f.write(f"**Total Cataloged Files:** {total_files} | **Lines:** {total_lines:,} | **Words:** {total_words:,}  \n\n")

        f.write("## Scripture Witnesses & Invariants\n")
        f.write("- **Exodus 31:18**: *\"written with the finger of God\"*\n")
        f.write("- **Proverbs 24:27**: *\"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house.\"*\n")
        f.write("- **Proverbs 22:28**: *\"Remove not the ancient landmark, which thy fathers have set.\"*\n")
        f.write("- **Colossians 2:19**: *\"From whom the whole body fitly joined together and compacted by that which every joint supplieth...\"*\n\n")

        f.write("## Volume Manuals (Track-by-Track Architecture)\n\n")
        f.write("| Volume | Manual Title | System Track | Chapters | Files | Lines | Words | Size |\n")
        f.write("|---|---|---|---|---|---|---|---|\n")
        for v in volume_manifest:
            f.write(f"| **{v['volume_id']}** | `{v['filename']}` | {v['track']} | {v['chapter_count']} | {v['file_count']} | {v['line_count']:,} | {v['word_count']:,} | {v['size_mb']} MB |\n")

        f.write("\n## Chapter Curriculum Map for AI Model Training\n\n")
        f.write("| Ch | Volume | PDF Filename | Files | Lines | Words | Size | Recommended Cognitive Prompt |\n")
        f.write("|---|---|---|---|---|---|---|---|\n")
        for c in chapter_manifest:
            size_str = f"{c['size_kb']/1024.0:.2f} MB"
            f.write(f"| **{c['chapter_id']}** | {c['volume_id']} | `{c['filename']}` | {c['file_count']} | {c['line_count']:,} | {c['word_count']:,} | {size_str} | {c['prompt']} |\n")

        f.write("\n## How to Ingest into Google NotebookLM\n\n")
        f.write("1. Open **[NotebookLM](https://notebooklm.google.com)** with your Google AI account.\n")
        f.write("2. Create a new notebook named **`OmniCode Operating System & Language`**.\n")
        f.write("3. **Recommended Strategy - Modular Chapter Ingestion:** Upload the individual Chapter PDFs (`OmniCode_Ch00_*.pdf` through `OmniCode_Ch24_*.pdf`). Each chapter is bounded (10-35 pages), enabling high-resolution grounding and citation without token dilution.\n")
        f.write("4. **Track Manual Strategy:** Alternatively, upload the Volume Manuals (`OmniCode_Vol00_*.pdf` through `OmniCode_Vol06_*.pdf`) to focus on a specific track.\n")
        f.write("5. **Omnibus Strategy:** Upload `OmniCode_Master_Omnibus_Manual.pdf` for repository-wide synthesis.\n")

    print(f"  • Markdown Ingestion Manifest: {manifest_md}")
    print(f"  • Structured JSON Manifest:    {manifest_json}")

    print("\n" + "=" * 80)
    print("✓ MULTI-VOLUME & CHAPTER MANUAL PIPELINE COMPLETED SUCCESSFULLY")
    print("=" * 80)

if __name__ == "__main__":
    main()
```